

2nd
Edition

Building Decentralized Blockchain Applications

Learn how to use blockchain as the foundation
for Next-Gen Apps



Shahid Shaikh



2nd
Edition

Building Decentralized Blockchain Applications

Learn how to use blockchain as the foundation
for Next-Gen Apps



Shahid Shaikh



Building Decentralized Blockchain Applications

*Learn how to use blockchain as
the foundation for Next-Gen
Apps*

2nd Edition

Shahid Shaikh



www.bpbonline.com

Second Revised and Updated Edition 2025

First Edition 2021

Copyright © BPB Publications, India

ISBN: 978-93-65897-470

All Rights Reserved. No part of this publication may be reproduced, distributed or transmitted in any form or by any means or stored in a database or retrieval system, without the prior written permission of the publisher with the exception to the program listings which may be entered, stored and executed in a computer system, but they can not be reproduced by the means of publication, photocopy, recording, or by any electronic and mechanical means.

LIMITS OF LIABILITY AND DISCLAIMER OF WARRANTY

The information contained in this book is true to correct and the best of author's and publisher's knowledge. The author has made every effort to ensure the accuracy of these publications, but publisher cannot be held responsible for any loss or damage arising from any information in this book.

All trademarks referred to in the book are acknowledged as properties of their respective owners but BPB Publications cannot guarantee the accuracy of this information.

To View Complete
BPB Publications Catalogue
Scan the QR Code:



www.bpbonline.com

Dedicated to

*My parents
and
My wife and daughter*

About the Author

Shahid Shaikh is a software architect with over a decade of experience working with emerging technologies. He is the author of four books and an active contributor to open-source projects. He specializes in blockchain, distributed systems, and software architecture.

Beyond software engineering, he is a vintage bike enthusiast, often riding his classic Royal Enfield to various destinations. He is also deeply interested in carbon-free energy generation, space technologies, and quantum computing.

About the Reviewers

- ❖ **Gowtham Paramasivam** is a distinguished Software Engineer with over 8 years of experience in the IT and blockchain industry. With a strong foundation in software development, he has honed his expertise in designing and implementing innovative solutions across diverse technology stacks. His work spans multiple domains, reflecting his adaptability and commitment to delivering cutting-edge, efficient systems.

Throughout his career, Gowtham has been instrumental in architecting and delivering impactful projects, ranging from decentralized finance (DeFi) platforms to enterprise-grade applications. His expertise extends to smart contract development, distributed systems, and seamless integration of blockchain technologies with traditional infrastructures. As a team leader, he has successfully overseen cross-functional teams, ensuring projects meet business goals while maintaining the highest standards of quality and performance.

- ❖ **Subhasish Goswami** is a blockchain researcher and developer with deep expertise in smart contract development, decentralized applications, and blockchain security. Having worked with the early blockchain team at WazirX and the blockchain research team at Siemens, he currently works with Superform Labs, building multi chain applications, His

research work has contributed to the field of blockchain technology, particularly in areas of oracle design, transaction analysis, and smart contract security.

His expertise spans smart contract development, blockchain architecture, and decentralized systems, making him well-qualified to evaluate technical content in the blockchain domain.

Acknowledgement

I would like to express our sincere gratitude to all those who contributed to the completion of this book.

First and foremost, I extend my heartfelt appreciation to my family and friends for their unwavering support and encouragement throughout this journey. Their love and encouragement have been a constant source of motivation.

I am immensely grateful to BPB Publications for their guidance and expertise in bringing this book to fruition. Their support and assistance were invaluable in navigating the complexities of the publishing process.

I would also like to acknowledge the reviewers, technical experts, and editors who provided valuable feedback and contributed to the refinement of this manuscript. Their insights and suggestions have significantly enhanced the quality of the book.

Last but not least, I want to express my gratitude to the readers who have shown interest in this book. Your support and encouragement have been deeply appreciated.

Thank you to everyone who has played a part in making this book a reality.

Preface

Bitcoin, upon its inception in 2008, took the financial world by storm. It became the first alternative to the debt-based paper currency. Bitcoin is powered by the underlying technology that shocked the technology world. The core technology was named blockchain - immutable proof of records linked together in a chain and distributed across every peer in the world.

This simple yet powerful concept opens a new branch in the computing world. Blockchain is being conceptualized to be used in sectors like supply chains, medical records, and land records.

I ventured into blockchain and cryptocurrency in early 2015. To learn more about it, I joined one of India's largest crypto exchanges and helped them build and scale their infrastructure. I gained treasure knowledge while working on the exchange.

One of the branches of blockchain is the decentralized application. This is the new set of applications that offer alternatives to the traditional client/server model of web development. Using this approach, developers can build next-generation applications with the core blockchain technology.

Decentralized applications are slowly venturing into real-world applications, and this was my motivation to author this book—to provide in-depth information and knowledge

about blockchain and decentralized applications to the readers. This book, "Building Decentralized Blockchain Applications, Second Edition" is your comprehensive guide to unlocking the power of decentralized applications (DApps). Whether you are a seasoned engineer or a curious newcomer, this book equips you with the knowledge and tools to embark on your next DApp development journey.

Chapter 1 discusses blockchain and explains the core technologies that power the blockchain. We will also learn about Bitcoin and how it works in depth.

Chapter 2 looks at the Ethereum and smart contracts in detail. The smart contract is the turing complete programming language to build the application that can be executed on the Ethereum blockchain.

Chapter 3 helps you learn about the core engine of blockchain technology which is the consensus algorithms. You will learn different types of consensus algorithms and their pros and cons. You will also get a complete idea of the challenges faced by each algorithm and the potential solutions.

Chapter 4 focuses on blockchain scaling challenges which is very important for the current ecosystem. The scaling of blockchain is the core problem that almost every platform faces. You will learn about various challenges and solutions related to scaling blockchain platforms.

Chapter 5 helps you learn about blockchain interoperability and how to use them to build decentralized applications. Blockchain interoperability allows different blockchain platforms to communicate and pass data to each other, and this is crucial to building next-generation decentralized applications.

Chapter 6 helps you learn about interplanetary file systems and how to use them to build decentralized applications. IPFS is a ground breaking technology to build next-generation blockchain applications.

Chapter 7 introduces the first decentralized database of this book— OrbitDB. It is a serverless peer-to-peer decentralized database software that runs on top of IPFS. OrbitDB provides easy-to-use APIs to help developers integrate and build applications.

Chapter 8 discusses the BigchainDB—a blockchain-based database. BigchainDB is an asset-focused database that provides all the blockchain features, along with modern database capabilities such as indexing, querying, and processing.

Chapter 9 reviews the Amazon Quantum Ledger Database —QLDB. Amazon QLDB is a cloud-based database solution that offers ledger-based record management.

Chapter 10 introduces the DTube—a decentralized version of YouTube. We will look at how DTube works and its architecture, along with how we can use it to host and share videos.

Chapter 11 takes you through the Ocean Protocol—a decentralized data exchange protocol.

Code Bundle and Coloured Images

Please follow the link to download the **Code Bundle** and the **Coloured Images** of the book:

<https://rebrand.ly/8b2dba>

The code bundle for the book is also hosted on GitHub at **<https://github.com/bpbpublications/Building-Decentralized-Blockchain-Applications-2nd-Edition>**.

In case there's an update to the code, it will be updated on the existing GitHub repository.

We have code bundles from our rich catalogue of books and videos available at **<https://github.com/bpbpublications>**. Check them out!

Errata

We take immense pride in our work at BPB Publications and follow best practices to ensure the accuracy of our content to provide with an indulging reading experience to our subscribers. Our readers are our mirrors, and we use their inputs to reflect and improve upon human errors, if any, that may have occurred during the publishing processes involved. To let us maintain the quality and help us reach out to any readers who might be having difficulties due to any unforeseen errors, please write to us at :

errata@bpbonline.com

Your support, suggestions and feedbacks are highly appreciated by the BPB Publications' Family.

Did you know that BPB offers eBook versions of every book published, with PDF and ePub files available? You can upgrade to the eBook version at www.bpbonline.com and as a print book customer, you are entitled to a discount on the eBook copy. Get in touch with us at :

business@bpbonline.com for more details.

At www.bpbonline.com, you can also read a collection of free technical articles, sign up for a range of free newsletters, and receive exclusive discounts and offers on BPB books and eBooks.

Piracy

If you come across any illegal copies of our works in any form on the internet, we would be grateful if you would provide us with the location address or website name. Please contact us at business@bpbonline.com with a link to the material.

If you are interested in becoming an author

If there is a topic that you have expertise in, and you are interested in either writing or contributing to a book, please visit www.bpbonline.com. We have worked with thousands of developers and tech professionals, just like you, to help them share their insights with the global tech community. You can

make a general application, apply for a specific hot topic that we are recruiting an author for, or submit your own idea.

Reviews

Please leave a review. Once you have read and used this book, why not leave a review on the site that you purchased it from? Potential readers can then see and use your unbiased opinion to make purchase decisions. We at BPB can understand what you think about our products, and our authors can see your feedback on their book. Thank you!

For more information about BPB, please visit www.bpbonline.com.

Join our book's Discord space

Join the book's Discord Workspace for Latest updates, Offers, Tech happenings around the world, New Release and Sessions with the Authors:

<https://discord.bpbonline.com>



Table of Contents

1. Introduction to Blockchain and Decentralized Network

Introduction

Structure

Objectives

Introduction to blockchain

Knowing blockchain

Types of blockchain

Public blockchain

Federated or consortium blockchain

Private blockchain

History of blockchain

Blockchain core components

Node

Transaction

Block

Chain

Miners

Consensus

Proof of Work

Proof of Stake

Byzantine Fault Tolerance

Bitcoin

How Bitcoin works

Double spending problem

Blocks

Merkle tree

Bits

Nonce

Transactions

TXID

Transaction data

Fees

Weight

UTXO

Blockchain and decentralization

Decentralized application

Working of decentralized applications or DApps

DApps protocols

Networking layer

Data definition and naming layer

Data layer

Conclusion

References

2. Ethereum, Smart Contracts, and DApps

Introduction

Structure

Objectives

- Introduction to Ethereum
- Bitcoin and Ethereum
- History of Ethereum
- Components of Ethereum
- Ethereum and DApps
- Ethereum clients
- Ethereum networks
- Installing Geth
- Synchronizing the testnet blockchain
- Installing Ganache
- Infura
- JSON Remote Procedure Call API
- Smart contracts and Solidity language
- Solidity data types
- Contract structure
 - State variables*
 - Functions*
 - Function modifier*
 - Events*
 - Struct*
 - Enum*
- Writing our first Solidity program
- Decentralized Applications using Ethereum
- DApps stack
 - Frontend*
 - Smart contracts*
 - Storage*

IPFS

Swarm Bee

Decentralized message communications protocols

Conclusion

3. Blockchain Consensus Algorithms

Introduction

Structure

Objectives

Proof of Work consensus

Proof of Work challenges and limitations

Proof of Stake algorithm

Delegated Proof of Stake

Fault tolerant algorithms

Practical Byzantine Fault Tolerance consensus algorithm

*Working of Practical Byzantine Fault Tolerance
Algorithm*

Tendermint Core

Setting up Tendermint Core

Limitations of Tendermint Core

Conclusion

4. Blockchain Scaling Challenges

Introduction

Structure

Objectives

Blockchain scalability challenges

Sharding

Advantages of sharding

Layer 2 approach

State channels

Payment channels

Sidechains

Plasma framework

Rollups

Optimistic rollups

Zero-knowledge rollups

Lightning Network

Use cases and applications

Conclusion

5. Blockchain and Smart Contracts Interoperability

Introduction

Structure

Objectives

Inter-Blockchain Communication protocol

Overview of Inter-Blockchain Communication protocol

Technical components of IBC protocol

Use cases and applications of IBC protocol

Implications for the blockchain industry

Cross-chain atomic swaps

Hubs for interoperability in blockchain

Key components of hubs

Relays for blockchain interoperability

Use cases and applications

Security concerns for interoperable blockchain
Conclusion

6. Interplanetary File System

Introduction

Structure

Objectives

Introduction to Interplanetary File System

Working of Interplanetary File System

Content-based addressing

Directed acyclic graph

Distributed hash table

Installing and configuring Interplanetary File System

Interplanetary File System command line

Configuring Interplanetary File System node

Addresses

Bootstrap

Hosting a static website in IPFS

Building a full-stack app using IPFS

IPFS Cluster

libp2p

Conclusion

References

7. OrbitDB, a Peer-to-peer Distributed Database

Introduction

Structure

Objectives

- Introduction to OrbitDB
- Installing OrbitDB
- OrbitDB data models
 - Events*
 - Documents*
 - Key-value*
- Building DApps using OrbitDB
- Technologies
 - Architecture*
- Conclusion

8. BigchainDB

- Introduction
- Structure
- Objectives
- Introduction to BigchainDB
- BigchainDB features
 - Decentralization*
 - Immutability*
 - Owner-controlled assets*
 - High transaction rate*
 - Low latency and fast finality*
 - Indexing and querying structured data*
- BigchainDB use cases
 - Supply chain*
 - Intellectual property rights management*
 - Identity*

Internet of Things

Data governance

Audit trails

Running a BigchainDB node

Conclusion

9. Amazon Quantum Ledger Database

Introduction

Structure

Objectives

Amazon Quantum Ledger Database

Features of Amazon Quantum Ledger Database

Amazon Quantum Ledger Database use cases

Using Amazon Quantum Ledger Database

Amazon Quantum Ledger Database drivers

Conclusion

10. DTube

Introduction

Structure

Objective

Introduction to DTube

Steem blockchain

Features of DTube

DTube architecture

Workings of DTube

Conclusion

11. Ocean Protocol

Introduction
Structure
Objectives
Introduction to Ocean Protocol
Use cases of Ocean Protocol
Workings of Ocean Protocol
Ocean Protocol architecture
Conclusion

Index

CHAPTER 1

Introduction to Blockchain and Decentralized Network

Introduction

In this chapter, we will learn about blockchain as a cryptographically secure chain of records linked together and replicated across several peers around the world. Blockchain works with various underlying technologies such as immutable Merkle tree, hashing, decentralized networking, replication, and so on.

Structure

In this chapter, we will cover the following topics:

- Introduction to blockchain
- Types of blockchain
- History of blockchain
- Blockchain core components

- Consensus
- Bitcoin
- Bits
- Blockchain and decentralization
- Decentralized application
- Working of decentralized applications or DApps
- DApps protocols
- Networking layers
- Data definition and naming layer
- Data layer

Objectives

After reading this chapter, you should be able to understand how a blockchain works and the types of blockchain present. You will also understand decentralization and how a consensus is reached in a decentralized network.

Introduction to blockchain

The Times 03 January 2009 Chancellor on brink of second bailout for banks:



Figure 1.1: Newspaper snippet

On 3 January 2009, *Satoshi Nakamoto* mined the first-ever block of Bitcoin—a peer-to-peer decentralized cryptocurrency—and added the headline of *The Times* as a message in the block.

You might be wondering why this message is the first phrase of the book? This is because the message is marked in history as the day when a revolution in the century-old finance industry began.

In this chapter, we will study, in brief, the history of blockchain and Bitcoin. We will also dive into the secret life of *Satoshi Nakamoto*, the creator of blockchain, and distributed ledger technology and Bitcoin.

We will also learn about the different types of blockchain and their uses. We will look at how Bitcoin works in-depth and how this decentralized network manages the most important thing in history—money.

However first, let us understand the basics of blockchain in brief.

Knowing blockchain

Blockchain is an immutable, peer-to-peer, distributed ledger containing the records in a format called **blocks** that are linked together using cryptography, and hence the name blockchain.

Blockchain is distributed, which means each computer running the program contains a copy of the entire blockchain. It improves the reliability of the data across the network.

Blockchain is peer-to-peer based, and there is no central server required. Anyone with a computer and an internet connection can become a part of the network.

The distributed ledger was built for Bitcoin to handle the money aspect of the system. Ledger is a financial recording system containing the credit and debit of a particular person or account. While banks have a central ledger, blockchain provides distributed ledger spread across thousands of machines.

Blockchain was built by *Satoshi Nakamoto*—a pseudonym for the programmer who built the Bitcoin, the first cryptocurrency ever. Blockchain was built to remove the trust factor from the system and allow computers to send or receive money.

Blockchain is linked using cryptographic functions. Each block in a chain knows the address of the previous block till the genesis block.

Tip: The genesis block is the first block that is not mined but hardcoded in the system. You can check the genesis block code here:

<https://sourceforge.net/p/bitcoin/code/133/tree/trunk/main.cpp#l1613>

Blockchain stores the blocks that consist of transactions. Once the entry is made in the blockchain, it cannot be reversed, so, it is immutable. In the upcoming section, we will learn how the entry of blocks is done in the chain.

The key characteristics of blockchain are as follows:

- **Immutability:** One of the key characteristics of the blockchain is immutability. Once data is submitted in the chain and has been confirmed by 51% of the peers in the network, you can never alter it.
- **Cryptographic:** Blockchain is cryptographically secure and uses asymmetric cryptography, where only an intended party with the key can decrypt and see the data.
- **Decentralization:** Blockchain is decentralized, which means there is no central authority controlling the network. It is controlled by the participants of the network.
- **Provenance:** Blockchain provides the record of ownership. So, it supports true provenance.
- **Anonymity:** Blockchain provides true anonymity to the end user. No user information such as name, IP,

and location is stored in the chain, ensuring zero chances of censorship and tracking.

- **Transparency:** Blockchain provides true anonymity to the end user. No private user information such as name, IP, and location is stored in the chain, ensuring zero chances of censorship and tracking. Blockchain ensures the transparency by maintaining the anonymity of the user information.

Before we proceed further, let us look at the types of blockchain.

Types of blockchain

Blockchain was meant to be a public ledger. However, due to corporations not wanting to share their data with the public, some blockchains are intended to serve corporate entities.

Blockchain, in a nutshell, can be divided into three groups:

- Public blockchain
- Federated (or consortium) blockchain
- Private blockchain

Let us learn about them in brief.

Public blockchain

A public blockchain, as the name suggests, is public, so, anyone can join the blockchain network to read the data and write to it.

The Bitcoin network is among the first few public blockchains in the world. You might be wondering how it is secure if it is completely open to the public.

A public blockchain such as Bitcoin has a consensus mechanism that ensure that the network is secure, and data is valid across every peer connected to the system. The second part is the encryption mechanism used by Bitcoin is asymmetric, so it needs the key of the sender and receiver both to decrypt the data, which makes it secure and fraud-proof.

The consensus mechanisms used by the public blockchain can vary between multiple consensus algorithms such as Proof of Work, **Proof of Stake (PoS)**, etc. We will learn and understand more about the consensus algorithm in the coming chapter.

Federated or consortium blockchain

Federated blockchains are shared blockchain among various federations or entities. In simple words, instead of going public, the blockchain is only shared among a few organizations.

Every read and write operation can be done only using the permissions granted by the authorities. As it has fewer peers, a federated blockchain is fast and efficient as compared to a public blockchain.

A federated blockchain does not mandatorily require any consensus algorithm to handle conflict issues as all the peers in the blockchain network are identified. However, we can use a consensus algorithm in the federated blockchain as well.

Private blockchain

Private blockchain, as the name suggests, is used only by the organization. It is not shared with anyone and is only

used within the network. Private blockchains do not use any consensus to solve conflict issues as it is predefined in the system. Only selected peers can insert a block in the network.

We have studied the blockchain and its types; now, let us understand the history of blockchain.

History of blockchain

The core principles of blockchain—distributed computing, cryptography, decentralization, and consensus algorithms—are not new. They have been around since the 90s and are taught in computer science classes all around the world.

So, one may ask, why distributed ledgers technology and blockchain came into existence in 2009.

Well, to find the answer to this, we must understand how our banking system works. Banks are the central authority that own your money and keeps a record of every transaction you make.

There was a time when paper currency was printed based on the number of natural resources one country possesses, mainly gold and silver. It is unlikely that we find gold and silver mines every day, so the supply was limited, and hence, the value of the currency.

The paper currency you currently hold is printed based on credit without the backing of natural resources such as gold and silver. This move allows federal banks to print as much money as they want.

Remember *Figure 1.1*. That is the time when this economic model failed and when banks closed like small shops. That was the 2008 global recession.

So, where is the problem?

Satoshi thinks it is the central authority of the banks. *Satoshi* had a vision of the currency being supported globally and transferred anywhere without a bank. This vision later became Bitcoin.

However, is Bitcoin the first cryptocurrency ever?

The first work on cryptocurrency can be traced back to 1991 by *Stuart Haber* and *W. Scott Stornetta*. This duo wanted to develop a **cryptographically** secured document timestamp system, where the document cannot be altered once stored in the system.

They used the Merkle tree concept to store and track the documents in the system. The major work that leads to the development of the first Blockchain is the development of hashcash—Proof of Work algorithm.

Hashcash was developed to prevent spams in the computing system. Later on, *Hal Finney* developed a reusable Proof of Work algorithm using hashcash.

Satoshi Nakamoto, in 2008, combined Merkle trees and reusable Proof of Work algorithm with building his blockchain to support the working of a new cryptocurrency.

The first-ever Bitcoin transaction was done on *12 January 2009* by *Satoshi Nakamoto*, where he sent 50 Bitcoins to *Hal Finney*.

Bitcoin transactions have since reached their high to around \$20000, as you can see in this graph:



Figure 1.2: *The rise of Bitcoin pricing*

At the time of writing this book, no one knew who *Satoshi* was—the creator of Bitcoin and blockchain.

Now, let us understand the components of the blockchain system.

Blockchain core components

Blockchain consists of the following core components:

- **Node:** Computer within the system.
- **Transaction:** Actual transaction of value, the smallest component of the blockchain.
- **Block:** It is a data structure where transactions are stored in a Merkle tree format.
- **Chain:** A sequence of blocks in a specific order.
- **Miners:** Specific nodes that perform the transaction and block validation.
- **Consensus:** A mechanism through which networks reach the agreement.

In a nutshell, here is how a Blockchain works in six steps:

1. Node submits the transaction in the network.
2. Miners pick up the transaction and keep it in the memory pool.

3. Miners add the transaction in the block.
4. Miners submit the block and compete in the network to find the hash of the block (if consensus is based on Proof of Work system).
5. Once the proper hash is found, the block is submitted to the network.
6. Every other node verifies the hash, and if 51% of the nodes verify and find the hash to be correct, the network reaches consensus, and the block becomes immutable.

Let us understand each component in brief.

Node

Each participant in the network is called a **node**. A node is a computer running the software required to join the blockchain network.

It does require an internet connection to join and participate in the network. It can be a simple computer, laptop, or a huge data center.

In a public blockchain, anyone can become a node and join the network. The number of nodes is important to make the system more secure and hard to attack.

Transaction

The transaction is the basic building block of the blockchain. It is generated by the node and submitted in the chain, which is later picked up by the miner and added in the block.

The transaction consists of the following information:

- ID (hash)

- Address (hash)
- Receiver (hash)
- **Amount:** Amount to send in crypto
- **Fees:** Fees to send the transaction
- **Signed hash:** Hash generated by the sender

There is little more information depending on the cryptocurrency, but the preceding information is mandatory.

Note: One thing I want to point out here is that there is no user information stored. This is because blockchain promises anonymity, so no information about the sender is stored except their address, which is again a hash.

Block

Block is a data structure that stores a bunch of transactions and submits it to the blockchain for validation. If validation is successful, the block becomes immutable.

The miner node generates blocks. Miner node picks up transactions from the memory pool and generates a block. This block is also called **candidate block**.

Every block consists of the following:

- **Block header:** Contains version, last block, time, and target.
- **Nonce:** Number to consider for hash generation.
- **Transactions:** List of transactions in a Merkle tree format.

Bitcoin uses this block structure. More or less the structure is similar in the other blockchain platforms. You may find

some platforms not using nonce while some uses different nesting of data in the headers and transactions.

Block header consists of critical information such as version number, last block hash, timestamp, target number, and so on.

The block is added in the chain by hashing the block header and hoping that data is less than the target number.

The target number is decided using the difficulty number. Difficulty is a number that decides the time taken by the miner to mine the block. It should be there around 10 minutes on the Bitcoin network.

A nonce is a number that is used by the miner to generate the hash to meet the target number. The nonce is incremented by the miner to meet the required target value.

Once the hash is found, the block is mined and submitted to the chain. Each block is linked with the previous block, forming the chain as seen in the following figure:

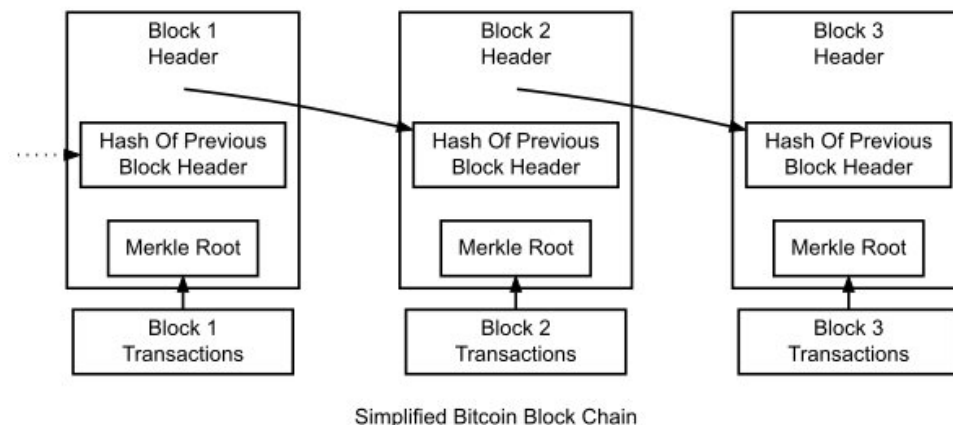


Figure 1.3: Blockchains linked together

Chain

The chain is the blocks linked together using cryptography and shared across every node. Each node maintains the latest copy of the chain, forming a network of strong, reliable, and consistent nodes.

The chain consists of blocks. Blocks contain transactions, as you saw earlier. Each transaction is associated with the account, forming the ledger. Using the chain, one can determine the ledger information of a particular address and verify the transactions.

Since the chain is distributed across the network, there is no chance of fraud. If someone tries to add a false block from their machine, the network will verify and reject the block. To take over the network, one must have 51% control of the node (hashing/computing power of the node) in the network.

It is also called a **51% attack**.

Tip: 51% attack is the distributed attack in which attackers try to control the blockchain network by infecting and controlling hashing power of 51% of the nodes. It is nearly impossible for the network, such as Bitcoin, but it can be implemented for smaller networks.

The chain is shared across the nodes in the network in the following steps:

1. New transactions are broadcasted to all nodes.
2. Each node collects new transactions into a block.
3. Each node works on finding a difficult Proof of Work for its block.
4. When a node finds Proof of Work, it broadcasts the block to all nodes.

5. Nodes accept the block only if all transactions in it are valid and not already spent.
6. Nodes express their acceptance of the block by working on creating the next block in the chain, using the hash of the accepted block as the previous hash.

These steps are mostly the same for most of the proof of the work-based system.

The chain is stored in the binary format on the nodes. Depending on the operating system and chain network, it is stored at the following location by default, and we are taking the Bitcoin chain as an example:

- **Linux:**
`/home/[username]/.bitcoin/blocks/`
- **Windows:**
`C:\Users\[username]\AppData\Roaming\Bitcoin\`
- **Mac:**
`~/Library/Application Support/Bitcoin/`

You can, of course, store it at different locations based on the chain configuration.

Let us proceed to learn about miners.

Miners

Miners are the nodes that perform the process to add a new block in the chain, adding the transactions sent by the user.

Miners do this process by performing a deep level of hash calculations that takes around 10 minutes in the case of Bitcoin.

This calculation costs a lot of computing power, so it is required to pay the miners to run the operation in the surplus profit. Every time a miner performs a successful block insertion in the chain, the system rewards them with an incentive. In the case of Bitcoin, it is 3.125 Bitcoin per successful block at the time of updating this book.

The mining process begins with building a new candidate block by picking up the transactions from the memory pool.

The candidate block contains the block header. Miners try to calculate the hash by passing the block header to the SHA-256 function.

Tip: SHA-256 is the hash function used to generate a hash-based on the input.

The hash returned should be lower than the target value defined by the system at that moment. If not, the miner increments the nonce and calculates the hash again. This process continues until the correct hash is found. Hash is a CPU intensive process, so most miners use advanced CPUs to perform this job.

The mining process is energy-dominant, so it requires lots of electricity to perform the hash calculations. Miners also collect the transaction fee provided in each transaction, plus the mining reward generated by the system.

Let us learn how consensus is being achieved in the blockchain network.

Consensus

The consensus is the set of rules agreed by all parties of the network. There are different types of consensus algorithms we are going to cover the following in this

section. We will cover more about the following consensus algorithms in the coming chapters:

- Proof of Work
- PoS
- **Byzantine Fault Tolerance (BFT)**

Proof of Work is the original consensus mechanism created by *Satoshi Nakamoto* for the Bitcoin cryptocurrency. Let us understand all of them in detail.

Proof of Work

Proof of Work is a consensus mechanism in which one must produce data that is difficult to generate but easy to verify.

Bitcoin uses Proof of Work and asks miners to generate the hash, which is less than the target number. In Bitcoin, the miner must produce a hash that starts with certain 0s. For example, if the target is 10, a miner must produce hash starts with beginning with 0s that are less than 10.

It sounds simple, but it is extremely difficult to generate using the SHA-256 function. Once a hash that satisfies the target is generated, all the others can verify using the target that was used to generate the hash. So, it is easy to verify but hard to generate.

To prevent spams and denial of service attacks, Proof of Work asks miners to invest a ton of money to build an infrastructure to perform mining, and they are rewarded by the system.

This difficulty also ensures that the false transaction does not flood the system because it will take lots of time and money to flood the system with the bad transaction. Instead, one will mine and earn some good money!

Proof of Stake

Proof of Work is a solid consensus mechanism, but it consumes a lot of computing power and energy. The energy cost to generate my blocks is so high that one can power an entire city with that energy.

Proof of Work is faced with scaling issues. To solve these scaling issues, engineers across the world agreed to another form of consensus that is called **PoS**.

In Proof of Work, when the block is about to be mined, every miner in the network tries to generate the hash to earn the reward. Ultimately, only one miner among thousands will get the hash and the reward, which in turn, wastes the efforts of other miners.

In PoS, the miner aka validator is chosen based on various stakes such as random, several blocks mined, and so on. So, the network chooses the miner and asks them to mine the block, giving everyone a fair chance plus saving the effort of other miners.

In PoS, the stake could be money, ranking, or stake itself. When a validator tries to cheat and add a bad or corrupted block in the network, the stake is slashed, or a penalty is charged against the miner.

In general, it can be done randomly. The system picks up random validators from the network and assigns them the block.

The network assigns a formula using which it can decide the miner when a new block is ready to mine.

PoS is simple and saves lots of computational effort. Ethereum, the second most popular cryptocurrency in the

world made a switch to PoS-based consensus algorithm from Proof of Work.

Let us understand another popular consensus mechanism, that is BFT.

Byzantine Fault Tolerance

BFT is a consensus mechanism in a distributed system, wherein consensus is reached even if the nodes in the network fail to respond or respond with incorrect information.

BFT ensures that the system is up and running and performing its function even if the network is under attack or some technical issue occurs.

So far, we have studied the blockchain and its components. In the next section, let us learn about cryptocurrency, which created the revolution with its technology and approach to finance.

Bitcoin

Bitcoin is the world's first peer-to-peer electronic cash system. Bitcoin offers anonymity, transparency, and there is no third-party dependent system to exchange value such as money.

Satoshi Nakamoto invented Bitcoin in 2009, and it has been hailed as one of the most important inventions of the century.

Bitcoin was invented to solve trust-based dependency in the current commerce system. We as people trust banks, governments, and federal reserves to safeguard money and

provide a platform for the common man to trade and exchange using the paper currency.

Due to these trust-based models, the control of the economy is in the hands of a few people, who can use it for malicious purposes.

However, there was no alternative, and you had to go to the bank to handle your financial needs. Then, Bitcoin arrived as a new and better alternative.

To trade in Bitcoin, you do not need any bank or government approval. You can send money to your peers using your computer or mobile. It is as simple as that.

In this section, we will learn in-depth about how Bitcoin works, how each transaction is formed and added to the block, and how it has done more than just solve our money problems.

Note: I highly recommend our readers to print out the Bitcoin whitepaper written by Satoshi and give it a thorough read. You can download the paper from: <https://bitcoin.org/bitcoin.pdf>

How Bitcoin works

Bitcoin is a peer-to-peer electronic payment system created in 2009 by *Satoshi Nakamoto*. Bitcoin allows us to send money to anyone in the world, without the need for a central authority such as banks to issue accounts or process payments.

Bitcoin was created as a solution to the current financial system, where a marginal number of large corporations and banks control the financial world and the processing of transactions across the world.

It centralizes the control of finance and commerce and forces people to trust the banks to act responsibly.

As Satoshi said in the P2P Foundation:

The central bank must be trusted not to debase the currency, but the history of fiat currencies is full of breaches of that trust.

Due to this trust issue, a major financial crisis affected the world at the end of 2007. This crisis inspired *Satoshi* to create Bitcoin.

Bitcoin is a computer program. You can download it precompiled, or you can download the source code from GitHub and compile it yourself. Once you run the Bitcoin program on your computer, you become part of the network; you are a peer!

Once your computer becomes a part of the network, the program will connect to other computers running the same program. Once connected, computers will start sharing a file and try to sync the files with each other, this file is nothing but a Blockchain. Once your computer is up to date with the blockchain files, you can start submitting the transaction.

Note: It may raise a question: does everyone need to run the Bitcoin program to send or receive Bitcoin? Well, no. You can download a Bitcoin wallet and start making Bitcoin transactions. Those who wish to become a peer or miners in the network need to run the Bitcoin program as shown in the following figure:

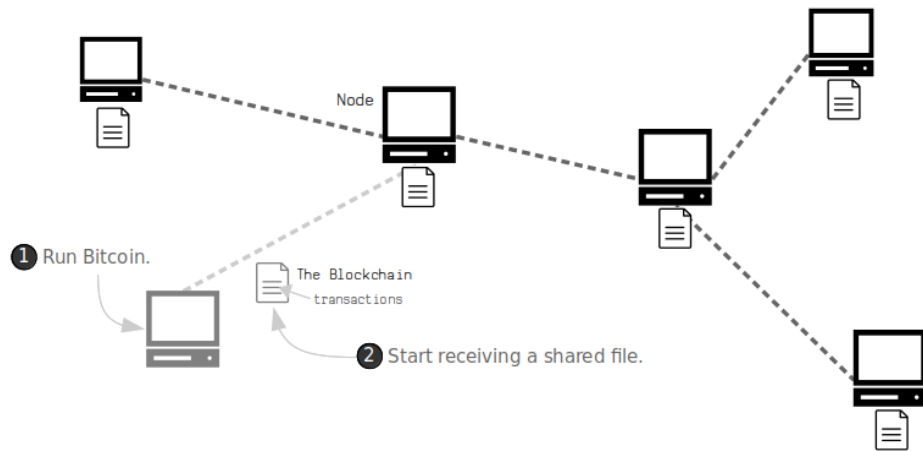


Figure 1.4: Peers in the network

Once your computer is synced with the latest copy of the blockchain, you are good to go. You can now submit transactions, mine the blocks, read the blockchain, and do whatever you wish to do.

When a new transaction is submitted in the blockchain, it is broadcast across the network. Every node then makes a copy of that transaction and adds it to their memory pool.

In the Bitcoin network, at around 10 minutes of interval, one of the miners will mine a new block and broadcast it across the network. Every node will then update its blockchain, and the network will be up to date. This is illustrated in the following figure:

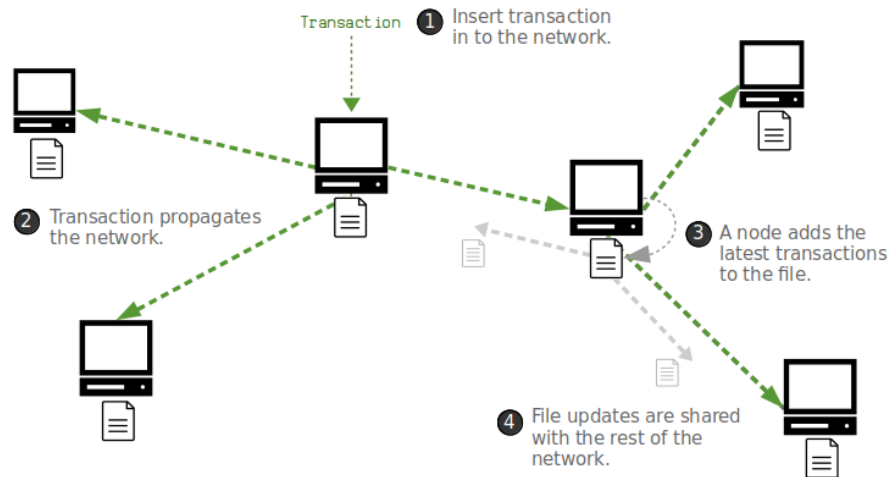


Figure 1.5: Transaction propagation in the network

This peer-to-peer nature of the system creates a trust-free, electronic cash system that runs with the solid consensus mechanism.

Before we proceed ahead with the technical aspects of Bitcoin, let us understand one of the key issues that Bitcoin solved—double spending.

Double spending problem

Bitcoin, a peer-to-peer system, must handle scenarios where two nodes submit the same transaction simultaneously. Without a central authority, it resolves this by employing a robust mechanism to maintain the integrity of its decentralized network. This issue is critical and groundbreaking; let us explore how Bitcoin effectively addresses it.

Bitcoin solved this problem by introducing a memory pool. The memory pool is a buffer that every node must maintain. Memory pool contains the transactions that are later written to the blockchain.

When a node submits a new block, it is broadcast throughout the network.

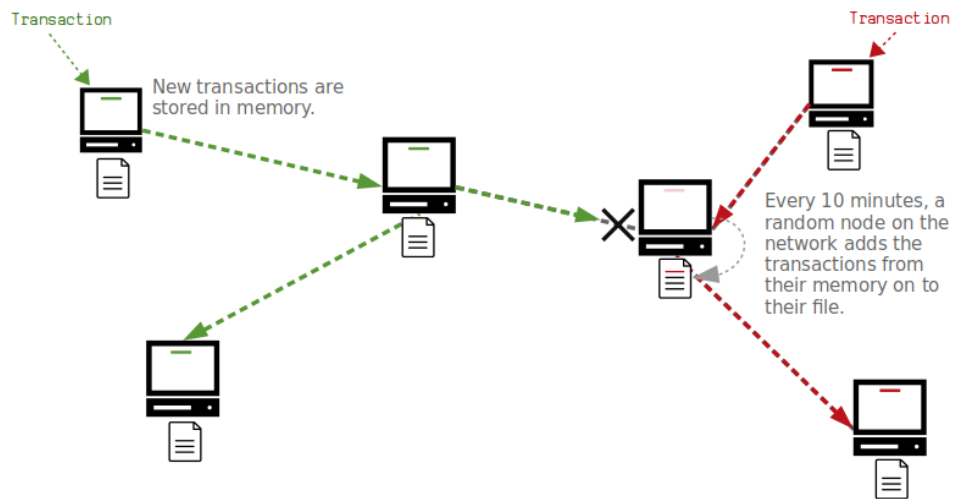


Figure 1.6: Block formation in the network

When a new block is submitted to the network, every node must accept that as a correct transaction and remove any conflicting transactions from their memory pool.

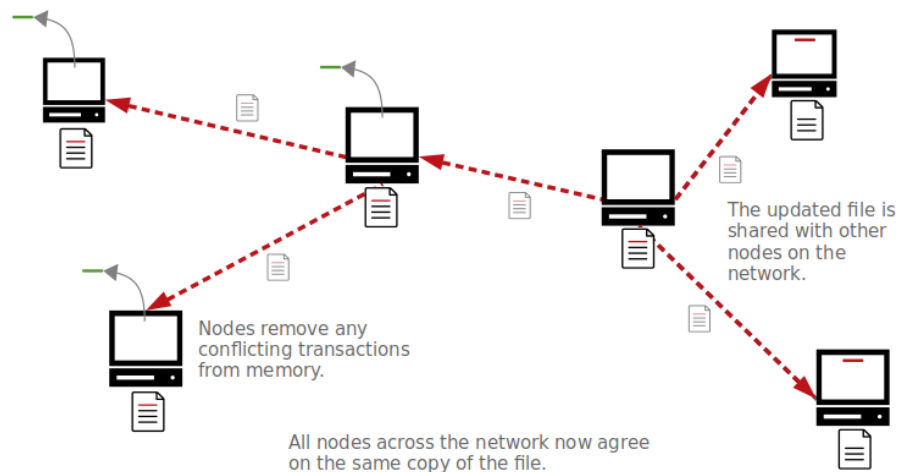


Figure 1.7: Node agreement on the same copy of a file

As a result, no chance of double spending can occur because it will not persist in the network if it is already been added in the block.

This double spending issue is groundbreaking and opens the way for many other projects such as Ripple cryptocurrency, which works with the current finance system but offers a solution to the double spending.

Now, let us look over the technical aspects of Bitcoin.

Blocks

Block is the data structure that holds transactions in an immutable form. Bitcoin stores users transactions in the blocks, and later form a chain to build a blockchain network.

As we discussed in the previous sections, the block is generated by the miner by the process of mining. The mining process requires a consensus algorithm to reach the agreement in the network.

Block consists of the block header, which is used by the miner to generate the hash. Block header consists of the following:

- **Version:** The version of the block.
- **Previous block hash:** Hash of the last identified block, thus forming a chain.
- **Merkle root:** Hash of all the transactions in the block.
- **Time:** Timestamp.
- **Bits:** Hexadecimal version of the target value.
- **Nonce:** It is a 32 bit random whole number that is adjusted by the miners, so that it becomes a valid number to be used to provide the hash of the block that should be less than the target hash.

The first two fields are self-explanatory, and we have already discussed them in the previous sections. Let us learn about the Merkle tree.

Merkle tree

Merkle tree is the data structure in which every leaf node is labeled with the cryptographic hash of the data, and every non-leaf node is labeled with the hash of its child nodes, forming a tree to the root with one hash value.

As you can see in the following figure, Hash 0-0 and Hash 0-1 is the hash value of data block L1 and L2. Later, Hash 0-0 and 0-1 will be hashed together to form Hash 0 and so on until we reach a root hash:

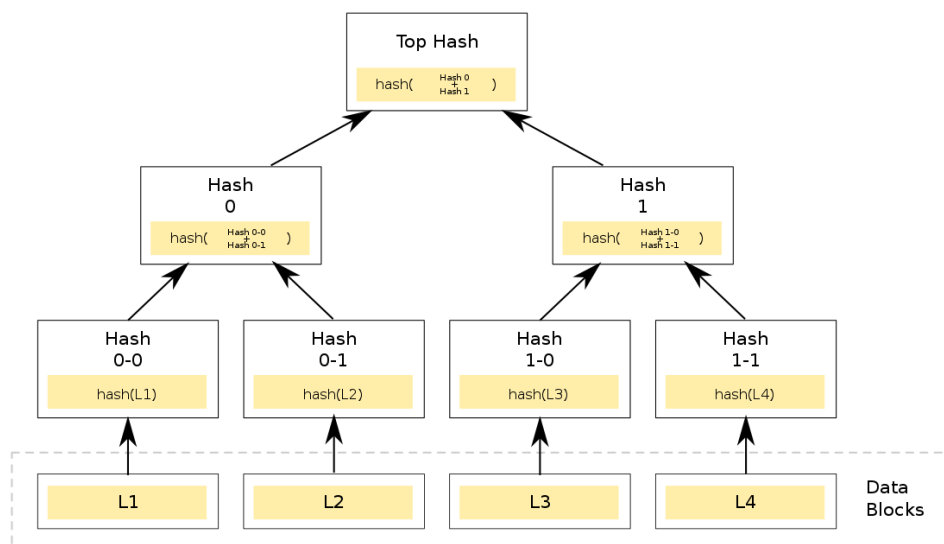


Figure 1.8: Merkle tree

Using a Merkle tree, we can generate a hash of a long series of data in a compact and secure format.

Bitcoin uses the Merkle tree to store transactions in a hashed format:

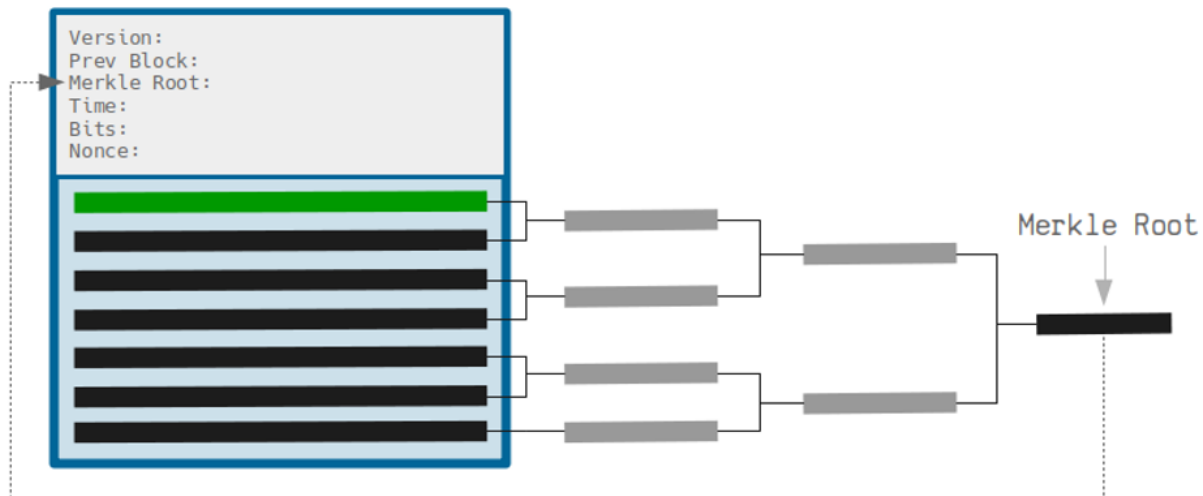


Figure 1.9: Merkle hash and root

As shown in the preceding figure, the Merkle root contains transactions that are hashed together in a way the binary tree is formed.

Merkle tree is formed by hashing the **transaction IDs (TXIDs)** in a pair. TXIDs are used if needed, and we can perform a recheck using the TXID in a tree.

To generate a Merkle tree, we take a pair of TXID and hash them twice using the SHA-256 algorithm. The resulting hash is then hashed with another hashed pair twice to generate a single hash. This goes on till we generate a root hash.

Merkle trees are best suited for applications such as Bitcoin, where the size of the data matters. Due to the use of the Merkle tree, the block size is retained to 1 MB max in Bitcoin.

Tip: Learn in-depth about the Merkle trees; it is one of the most frequently asked questions on Facebook, Amazon, Apple, Netflix, Google (FAANG)

In the next section, we will be looking at the field the miner used to generate the hash of the block that is lower than the target value.

Nonce

The nonce is the field in the block header that is used by the miner to generate a hash. Bitcoin uses Proof of Work consensus algorithms. In Proof of Work, miner must generate a hash that is lower than the target value decided by the network.

Target value starts with several 0s, and it is really difficult to get such a hash using the SHA-256 algorithm on a random basis. So, nonce comes into the picture, miner uses the nonce field to generate a hash, and if it is not lower than the target value, miner increments the nonce and tries again.

When the lucky number or nonce is found, a hash is generated. Miner announces the block in the network, and every other node verifies and makes a copy of it in their blockchain.

So, nonce plays a crucial role in Bitcoin mining.

We have studied about the block and important block header fields, so let us move ahead and learn about Bitcoin transactions.

Transactions

When someone tries to transfer Bitcoin from one account to another, a new transaction is formed and added to the block by the miner.

The transaction contains data that is required for proper Bitcoin transfer. Transaction data consists of the amount that the user wants to send, account information such as from and to addresses, transaction information required for the network such as TXIDs, and so on.

Transaction in Bitcoin contains the following information:

- The TXID is also referred widely as TXID.
- Transaction data such as input, output, and so on.
- Fees
- Weight
- **Unspent transaction output (UTXO)**

Let us learn about each one of them in brief.

TXID

TXID is a 32 bytes hexadecimal number that is referred to as TXID and used to identify the transaction in Bitcoin.

TXID is generated by hashing the transaction data twice using the SHA-256 hashing algorithm.

Satoshi Nakamoto sent the first-ever Bitcoin transaction to *Hal Finney*, and the ID of the transaction is this:

**f4184fc596403b9d638783cf57adfe4c75c605f6356fbc
91338530e9831e9e16**

You can still view this transaction in the blockchain explorer:

<https://www.blockchain.com/btc/tx/f4184fc596403b9d638783cf57adfe4c75c605f6356fbc91338530e9831e9e16>

BLOCKCHAIN.COM

ProductsDataExplorerQ

LoginSign Up

Details

Hash	f4184fc596403b9d638783cf57adfe4c75c605f6356fbc91338530e9831e9e16
Status	Confirmed
Received Time	2009-01-12 09:00
Size	275 bytes
Weight	1,100
Included in Block	170
Confirmations	617,457
Total Input	50.00000000 BTC
Total Output	50.00000000 BTC
Fees	0.00000000 BTC
Fee per byte	0.000 sat/B
Fee per weight unit	0.000 sat/WU
Value when transacted	\$0.00

- ADVERTISEMENT -

Figure 1.10: Blockchain explorer

Let us look over the transaction data required to form transaction payload.

Transaction data

Transaction data contains the information required to form input and output. In Bitcoin, when you send someone a few coins, you unlock a few outputs that are required and lock them again with the keys of the receiver so that only they can lock it.

This may sound confusing, but you will get clarity in the coming paragraph.

Let us take an example:

Imagine you have five Bitcoins, and you want to send three Bitcoins to your friend. Here, the five Bitcoins you hold is your output, and it is split into two outputs. One is worth three Bitcoins and another worth two Bitcoins. The first output will then be locked with the receiver keys so that

they can unlock it. The second output is your money, so it is again locked with your credentials for future uses.

In a nutshell, when a new transaction is formed, the whole amount is included in the transaction payload. Whatever remains is being sent back to you as an output.

So, this approach of graph-based input and output structure provides a history of ownership of Bitcoin.

Fees

Every transaction in the Bitcoin pays some amount of money in the form of Bitcoin to the miner. The remainder of the transaction output is also associated with the fees and paid to the miner for the work they do.

Generally, by adding lucrative fees, there is a high chance that a miner will pick your transaction from the memory pool to add into the next block. However, this is completely optional. If you do not pay the fee at that moment, it may take some time, but your transaction will be mined by some miners scattered across the world.

Here is the fee associated with the transaction. You can also check the miner award, which is 3.125 Bitcoins at the time of updating this book:

UTXO when a new transaction is formed and a new output is generated, the one which is spent or sent to other users cannot be spent again. However, the outputs left or not spent can be used in future transactions.

These unspent outputs are called **UTXOs**.

UTXO is used to calculate the balance of a particular user and verify whether they have the amount to spend. UTXOs are a key part of forming new transactions and preventing double spending.

These are the key factors that are involved in the Bitcoin transaction.

We hope you understood the concepts and the reasoning behind the technical changes done to make Bitcoin and blockchain what they are now. In the next section, we will address one of the unique approaches to build applications that are forked from the Blockchain technology—**decentralized applications (DApps)**.

Blockchain and decentralization

Cryptocurrency is a small subset of the blockchain, like the email of the internet.

As we studied in the previous sections, blockchain is a peer-to-peer consensus-based network of computers. Each computer communicates with each other and exchanges data.

As there is no central authority involved in the working of blockchain, this approach is called the **decentralized approach**.

After the huge success of blockchain technology in supporting the most crucial and important factor of human

beings—money—engineers and technology gurus started wondering about the usage of Blockchain in other aspects of computing, such as e-commerce, hospitals, property holdings, and others.

The blockchain that is built to support Bitcoin cannot support such a wide range of data due to various reasons. Hence, it is best to build a new blockchain-based system using the core technology behind the blockchain built by *Satoshi*: Decentralization, peer-to-peer, consensus, anonymous.

This movement is also referred to as decentralized Web or Web 3.0, as most of these technologies will first be implemented on the internet.

In this section, we will learn about the following:

- DApps
- How does it work
- DApps technologies

Decentralized application

Generally, applications work on client and server models, so there is a central authority that serves those clients. As opposed to that, DApps run on a peer-to-peer-based network.

DApps are already present in the market, such as torrent and Tor network. The key change here is the data storage on the blockchain network, which, in turn, makes it immutable, fraud-proof, and anonymous.

The DApps makes communication with the blockchain deal with data. Rest everything should be the same, such as the user interface, development approach, and so on.

The frontend of the DApps should look and work like a traditional application. The business logic, however, involves cryptography and blockchain to deal with data generated by the application.

This is how a traditional application stack looks:

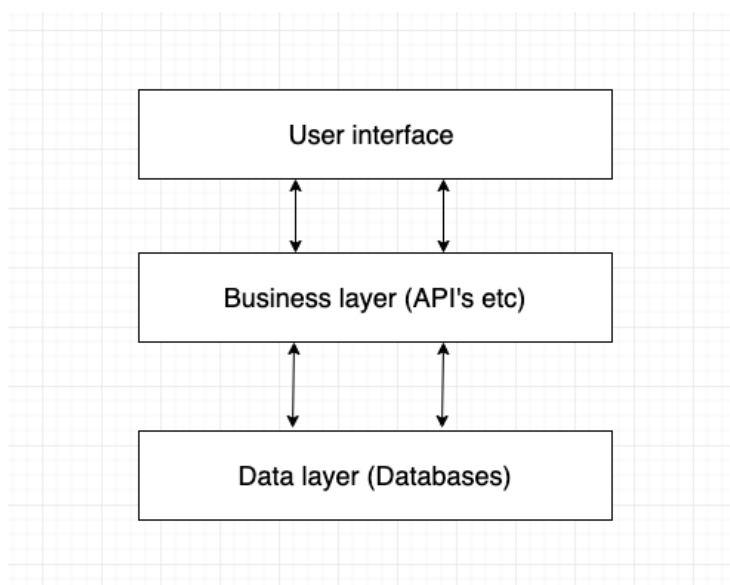


Figure 1.12: *Software building layers*

Here is how a DApps stack looks like:

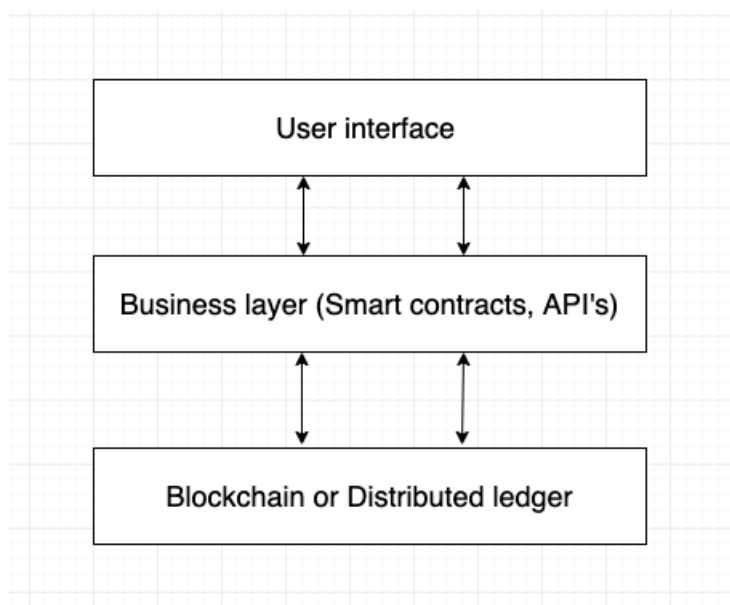


Figure 1.13: *Blockchain application design*

The change is in the business layer and the data layer, and we will understand the technologies being developed and used to build the business and data layer for the DApps in the next section.

Working of decentralized applications or DApps

Let us learn by example.

Imagine you have an application installed on your phone, and it works like *Instagram*, where you can share your photos with the world.

In general, when you upload a new photo in the application, it uploads your image in a storage, makes an entry in the database, and updates the newsfeed so that everyone who follows you can see the post.

Simple, right?

Let us do the same with the help of the blockchain.

So, here, you will upload the image in the application, and it will be stored in decentralized storage such as the **Interplanetary File System (IPFS)**. The storage will provide you with a hash, which is the identifier for the object.

Now, the hash would be added in the Blockchain to make a permanent entry of the photo, and then the newsfeed would be updated so that the people who follow you can see the content.

In DApps, every action is recorded in the chain, and even the delete action would be recorded in the chain. As mentioned earlier, you will not store actual photos in the

blockchain; instead, you will store the action, and action is immutable.

So far you have studied what the DApps is and how it works, let us understand the technologies and protocols to build the DApps.

DApps protocols

Bitcoin uses the blockchain to store the actual coins, and that is pretty much it. The next big thing after Bitcoin is Ethereum, which introduced something called **smart contracts**.

Smart contracts are programs that are executed on the Ethereum blockchain. This invention opened the path for the next-generation DApps.

Ethereum saw this opportunity, and they developed a complete set of tools and protocols required to build DApps on the Ethereum ecosystem.

As you can see in the following figure, Ethereum suits consist of Swarm, which acts as storage for the Ethereum applications, and Whisper to perform communications between the applications:

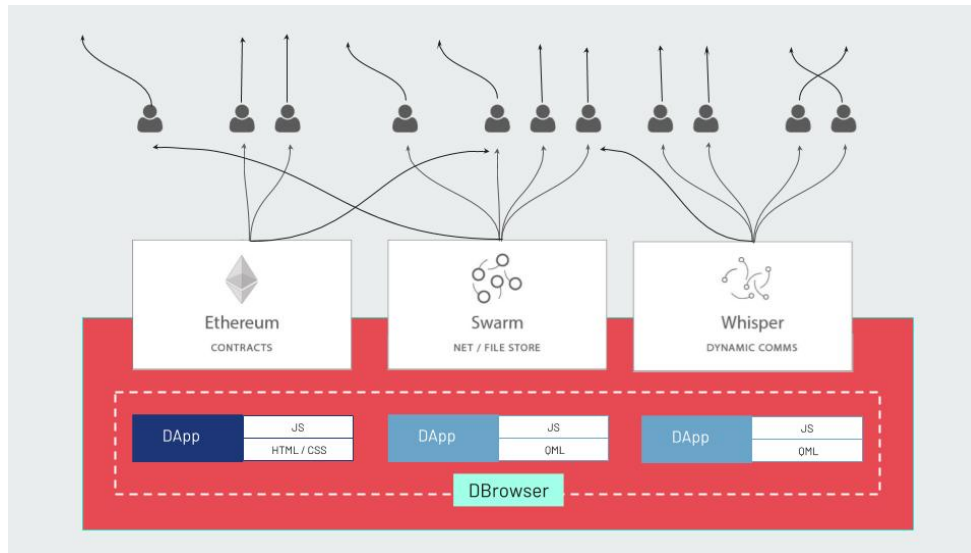


Figure 1.14: DApp system layer

You can develop Ethereum applications using the web3.js framework. This complete ecosystem is supported by the Ethereum foundation.

The only drawback is the ecosystem itself, and you are restricted from developing and running it in the Ethereum ecosystem.

The next protocol that we will learn is completely open-sourced and can be used to develop DApps using a mix of the technology stack.

The next protocol is a project by the protocol.ai team, which is called the **IPFS Stack**. First, let us look at the stack:

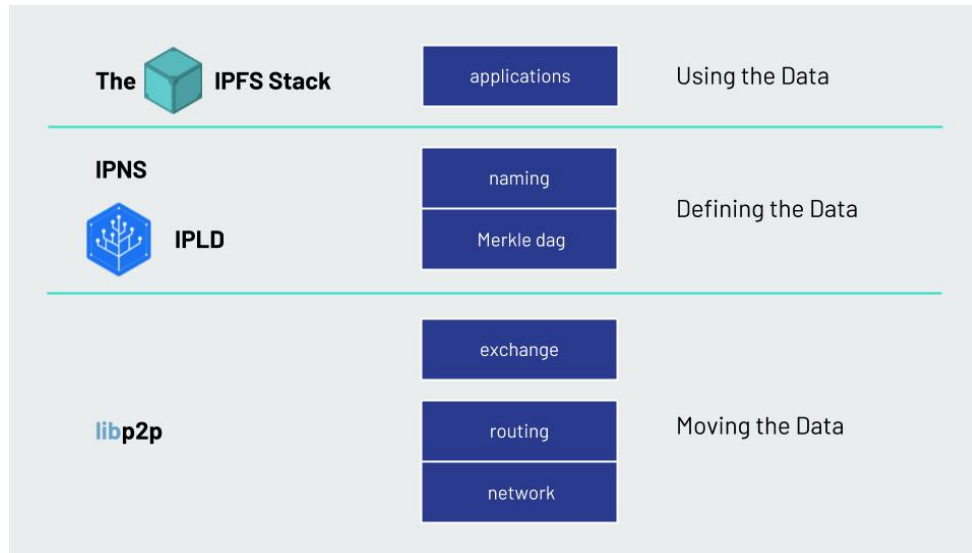


Figure 1.15: IPFS Stack

As in the preceding screenshot, the IPFS Stack is divided into three subset layers:

- Networking layer
- Data definition and naming layer
- Data layer

Let us look at each sub stack in brief.

Networking layer

One of the core challenges in the decentralized web is the peer-to-peer network, and designing the protocols that work in a peer-to-peer network in parallel to the centralized system.

The libpeer-to-peer project addresses this challenge with protocols labs. Libpeer-to-peer provides the modular stack that one can use to build a peer-to-peer network in conjunction with existing protocols such as WebRTC or any new transport layer protocols.

So, libpeer-to-peer is transported agnostics.

The features of libpeer-to-peer are:

- Libpeer-to-peer is a modular networking stack. You can use all of it or use part of the stack to build your application.
- Libpeer-to-peer provides transport and peer-to-peer protocols to build large, robust, and scalable network applications.
- Libpeer-to-peer is transport protocol agnostics. It can work with TCP, UDP, WebRTC, WebSockets, and UDP.
- Libpeer-to-peer offers several modules such as transport interface, discovery, distributed hash lookup, peer discovery, and routing.
- Libpeer-to-peer offers built-in encryption to prevent eavesdropping.
- Libpeer-to-peer offers built-in roaming features so that your service can switch networks without any intervention and loss of packets.

Libpeer-to-peer is the solution upon which the networking layer of peer-to-peer can be built.

Data definition and naming layer

Content addressing through hashes is widely used in a distributed system. We use hash-based content addressing in Git, cryptocurrencies, and so on. The same is also used in peer-to-peer networking.

IPLD provides a unified namespace for all hash-based protocols. Using IPLD, data can be traversed across various protocols to explore and view the data spread across a peer-to-peer network.

IPNS is a system to create a mutable address for the contents stored on IPFS. The reason why they are mutable is because the addresses of the content change every time the content changes.

Data layer

The topmost layer in the stack is the data layer. The IPFS is the peer-to-peer hypermedia protocol. IPFS provides a way to store and retrieve data across a peer-to-peer network.

It uses IPNS and libpeer-to-peer to create, name, and distribute content across a peer-to-peer network.

Anyone can become an IPFS peer and start looking for the content using hashes. An IPFS peer does not need to store all the data, they only need to store the data created by them, or the data they have searched for in the past.

The features of IPFS include the following:

- IPFS provides peer-to-peer hypermedia protocols for web 3.0.
- IPFS can work with existing protocols and browsers. It gives a smooth transition from centralized web to web 3.0.
- IPFS uses libpeer-to-peer to support peer-to-peer networking.
- IPFS data is cryptographically secure.
- IPFS can save tons of bandwidth in streaming services.

IPFS is under development, and there are some great applications built by developers that are running on top of the IPFS. One of the applications of our choice is DTube.

It is a decentralized version of *YouTube* with a built-in incentive and reward feature provided by Steem blockchain.

IPFS is also trying to solve the incentive issue. We cannot expect every person connected to the internet to act as a peer. To provide an incentive to those who participate and contribute value to the network, IPFS has created a coin called **Filecoin**, which can be paid to peers in the network by the user.

Other projects that we will learn about in the upcoming chapters are as follows:

- **OrbitDB**: Peer-to-peer database
- **Amazon QLDB**: Distributed immutable database
- **Ocean Protocol**: Data exchange protocol

These protocols provide the capability a developer needs to build a next-generation DApps. We will learn about these protocols in detail in the upcoming chapters.

Conclusion

We explored the blockchain and its basic components in this chapter. We also studied Bitcoin and its working in detail, along with the way it handles double spending issues and so on. We then moved to DApps and understood the protocols and frameworks required to build DApps in brief.

In the next chapter, we will learn about Ethereum and its working in detail. We will also learn about the Ethereum protocols for DApps.

References

- **Bitcoin official whitepaper:**
<https://www.bitcoin.com/bitcoin.pdf>
- **P2P Foundation:** <https://p2pfoundation.net/>
- **Satoshi Nakamoto's emails:**
<https://satoshi.nakamotoinstitute.org/emails/>
- **Learn me a Bitcoin:** <http://learnmeabitcoin.com/>

Join our book's Discord space

Join the book's Discord Workspace for Latest updates, Offers, Tech happenings around the world, New Release and Sessions with the Authors:

<https://discord.bpbonline.com>



CHAPTER 2

Ethereum, Smart Contracts, and DApps

Introduction

Ethereum is a revolution in the crypto world. There is no denying that. The introduction of smart contracts programs allowed developers to build their own cryptocurrency or apps and host it on blockchain, was a great move and a major success.

Structure

In this chapter, we will learn about Ethereum in-depth and cover the following topics:

- Introduction to Ethereum
- Bitcoin and Ethereum
- History of Ethereum
- Components of Ethereum
- Ethereum and DApps

- Ethereum clients
- Ethereum networks
- Installing Geth
- Synchronizing the testnet blockchain
- Installing Ganache
- Infura
- JSON Remote Procedure Call API
- Smart contracts and Solidity language
- Solidity data types
- Contract structure
- Writing our first Solidity program
- Decentralized Applications using Ethereum
- DApps stack
- Decentralized message communications protocols

Objectives

We will dive in-depth on each topic and cover all the details required to program the smart contract and build applications that are running on the Ethereum blockchain. We will learn the idea behind Ethereum and how it is different from Bitcoin.

Introduction to Ethereum

Ethereum is an open-source, **peer-to-peer (P2P)**, globally distributed public computing infrastructure that runs programs called **smart contracts** and maintains the system's state changes using the blockchain and cryptocurrency called **ether**.

That is a lot to take in one statement. The breakdown is as follows:

- **Open-source:** Ethereum is an open-source project. Anyone can participate and contribute to the codebase of Ethereum. Here is the codebase repository—<https://github.com/ethereum>.
- **P2P:** Ethereum runs on a decentralized P2P network like Bitcoin. There is no central authority to manage the conflict. Conflict is managed using consensus. We will learn about the consensus of Ethereum in the upcoming sections.
- **Globally distributed infrastructure:** Ethereum executes smart contracts on a public computing infrastructure formed using the P2P network. Think of it like a supercomputer but not in one place!
- **Smart contracts:** Smart contracts are programs that are executed on the Ethereum public distributed infrastructure. We will learn and code smart contracts in the upcoming sections.
- **Blockchain:** Ethereum uses blockchain to store the system's state and changes to maintain a conflict-free, transparent, and decentralized system.
- **Ether:** Ether is a native token used to charge the incentive for executing the programs on the network. It is also used to reward validators for helping validate the blocks in the network.

After the introduction of Bitcoin, Ethereum was the way to go when the world realized the power of blockchain beyond finance, and it serves well. Let us learn briefly about difference between Bitcoin and Ethereum.

Bitcoin and Ethereum

Ethereum used to share many common elements with Bitcoin such as a P2P network, proof of work consensus algorithm to maintain conflict-free state, cryptocurrencies such as Ether to support incentive and utility payment in the network, wallets, and digital signature.

Now, at the time of updating this book, Ethereum has moved away from proof of work consensus to proof of stake algorithm which draws a fundamental shift and different in both of the platforms.

However, Ethereum differs from Bitcoin in design and purpose. While Bitcoin is designed to support the global cryptocurrency system, Ethereum is designed as a programmable **virtual machine (VM)** capable of executing programs on the blockchain.

Bitcoin supports simple code execution in the form of scripts that support normal if/else statements, and Ethereum supports Turing complete programming language.

In short, Bitcoin is limited to cryptocurrency, Ethereum is an open and general-purpose Turing complete VM.

Ethereum, with its general-purpose programming language and a wide range of software development kit, allows developers across the globe to build their applications. One of the famous applications is CryptoKitties.

CryptoKitties is a game that runs on Ethereum blockchain that allows players to buy, breed, and sell virtual cats on the blockchain. It is extremely popular and gave the world a bird-eye view of the capability of the blockchain.

Ethereum's scope is wide and allows developers to write their applications without the need for creating a

blockchain and network by themselves. Developers can focus on building applications, while network infrastructure, state management and code execution are handled by the Ethereum network.

Let us learn a little about the history of Ethereum to understand the reason behind the invention of this amazing project.

History of Ethereum

All great innovations solve big problems, and Ethereum is no exception. Ethereum was invented at a time when people recognized the widespread application of the Bitcoin model, and were trying to move beyond cryptocurrency.

Though it was not easy, developers faced a problem that required a new set of protocols and tools. They had to either build a new application on top of Bitcoin or start a new blockchain (it is a hard thing to do for every single application).

If developers choose to build on top of the Bitcoin model, they face issues, mainly technical, because Bitcoin is developed to support cryptocurrency, and to move beyond it requires an immense amount of effort and probably a fork in the chain.

For projects that needed more freedom and flexibility while using blockchain, a new blockchain was the only option. But that meant a lot of work: Bootstrapping all the infrastructure elements, development, team building, testing, and so on.

In December 2013, *Vitalik Buterin*, a *Russian-Canadian* programmer and Bitcoin enthusiast, published a paper that

coined the idea of Turing complete general-purpose blockchain:



Figure 2.1: *Vitalik Buterin speaking at Forbes*

Here is the link to the Ethereum white (yellow) paper (it is recommended that all of you read it once):
<https://ethereum.github.io/yellowpaper/paper.pdf>.

The idea behind Ethereum was to design a general-purpose blockchain that is programmable so that anyone around the world can program the applications and run it on the blockchain without developing the underlying mechanisms of P2P networks, blockchains, consensus algorithms, and so on.

In July 2015, the first Ethereum block was mined. It marks the date for the general-purpose world computer to begin its work. The following figure marking the first Ethereum transaction:

Block #0

Sponsored:  AMFEIX - INVEST WITH AMFEIX (45%+ Estimated APR). Check it out on AMFEIX.com.

Overview	Comments
Block Height:	0 < >
Timestamp:	1672 days 22 hrs ago (Jul-30-2015 03:26:13 PM +UTC)
Transactions:	8893 transactions and 0 contract internal transaction in this block
Mined by:	0x00000000000000000000000000000000 in 15 secs
Block Reward:	5 Ether
Uncles Reward:	0
Difficulty:	17,179,869,184
Total Difficulty:	17,179,869,184
Size:	540 bytes
Gas Used:	0 (0.00%)
Gas Limit:	5,000
Extra Data:	 N4[N ]( p  3  z8  zC  (Hex:0x11bbe8db4e347b4e8c937c1c8370e4b5ed33adb3db69cddb7a38e1e50b1b82fa)

[Click to see more](#) ↓

Figure 2.2: First Ethereum block mined

Source: <https://etherscan.io/block/0>

Ethereum has grown to become the largest blockchain network in the world and has opened ways for a new set of applications—**decentralized applications (DApps)**.

In the next section, let us learn about the components of Ethereum blockchain.

Components of Ethereum

In Ethereum, the following components play important roles:

- **P2P network:** Ethereum, like Bitcoin, runs on the mainnet network that is accessible by default at port **30303** on your computer. Ethereum P2P network

communicates with each other using a protocol called **DEVp2p Wire Protocol**.

- **Consensus mechanism:** Ethereum uses a proof of stake consensus algorithm.
- **Transactions:** Ethereum transactions include information about the network, along with the sender, recipient, value, and payload.
- **State machine:** This is a unique and important component of the Ethereum. Ethereum state transitions are managed by Ethereum VM, a VM that executes bytecode generated by programs called **smart contracts**.
- **Data structure:** Ethereum state is also locally stored in each node. Ethereum by default uses the database (also called **LevelDB**), which contains the transactions and system state in a Merkle tree format.
- **Clients:** Ethereum provides several client software, such as go-ethereum also called **Geth** and **Parity** (deprecated).

In the next section, we will look at the transition of Ethereum from a general-purpose blockchain to DApps.

Ethereum and DApps

Project Ethereum was started as a means to build a general-purpose blockchain that could be programmed for a variety of uses. Ethereum's vision swiftly expanded to become a platform for programming and hosting DApps.

DApps portray a wider perspective than smart contracts. A DApp is, at the minimum, a smart contract, and a web user interface. More broadly, a DApp is a web application that is

developed on top of open, decentralized, P2P network and infrastructure.

A DApp is composed of at least the following:

- Smart contracts on a blockchain.
- A web frontend user interface.

Also, many DApps include other decentralized components, such as:

- A decentralized (P2P) storage protocol and platform such as **Interplanetary File System (IPFS)** and Swarm.
- A decentralized (P2P) messaging protocol and platform such as libp2p and Whisper.

The idea and origin of DApps is aimed to take the **World Wide Web (WWW)** to its next natural evolutionary stage, introducing decentralization with P2P protocols into every aspect of web application. The term used to describe this evolution is Web 3.0, meaning the third version of the web. The WWW controlled by no central authority.

In the next section, we will explore the capability of Ethereum DApps in detail.

Let us start with the development of smart contracts. Before writing the smart contracts, we must learn about Ethereum clients, that is, running blockchain in your system or using free third-party services.

Ethereum clients

Ethereum provides client software to become a peer in the network. You can download and install these clients to become a peer and perform operations such as querying

the blockchain, performing transactions, executing smart contracts, and so on.

There are different open-source versions of Ethereum client, and they interoperate if they comply with Ethereum reference specifications and standard communication protocols. While different teams and people develop these clients, they all communicate through the standard communication protocol.

Ethereum is an open-source and community-driven project licensed under LGPL v3.0. It is free to download and can be used for any purpose. Due to these clear specifications and licensing, different projects are independently developed by the people and companies around the world.

At the time of writing this book, there are six main implementations of the Ethereum protocol written in six different programming languages, as follows:

- **Geth:** Written in Go programming language.
- **Nethermind:** Written in C# and .Net framework.
- **Besu:** Written in Java programming language.
- **Erigon:** Written in Go programming language.
- **Reth:** Written in Rust programming language.
- **EthereumJS:** Written in TypeScript is in beta at the time of writing this book.

We will learn about one of the most used clients—Geth—which is also a sort of official Ethereum client, as the Ethereum foundation manages it.

Before running these clients in our machine, let us understand the types of Ethereum networks and which one we should use.

Ethereum networks

Ethereum networks are classified into two types:

- **Mainnet (also known as full node):** Live network handling real ethers, that is, money.
- **Testnet:** Test network designed for research and development.

As shown earlier, testnet is best suited for development purposes, while mainnet is the actual P2P network.

Running a mainnet in your machine incurs hardware resources cost and bandwidth. It is not recommended to run one in your local machine. You can do almost everything you need to do with the testnet node.

We can also use a private blockchain like Ganache or a cloud-based client-provider such as *Infura*. We will learn about them also in this chapter.

So, let us install and run Geth in our machine.

Installing Geth

Geth is the Ethereum client developed in Go language that is actively developed by the *Ethereum Foundation*, so it is considered the official implementation of the Ethereum client. Generally, every Ethereum-based blockchain will have its Geth instance running.

Let us look at how to install Geth when you are running Ubuntu in your system. If you are using Windows or macOS, you can use the following links for installation:

- **Installation instruction for Windows:**
<https://geth.ethereum.org/downloads/>

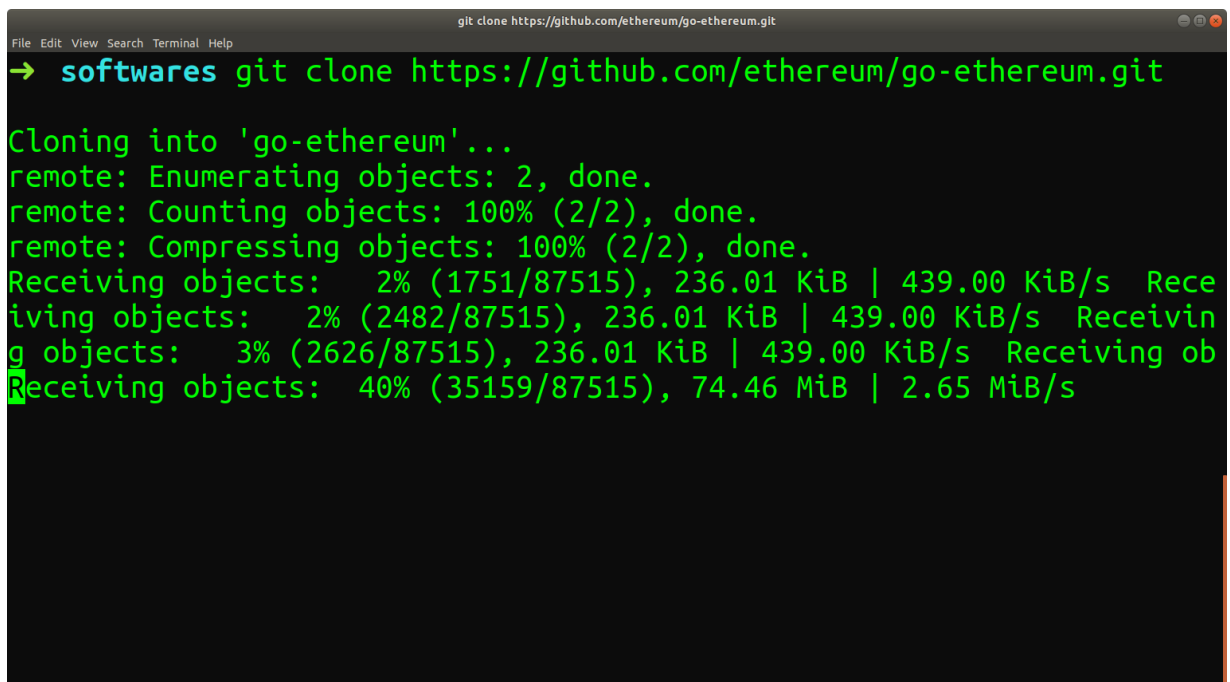
- **Installation instruction For Mac:**
<https://geth.ethereum.org/docs/install-and-build/installing-geth#install-on-macos-via-homebrew>

Let us install, open a terminal, and clone the repository in your local system.

Here is the command:

git clone <https://github.com/ethereum/go-ethereum.git>

It may take some time to download and install Geth in your system as shown in the following figure:

A screenshot of a macOS Terminal window. The title bar at the top reads "git clone https://github.com/ethereum/go-ethereum.git". The terminal has a menu bar with "File", "Edit", "View", "Search", "Terminal", and "Help". The command prompt shows a green prompt character followed by "softwares git clone https://github.com/ethereum/go-ethereum.git". The output of the command is displayed in green text: "Cloning into 'go-ethereum'...", "remote: Enumerating objects: 2, done.", "remote: Counting objects: 100% (2/2), done.", "remote: Compressing objects: 100% (2/2), done.", "Receiving objects: 2% (1751/87515), 236.01 KiB | 439.00 KiB/s", "Receiving objects: 2% (2482/87515), 236.01 KiB | 439.00 KiB/s", "Receiving objects: 3% (2626/87515), 236.01 KiB | 439.00 KiB/s", "Receiving objects: 40% (35159/87515), 74.46 MiB | 2.65 MiB/s".

```
File Edit View Search Terminal Help
→ softwares git clone https://github.com/ethereum/go-ethereum.git

Cloning into 'go-ethereum'...
remote: Enumerating objects: 2, done.
remote: Counting objects: 100% (2/2), done.
remote: Compressing objects: 100% (2/2), done.
Receiving objects: 2% (1751/87515), 236.01 KiB | 439.00 KiB/s
Receiving objects: 2% (2482/87515), 236.01 KiB | 439.00 KiB/s
Receiving objects: 3% (2626/87515), 236.01 KiB | 439.00 KiB/s
Receiving objects: 40% (35159/87515), 74.46 MiB | 2.65 MiB/s
```

Figure 2.3: Cloning the Ethereum codebase

Once it is downloaded, you need to switch to the directory and run these commands:

make geth

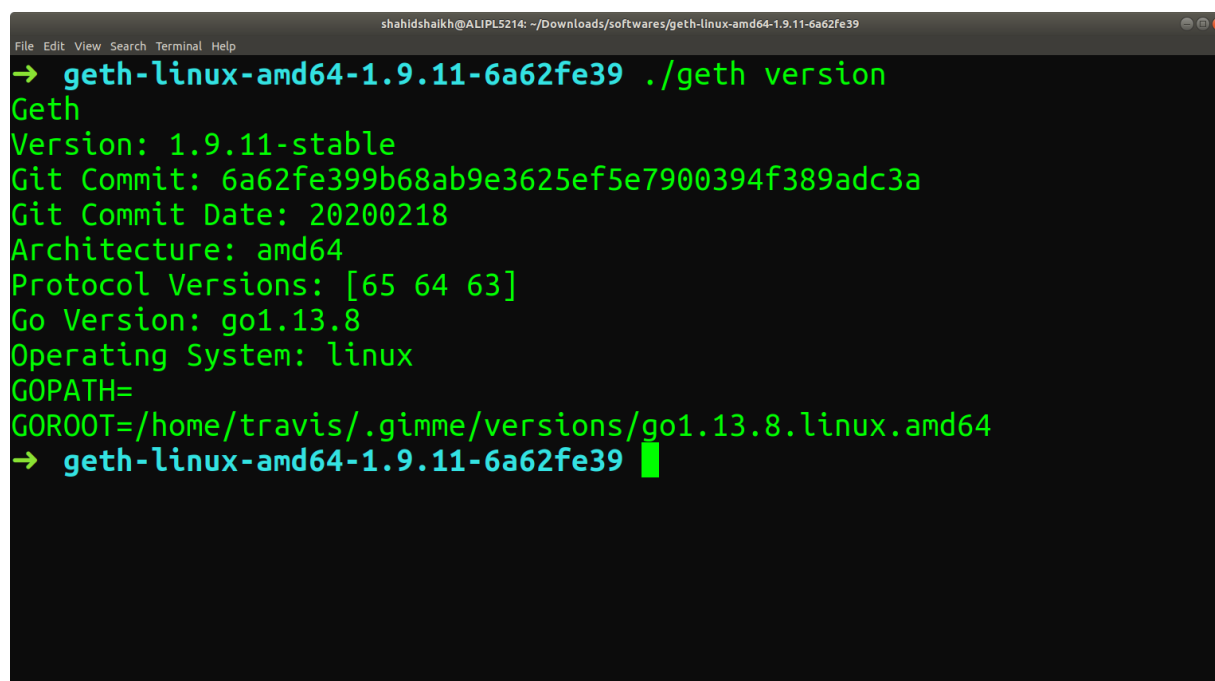
It will take some time to build Geth from the source. Once installed, the executable files will be placed in the **/build/bin** folder inside the **go-ethereum** folder.

You can also directly download the build package from the website and run it on your computer. To download the build files, visit the Ethereum Geth downloads page at **<https://geth.ethereum.org/downloads/>** and download the version suitable for your operating system.

Once you are done building it from source or downloading and unzipping it, go to the folder and run this command to check the Geth version:

./geth version

It should print the Geth version details like this:

A terminal window with a dark background and green text. The window title is 'shahidshalkh@ALIPL5214: ~/Downloads/software/gets-linux-amd64-1.9.11-6a62fe39'. The command prompt shows '→ gets-linux-amd64-1.9.11-6a62fe39 ./geth version'. The output is as follows:

```
Geth
Version: 1.9.11-stable
Git Commit: 6a62fe399b68ab9e3625ef5e7900394f389adc3a
Git Commit Date: 20200218
Architecture: amd64
Protocol Versions: [65 64 63]
Go Version: go1.13.8
Operating System: linux
GOPATH=
GOROOT=/home/travis/.gimme/versions/go1.13.8.linux.amd64
→ gets-linux-amd64-1.9.11-6a62fe39
```

Figure 2.4: Checking version of Ethereum

You may see slightly different information than shown in the preceding image, but the report format should look as shown.

Now that we have successfully downloaded the Geth, let us understand what we can do with it. First, let us synchronize the blockchain and become a peer in the public testnet network.

Synchronizing the testnet blockchain

We have studied that every peer has a copy of the complete chain in a blockchain network. To become a peer, one must first get a copy of the blockchain from the other peers in the network. It is called **synchronization** or **sync**, and this operation copies the blockchain data from the genesis block to the recent block.

Here is the command to start the synchronization process:

```
./geth --testnet --syncmode "fast"
```

Understand the command parameters:

- **--testnet**: This means we need to only sync the testnet data, which is smaller in size than mainnet.
- **--syncmode**: This selects various modes that we can use to sync the chain. In the fast mode, Geth does not validate every block, which reduces the time to sync the chain.

However, syncing the whole chain may take a day depending on your internet connection bandwidth.

You should see the following in the console after running the command:

```
./geth --testnet --syncmode "fast"
INFO [03-03|17:26:20.221] Allocated fast sync bloom      si
ze=512.00MiB
INFO [03-03|17:26:20.237] New local node record          se
q=1 id=eaab80f9ccff163b ip=127.0.0.1 udp=30303 tcp=30303
INFO [03-03|17:26:20.238] Started P2P networking        se
lf=enode://f18bf6ad67a43df003961a436bd897f53c57d50d94e65fe5bdb13fdb4e
41389e0b08407d508b1976fb261a519e98eb6bba161e790b9ae1e0bab0fbcadc1d100
a@127.0.0.1:30303
INFO [03-03|17:26:20.241] IPC endpoint opened           ur
l=/home/shahidshaikh/.ethereum/testnet/geth.ipc
INFO [03-03|17:26:20.266] Initialized fast sync bloom    it
ems=355 errorrate=0.000 elapsed=44.225ms
WARN [03-03|17:26:21.969] Dropping unsynced node during fast sync id
=c7e7675577416d6b conn=dyndial addr=52.176.100.77:30303 type=Parity-E
thereum/v2.5.11-stable-fc129b4-20191205/x86_64-linux-gnu/rustc1.39.0
INFO [03-03|17:26:25.704] New local node record          se
q=2 id=eaab80f9ccff163b ip=114.143.107.254 udp=30303 tcp=30303
```

Figure 2.5: *Ethereum testnet syncing*

To check sync status, you can check the terminal and find the pending values. Those are the number of blocks that are required to sync before our machine can become a peer in the Ethereum testnet network.

Let us see how to install a private blockchain software called **Ganache**, that is specially designed for development purposes.

Installing Ganache

Ganache is a software that allows you to quickly fire up a personal Ethereum blockchain that can be used to run tests and execute smart contracts without becoming a peer.

At the time of updating the book, Ganache was deprecated. We are keeping this part here for the understanding but not recommend you to use it. You can skip this to *Infura* section or read it out for the knowledge.

To download Ganache, visit the official website and download the software form: <https://www.trufflesuite.com/ganache>. Once you have downloaded the software, unzip it, and run it in your system.

Ganache looks like this upon running:

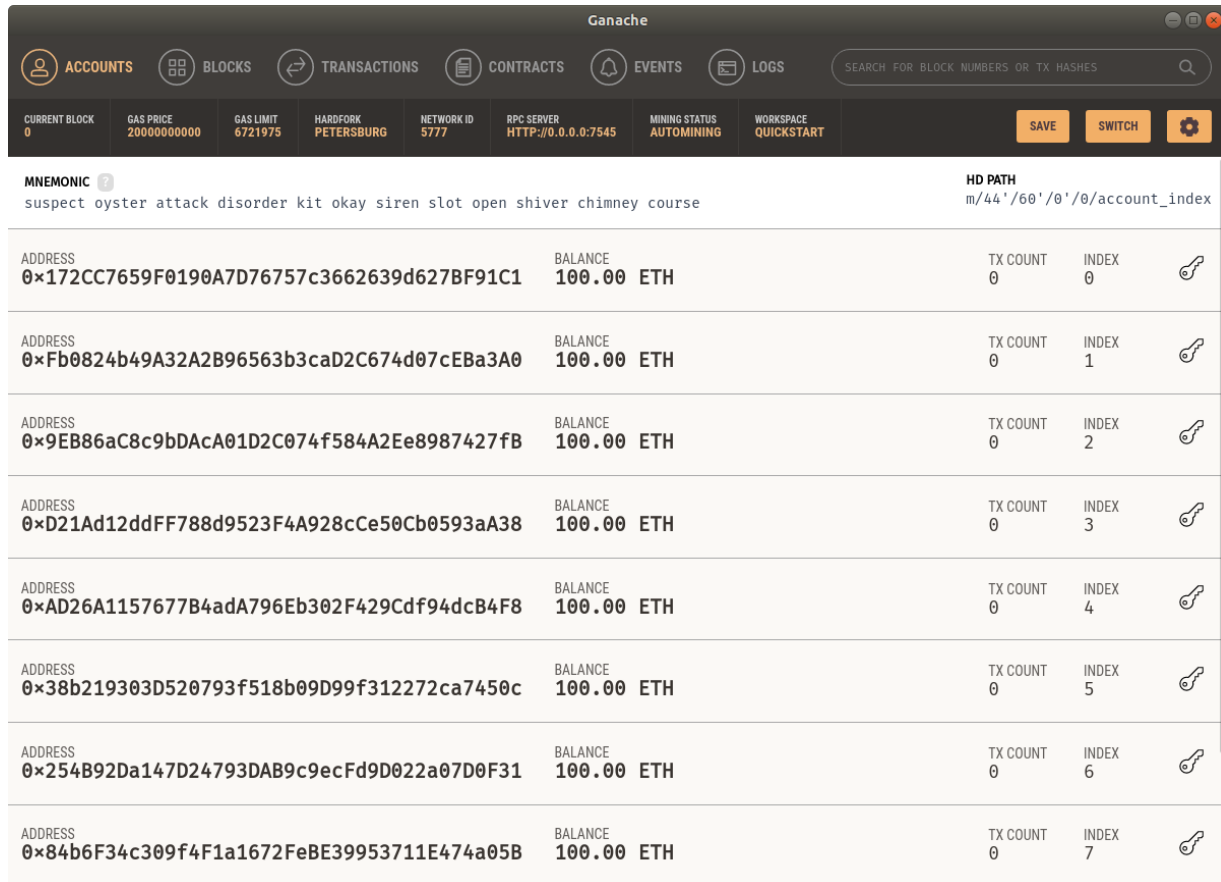


Figure 2.6: Ganache

Ganache provides multiple test wallets with test Ethers so that you can perform a test. You can check the block and transaction status.

Ganache also has a CLI package that we can install to interact with the Ganache **Remote Procedure Call (RPC)** server in a command-line mode.

Let us look at the remote Ethereum client provider and see how we can use it. We will use the Infura service.

Infura

Infura is a third-party Ethereum and IPFS API provider. You can use Infura infrastructure to run your applications for free. Infura handles all the synchronization and infrastructure hassles, while you, as a developer, can focus on the code. The Infura website is as follows:

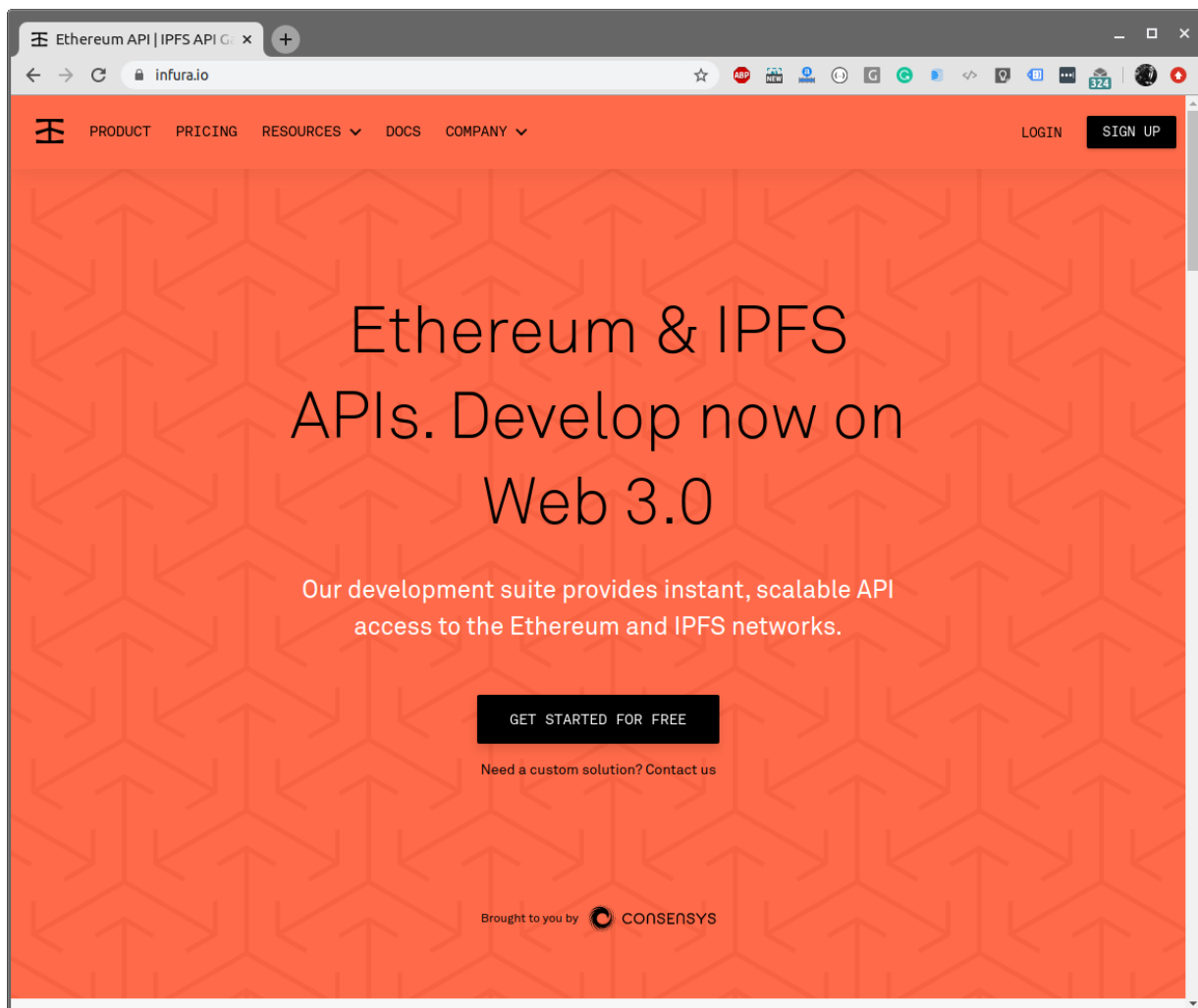


Figure 2.7: Infura website

Source: <https://infura.io/>

To use Infura services, you need to create your account using an email address. Once your account is created, you can log in to the Infura dashboard, which lists the projects you have created with Infura, along with some other details. It looks as shown in the following figure:

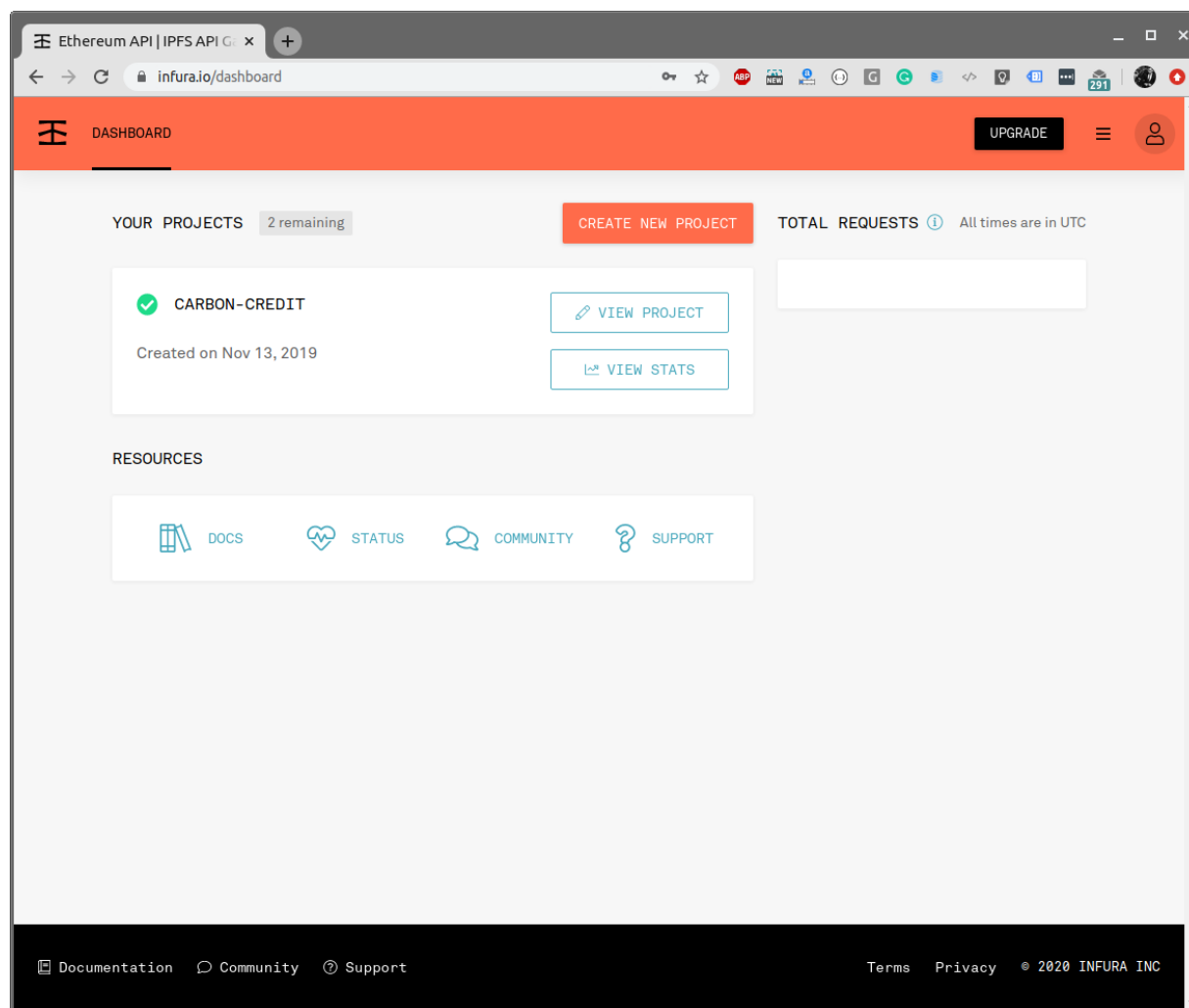


Figure 2.8: Infura dashboard

As you can see in the preceding image, a project has already been created. You can create your project by clicking on the button called **CREATE NEW PROJECT**. Give your project a proper name, and Infura will create a test and mainnet network for your project behind the scenes.

Once your project is created, you can view all the details such as, the peer endpoint that you can use to connect to your peer, and project ID and secret.

You can use the credentials to interact with the Infura network. Ensure that it is safe. In the next section, we will take a brief look at the JSON-RPC API provided by the Ethereum clients.

JSON Remote Procedure Call API

Ethereum clients provide an application programming interface and a set of RPC commands, which are encoded as **JavaScript Object Notation (JSON)**. We refer to this as the JSON-RPC API.

The JSON-RPC API is an interface that allows us to write programs that use an Ethereum client as a gateway to an Ethereum network and blockchain. We can interact with JSON-RPC server either by building an RPC client in a programming language of your choice, or we can directly call the HTTP endpoint.

First, let us make a simple HTTP call to the Ethereum RPC endpoint and see if it works. Here is the command that will return the client version:

```
curl -X POST -H "Content-Type: application/json" --data \  
'{"jsonrpc":"2.0","method":"web3_clientVersion","params":[],"id":1}' \  
http://localhost:7545
```

In the preceding command, we are sending the JSON-RPC call to the RPC server running at port **7545**.

It should return the following output in the Terminal:

```
{"id":1,"jsonrpc":"2.0","result":"EthereumJS  
TestRPC/v2.8.0/ethereum-js"}
```

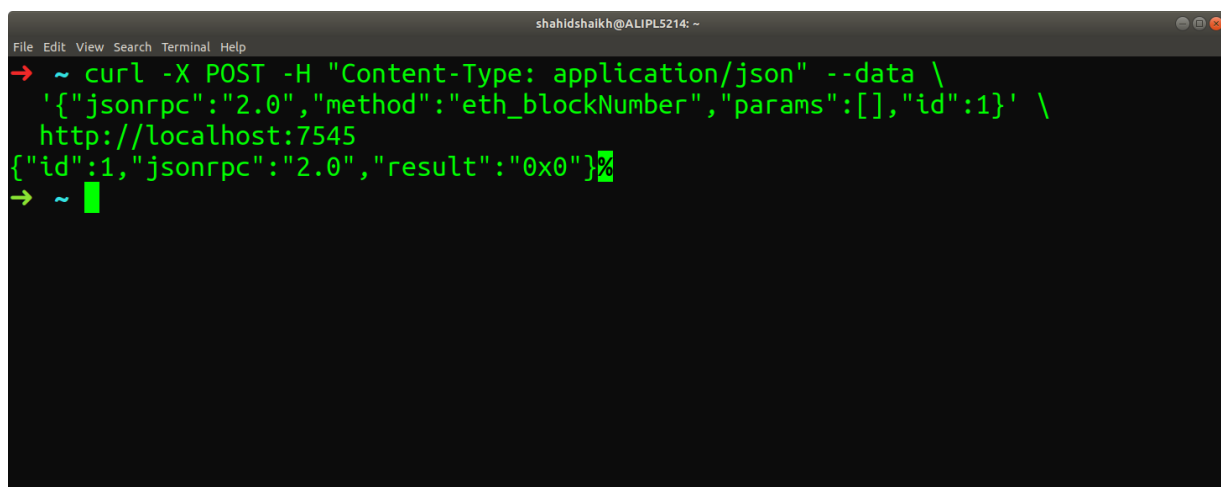
It may vary if you are using Geth or Infura. You can check out all of the commands in the GitHub wiki of Ethereum: <https://github.com/ethereum/wiki/wiki/JSON-RPC#json-rpc-methods>.

Let us make another RPC call. This time, we will ask the RPC server to return the latest block number.

Here is the command. Copy and paste it in your Terminal:

```
curl -X POST -H "Content-Type: application/json" --  
data \  
'{"jsonrpc":"2.0","method":"eth_blockNumber","para  
ms":[],"id":1}' \  
http://localhost:7545
```

This should return the following output:

A terminal window with a dark background and green text. The window title is 'shahidshalkh@ALIPL5214: ~'. The terminal shows a curl command being executed: 'curl -X POST -H "Content-Type: application/json" --data \'{"jsonrpc":"2.0","method":"eth_blockNumber","params":[],"id":1}\' http://localhost:7545'. The output of the command is a JSON object: '{"id":1,"jsonrpc":"2.0","result":"0x0"}'. The cursor is at the end of the command line.

```
shahidshalkh@ALIPL5214: ~  
File Edit View Search Terminal Help  
→ ~ curl -X POST -H "Content-Type: application/json" --data \  
  '{"jsonrpc":"2.0","method":"eth_blockNumber","params":[],"id":1}' \  
  http://localhost:7545  
{"id":1,"jsonrpc":"2.0","result":"0x0"}%  
→ ~
```

Figure 2.9: JSON-RPC API response

You can convert the **0x0** that is hexadecimal into a decimal by running the following command in the Terminal:

```
$ echo $((0x0))
```

```
$ 0
```

You should try other commands and interact with the RPC to gain a command over the functions and methods of Ethereum chain. We will use these functions in the upcoming section to program our smart contracts.

Till now, we studied Ethereum in detail and learned how to become a peer in the Ethereum network; we also looked at the Ethereum RPC calls and how to use and interact with it.

In the next section, we will learn about smart contracts and Solidity programming language.

Smart contracts and Solidity language

Smart contracts are programs that run on **Ethereum VMs (EVMs)** and govern the behavior of accounts in the Ethereum state.

Let us understand the following definitions:

- **Computer programs:** Smart contracts are simply computer programs like any other program. The word contract has no legal meaning per say.
- **Immutable:** Once a smart contract is deployed, it cannot be changed to meet the basic criteria of blockchain, that is, immutability. A new change must be deployed again with a new version number.
- **VM context:** Smart contracts run under a limited execution context. Smart contracts can access their state, the context of the transaction, and information about the most recent blocks if required.

- **Decentralized computer:** VMs run on each node around the world and share the same state, so the whole system operates as a single computer.

Smart contracts are widely written in the Solidity programming language.

Solidity is an object-oriented, high-level programming language for implementing smart contract programs for Ethereum.

Solidity is heavily inspired by the C++, Python, and JavaScript, and you will find it familiar if you have worked on any of these programming languages in your career. Solidity is statically typed (that means types are checked during compile time), and it supports inheritance, libraries, and user-defined types.

In this section, we will learn about Solidity projects and data types, and we will also develop a program in Solidity.

Solidity is a high-level programming language, but it converts its code into machine-level executable code with the help of solidity compilers, which are commonly referred to as **solc**.

You can install a Solidity compiler in your computer in many ways. The simplest way to install the compiler is by using the node package manager, that is, npm.

Assuming you have Node.js installed in your machine, you can run this command in the terminal to install a Solidity compiler:

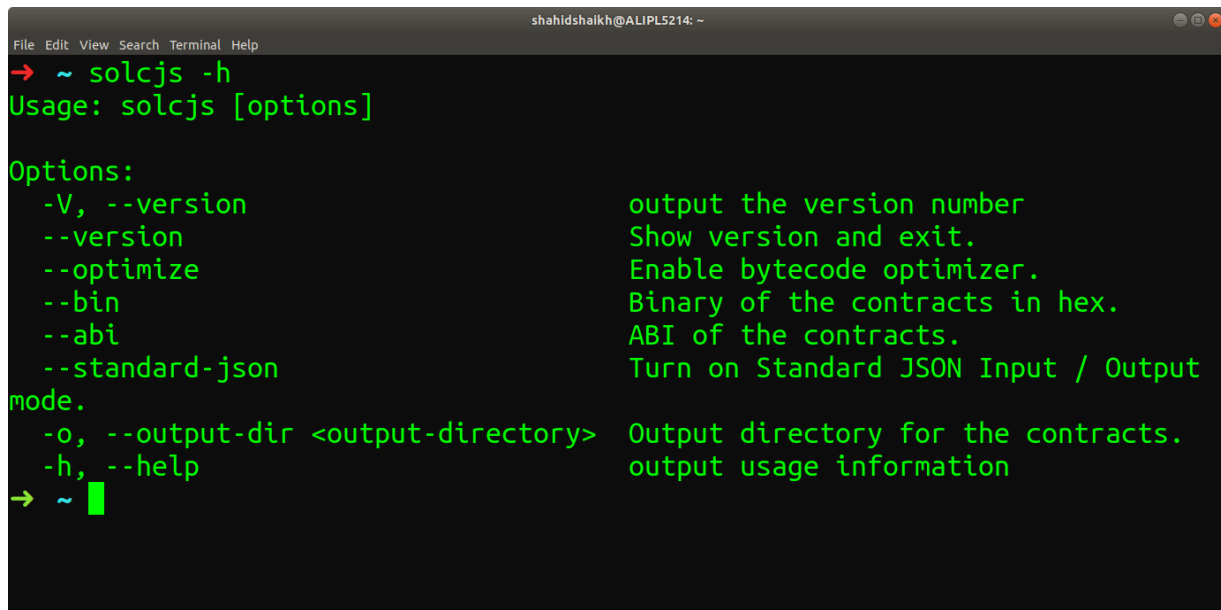
sudo npm install -g solc

This command will install a Solidity compiler in your machine. Once the installation is complete, you can verify it

by running the following command:

```
$ solcjs -h
```

This command will print the options related to Solidity compilers, as illustrated in the following figure:



```
shahidshalkh@ALIPL5214: ~  
File Edit View Search Terminal Help  
→ ~ solcjs -h  
Usage: solcjs [options]  
  
Options:  
  -V, --version          output the version number  
  --version              Show version and exit.  
  --optimize             Enable bytecode optimizer.  
  --bin                  Binary of the contracts in hex.  
  --abi                  ABI of the contracts.  
  --standard-json        Turn on Standard JSON Input / Output  
mode.  
  -o, --output-dir <output-directory> Output directory for the contracts.  
  -h, --help             output usage information  
→ ~
```

Figure 2.10: Available commands

You can visit the official website of Solidity for detailed installation instructions:

<https://solidity.readthedocs.io/en/latest/installing-solidity.html>.

Let us look over the data types of Solidity programming language. We can cover every detail about Solidity in one section, but the must-know stuff to help you start writing Solidity programs or smart contracts will be explained.

Solidity data types

Some of the basic data types of Solidity are as follows:

- **Boolean:** Boolean value; holds true or false.

- **Integer:** **Signed integer (int)** and **unsigned integer (uint)** are declared as a 256 bit integer value. However, we can provide the size suffix in increments of 8. For example, **int8**, **int16** to **uint256**.
- **Fixed point:** Fixed point numbers, declared with **fixedMxN** or **ufixedMxN**, where **M** is the size of bits in the increment of 8 and **N** is the number of decimals after the point. For example, **ufixed32x2** means unsigned fixed point with 32 bits in size and two decimal points.
- **Address:** It is a 20 byte Ethereum address type. Address payable is a type that provides additional members to transfer and send, and it is the address where one can send ethers.
- **Arrays:** It comes in two types:
 - Fixed types are fixed-size arrays declared with **bytes1** and **bytes32**.
 - Dynamic arrays vary in size. They are declared with **bytes** or **string**.
- **Enum:** It is a user-defined type for discrete values.
- **Struct:** It is a user-defined data container like the structures in C++.
- **Mapping:** It is a hash lookup table by mapping keys with values.

These are basic types to handle data. There are also time and ether units that follow standard units. Time converts to milliseconds and Ether to the wei base unit.

Note: wei is the smallest unit of the Ether.

Let us look over the structure of a contract to understand how to write contracts.

Contract structure

Each contract, in solidity, can contain the following:

- State variables
- Functions
- Function modifier
- Events
- Struct
- Enum

Contracts can also inherit other well-written contracts for code reusability. Let us understand the contract structure in detail.

State variables

State variables are variables whose values are permanently stored in the contract. Any variable declared with the types we learned earlier are state variables, and it is permanently restored.

Consider the following example:

```
pragma solidity 0.4.0;  
contract SimpleStorage {  
    uint storedData; // State variable  
    // ...  
}
```

Functions

Functions contain code and can be invoked by calling them from the program. Functions have visibility such as public,

private, and so on. They accept parameters and return variables.

Refer to the following example:

```
pragma solidity >=0.4.0 <0.7.0;  
contract SimpleAuction {  
function sayHello() public payable { // Function  
    // ...  
}  
}
```

Function modifier

Function modifiers can be used to change the behavior of functions in a declarative way. For example, we can use a modifier to execute a conditional code before executing functions.

Events

As the name suggests, it provides functionality to emit and subscribe to events. Events allow programmers to write modular and clean codebase.

Struct

Struct—short for structure—is the way to define a user-generated data structure.

Take a look at this example:

```
pragma solidity 0.4.0;  
contract UserInfo {  
    struct User { // Struct
```



```
        uint Id;  
byte32 name;  
address delegate;  
    }  
}
```

Enum

Enum data types allow developers to create their type in Solidity.

Consider the following example:

```
pragma solidity 0.4.0;  
contract Purchase {  
enum UserStatus { Active, Pending, Inactive } // Enum  
}
```

Let us write our first Solidity program, and any doubts you have will be cleared by going through the preceding example code.

Writing our first Solidity program

Let us write our first smart contract. You must have the following things installed in your machine:

- Node.js
- Hardhat
- Solc

You can choose the code editor of your choice, but it is highly recommended that you use **Visual Studio Code (VS Code)** editor.

You can download the editor from the official website through the following link:
<https://code.visualstudio.com/>

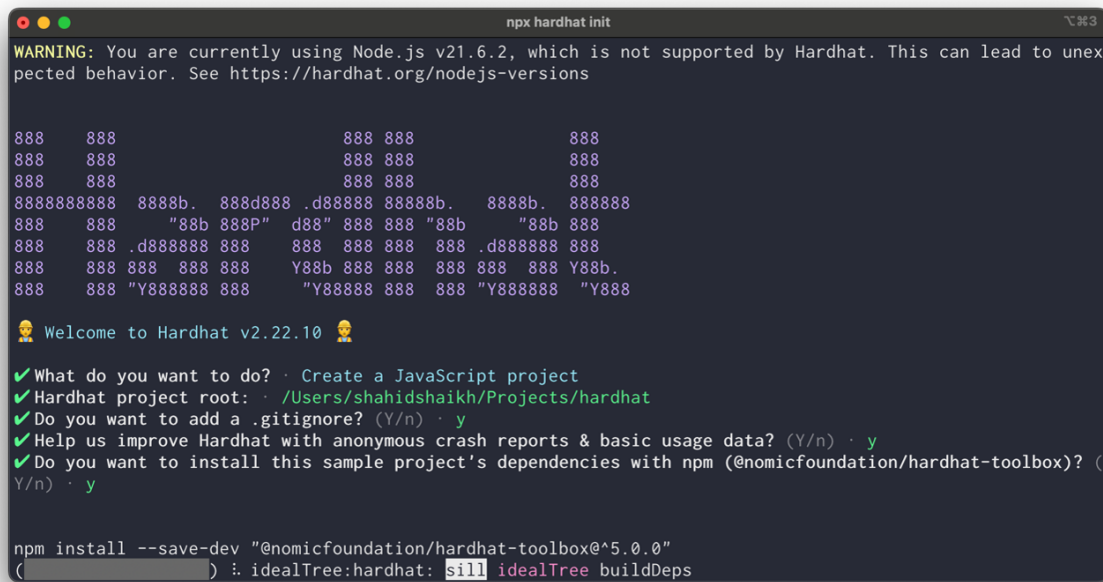
If you are using VS Code, it is recommended to install the Solidity extension. The steps to be taken are as follows:

1. To install, click on the Extension icon in the VS Code left bar and search for solidity.
2. Click on the **Install** button, and you are good to go.
3. Assuming that you have Node.js installed, here is the process to install Hardhat.

Hardhat is an Ethereum development framework suit, and it makes writing and deploying smart contracts easy and developer-friendly. Now, refer to the following steps:

1. Run the following command to install Truffle in your machine:
\$ npm install --save-dev hardhat
2. Once it is installed, we can move ahead with the project.
3. Create a new folder and navigate to it using the command line. Run this command to create a new Hardhat project.
4. To create a folder:
\$ mkdir solidityDemo
\$ cd solidityDemo
5. Then, run the following command:
\$ npx hardhat init

It should provide the following output in the Terminal:



```
npx hardhat init

WARNING: You are currently using Node.js v21.6.2, which is not supported by Hardhat. This can lead to unexpected behavior. See https://hardhat.org/nodejs-versions

      888      888      888 888      888
      888      888      888 888      888
      888      888      888 888      888
888888888888 8888b. 888d888 .d888888 88888b. 8888b. 8888888
888 888 "88b 888P" d88" 888 888 "88b "88b 888
888 888 .d888888 888 888 888 888 .d888888 888
888 888 888 888 888 Y88b 888 888 888 888 Y88b.
888 888 "Y888888 888 "Y88888 888 888 "Y888888 "Y888

👋 Welcome to Hardhat v2.22.10 👋

✔ What do you want to do? · Create a JavaScript project
✔ Hardhat project root: · /Users/shahidshaikh/Projects/hardhat
✔ Do you want to add a .gitignore? (Y/n) · y
✔ Help us improve Hardhat with anonymous crash reports & basic usage data? (Y/n) · y
✔ Do you want to install this sample project's dependencies with npm (@nomicfoundation/hardhat-toolbox)? (Y/n) · y

npm install --save-dev @nomicfoundation/hardhat-toolbox@^5.0.0
( ) : idealTree:hardhat: sill idealTree buildDeps
```

Figure 2.11: Hardhat init command

Now, open the project in your code editor. The structure of the project is as follows:

contracts

ignition

test

artifacts

hardhat-config.js

The **contracts** folder stores the code written in Solidity. The **ignition** folders contain the scripts to deploy contracts. The **hardhat-config.js** contains configuration options; we do not need to touch that file for our project.

Switch to the **contracts** folder and you should see **Lock.sol** file and content should be similar to the following:

```

pragma solidity ^0.8.24;
contract Lock {
    uint public unlockTime;
    address payable public owner;
    event Withdrawal(uint amount, uint when);
    constructor(uint _unlockTime) payable {
        require(
            block.timestamp < _unlockTime,
            "Unlock time should be in the future"
        );
        unlockTime = _unlockTime;
        owner = payable(msg.sender);
    }

    function withdraw() public {
        // Uncomment this line, and the import of
        "hardhat/console.sol", to print a log in your terminal

        // console.log("Unlock time is %o and block
        timestamp is %o", unlockTime, block.timestamp);

        require(block.timestamp >= unlockTime, "You can't
        withdraw yet");

        require(msg.sender == owner, "You aren't the
        owner");

        emit Withdrawal(address(this).balance,
        block.timestamp);

        owner.transfer(address(this).balance);
    }
}

```

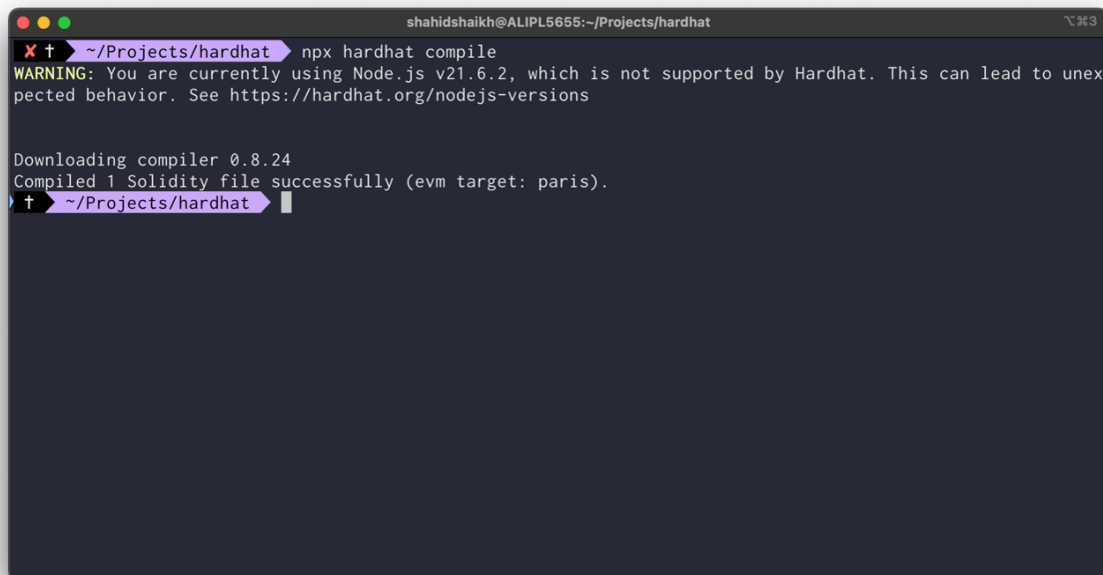
```
}  
}
```

Let us understand the program before moving ahead:

1. The first line tells the compiler which version of solidity to use, it is important. Change the version of solc to your version if it is different from **0.8.24**, as shown above.
2. The second line declares a new contract and inside it contains different functions of the contract.
3. Let us compile this program by running this command:

\$ npx hardhat compile

It should return the following output in the console:

A terminal window with a dark background. The title bar shows 'shahidshaikh@ALIPL5655:~/Projects/hardhat'. The prompt is '~/.Projects/hardhat' followed by a purple arrow icon. The command 'npx hardhat compile' has been entered. The output shows a warning about Node.js version, followed by 'Downloading compiler 0.8.24' and 'Compiled 1 Solidity file successfully (evm target: paris)'. The prompt is now '~/.Projects/hardhat' with a cursor.

```
shahidshaikh@ALIPL5655:~/Projects/hardhat  
X + ~/.Projects/hardhat npx hardhat compile  
WARNING: You are currently using Node.js v21.6.2, which is not supported by Hardhat. This can lead to unex  
pected behavior. See https://hardhat.org/nodejs-versions  
  
Downloading compiler 0.8.24  
Compiled 1 Solidity file successfully (evm target: paris).  
+ ~/.Projects/hardhat
```

Figure 2.12: Hardhat compile command response

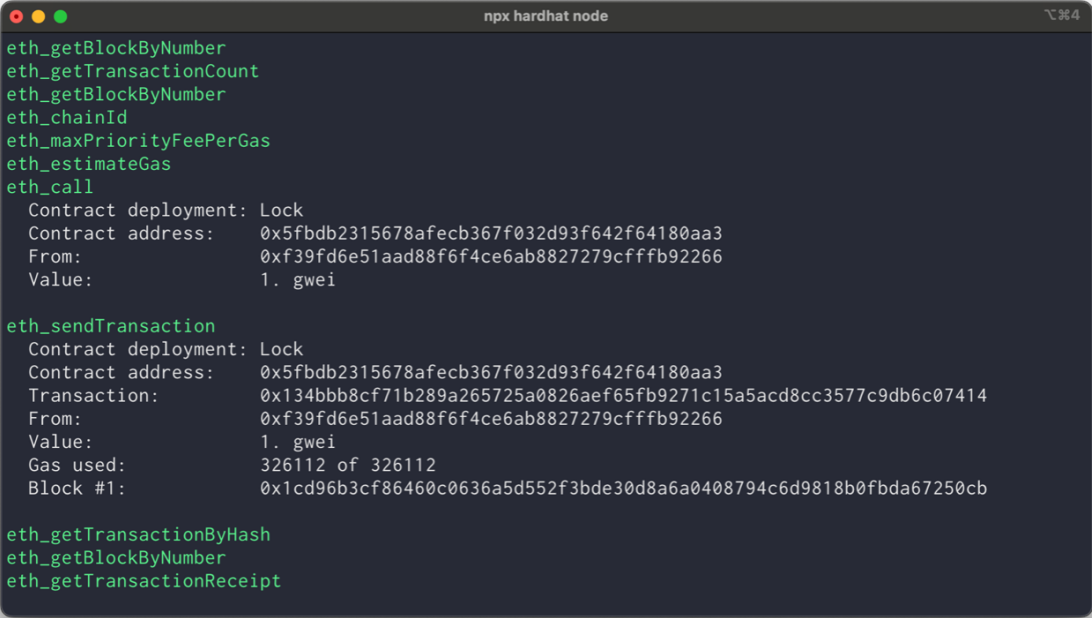
4. Now, let us deploy the contract. In a new terminal window, run this command to run a local Ethereum node to deploy smart contract.

\$ npx hardhat node

5. Now, let us deploy the compiled contract to the local blockchain node. Run the following command to deploy it. Do ensure that you add localhost value in the network parameter.

\$ npx hardhat ignition deploy ./ignition/modules/Lock.js --network localhost

It will deploy our smart contract in blockchain and return the transaction ID. This command should print something like this at your end:



```
eth_getBlockByNumber
eth_getTransactionCount
eth_getBlockByNumber
eth_chainId
eth_maxPriorityFeePerGas
eth_estimateGas
eth_call
  Contract deployment: Lock
  Contract address: 0x5fbdb2315678afecb367f032d93f642f64180aa3
  From: 0xf39fd6e51aad88f6f4ce6ab8827279cfff92266
  Value: 1. gwei
eth_sendTransaction
  Contract deployment: Lock
  Contract address: 0x5fbdb2315678afecb367f032d93f642f64180aa3
  Transaction: 0x134bbb8cf71b289a265725a0826aef65fb9271c15a5acd8cc3577c9db6c07414
  From: 0xf39fd6e51aad88f6f4ce6ab8827279cfff92266
  Value: 1. gwei
  Gas used: 326112 of 326112
  Block #1: 0x1cd96b3cf86460c0636a5d552f3bde30d8a6a0408794c6d9818b0fbda67250cb
eth_getTransactionByHash
eth_getBlockByNumber
eth_getTransactionReceipt
```

Figure 2.13: Deployment of smart contracts

You have successfully created, compiled, and deployed a smart contract in Ethereum blockchain.

We have covered Solidity in brief in this section. It will help you gain experience in the static type language with state management.

In the next section, we will learn about the DApps framework by Ethereum.

Decentralized Applications using Ethereum

In this section, we will explore DApp development using the Ethereum platform. A DApp built with Ethereum is an application that executes smart contracts in the backend and hosts frontend in the decentralized network.

In a nutshell, a DApp using Ethereum is composed of the following:

- **Frontend:** HTML, CSS, JavaScript, and static files
- **Backend:** Smart contracts, blockchain

An application may require more than just smart contracts, and Ethereum provides the following projects to cover these needs:

- **Swarm:** Decentralized storage network.
- **Whisper:** Dynamic communication protocol.
- **Smart contracts:** Turing complete programming language and interface:

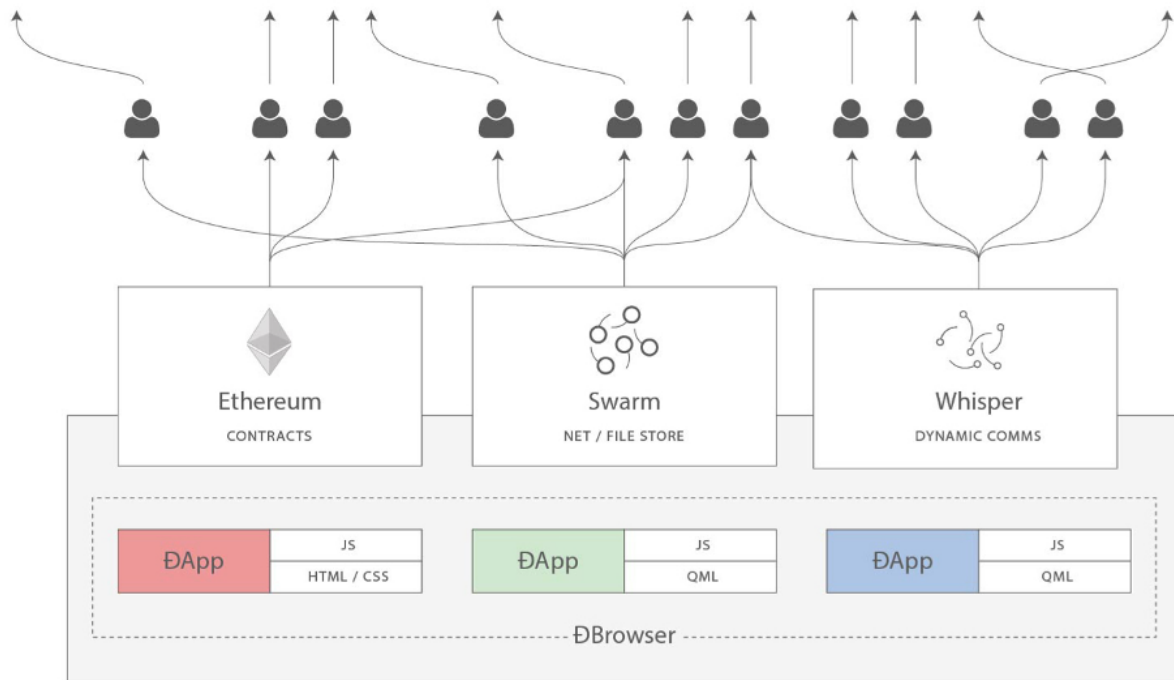


Figure 2.14: Decentralized web stack by Ethereum

Consider the following aspect of an application that may be decentralized:

- Frontend
- Backend
- Storage
- Communication
- Name resolution

Each of these aspects can be somewhat centralized or decentralized. For example, frontend can run as an independent centralized server, while backend is a smart contract running on blockchain. Storage can be used as decentralized, while name resolution can be a centralized service.

At this moment, there is no hard and fast rule that all aspects of the application must be decentralized. You must

use blockchain to put business logic in check with decentralization, and the rest can mix up with centralized service to improve user experience. Again, it is not necessary either.

Let us check out the advantages of the DApps:

- **Resiliency:** Business logic of the application is written in the smart contract, which, in turn, executes in blockchain. Unlike an application deployed on a centralized server, a DApp will have no downtime due to the network strength.
- **Transparency:** Any interaction with DApp is stored in the blockchain. Anyone can check that in the chain and be sure about the code. DApp code, execution, and result are completely transparent with zero censorship.
- **Censorship resistance:** As long as the network is up, anyone can access and use the application running on the chain. There is no stopping, censorship, and permission required to access the application. It is necessary right now, since, everything in the tech world is getting censored.
- **Trustless system:** Unlike in centralized applications where users trust companies to store their data in the safest possible way, everything is end-to-end cryptographically secured in DApps.
- **Security:** Due to the use of end-to-end cryptography, DApps are secure and solid. Nothing can be accessed, except by human error, as machines and networks do not leak any data.
- **No single point of failure:** Due to the decentralized nature of the application, there is no single point of

failure present in the system. Unless there is a node in the network alive, the application will work.

- **Payment:** Since DApps runs on top of the Ethereum blockchain, there is no need to perform third-party payment integration. Payment is facilitated in the crypto coins and recorded in the chain.

As you can see, DApp points to some of the core issues that we are facing right now. Censorship, financial frauds, documents tampering, and elections rigging are some of the common issues currently being faced. DApps and blockchain address these issues and help us evolve to the next version of the Web, that is, Web 3.0—internet by the people, for the people.

Let us look at the development stack of DApps.

DApps stack

The DApps stack in Ethereum comprises of the following:

- Frontend (HTML, CSS, JavaScript)
- Smart contracts
- Data storage (IPFS and Swarm Bee)
- Decentralized message communication (Whisper)

Let us understand them in detail.

Frontend

The frontend of DApps can be written in the standard technologies that have been used over the decades to build websites and applications.

Unlike the backend, where one needs to understand how Ethereum works and write code in Solidity, the frontend

can be written in HTML, CSS, and JavaScript, which are widely used to build web applications.

This allows traditional web developers to use familiar technologies and tools, frameworks, and libraries. The communication to blockchain can be carried out via API or RPC calls.

Ethereum also developed a JavaScript library called **web3.js**, which is used to bundle the frontend with the backend of the DApp.

Web3.js provides an easy and familiar way for traditional web developers to build decentralized apps that can communicate with the blockchain via the browser.

Let us look at the backend, that is, smart contracts stack.

Smart contracts

In DApps, smart contracts are used to write the business logic and related functions of your application. You can think of a smart contract replacing a server-side (aka backend) layer in a traditional application.

Developers must keep in mind that any computation executed in a smart contract is expensive in terms of ethers and should be kept to a minimum. So, it is important to identify the aspects of the application that need to run on a decentralized platform:

- Smart contracts let you build applications in which a network of smart contracts call and pass data between each other, reading and writing their state variables as they go.
- A major point to note in the smart contract architecture design is the inability to change the code

of a contract once it is deployed.

- A smart contract can be deleted if it is programmed with an available self-destruct opcode, but other than that, the code cannot be changed in any way.
- Another major point to note is that the size of DApps should be as light as possible. A large smart contract may cost way too much ethers to execute, making it inefficient.
- As a developer, you should only keep business logic in the smart contract. Storage can be either dealt with centralized services or using Swarm or IPFS.

Let us look over the storage stack and understand how to keep the storage part away from the smart contract so that we avoid building a very large contract that costs a fortune to run in the blockchain.

Storage

Due to high costs to execute contracts, smart contracts are not a good option for storing or processing large amounts of data such as photos, videos, and so on.

So, DApps should utilize off-chain data storage services, meaning they store the bulky data of the Ethereum chain on a data storage platform.

The data storage platform can be centralized (for example, a typical cloud database like *Amazon* or *Azure*), or the data can be decentralized, stored on a P2P network such as the IPFS or Ethereum's own Swarm platform.

Decentralized P2P network storage is ideal for storing and distributing large assets such as images, videos, and static resources such as normal files.

Let us look over the options we have at the time of writing this book to use as decentralized P2P storage.

IPFS

IPFS is a decentralized P2P network to store and share content. IPFS is a project by the protocol labs team and uses content addressing; so, a unique hash identifies each file added in the network. You can retrieve the file using the hash from any node in the network.

IPFS aims to replace the HTTP protocol for standard content delivery for the Web. IPFS has a network, and anyone can become a part of it, just like in Bitcoin or Ethereum.

In the next chapter, we will learn about IPFS in-depth. Let us check our next decentralized P2P network storage.

Swarm Bee

Swarm Bee is a project by *Ethereum Foundation*. It is a content-addressable P2P storage system very similar to IPFS, and it allows you to host and access a website using the network.

Let us run a simple Swarm Bee instance and check it for ourselves.

You can download Bee from the official website for your desired operating system from the following link:

<https://docs.ethswarm.org/docs/bee/installation/install>

The download and installation instruction for different operating systems are mentioned in the documentation link mentioned above. For Mac, the installation commands use

homebrew. Please make you have homebrew installed in your Mac machine.

Here is the installation command.

brew tap ethersphere/tap

brew install swarm-bee

If you want to run bee from the default configuration, then you can use the following command:

\$ brew services start swarm-bee

We can also use Swarm Desktop to run it easily in our machine for testing.

Head over to **<https://www.ethswarm.org/build/desktop>** and download the Swarm Bee desktop for your operating system.

Once the Swarm Bee Desktop is installed and running, you can visit the Swarm Bee gateway from the browser by navigating to **localhost:3054/dashboard**.

You should see a screen similar to the following figure:

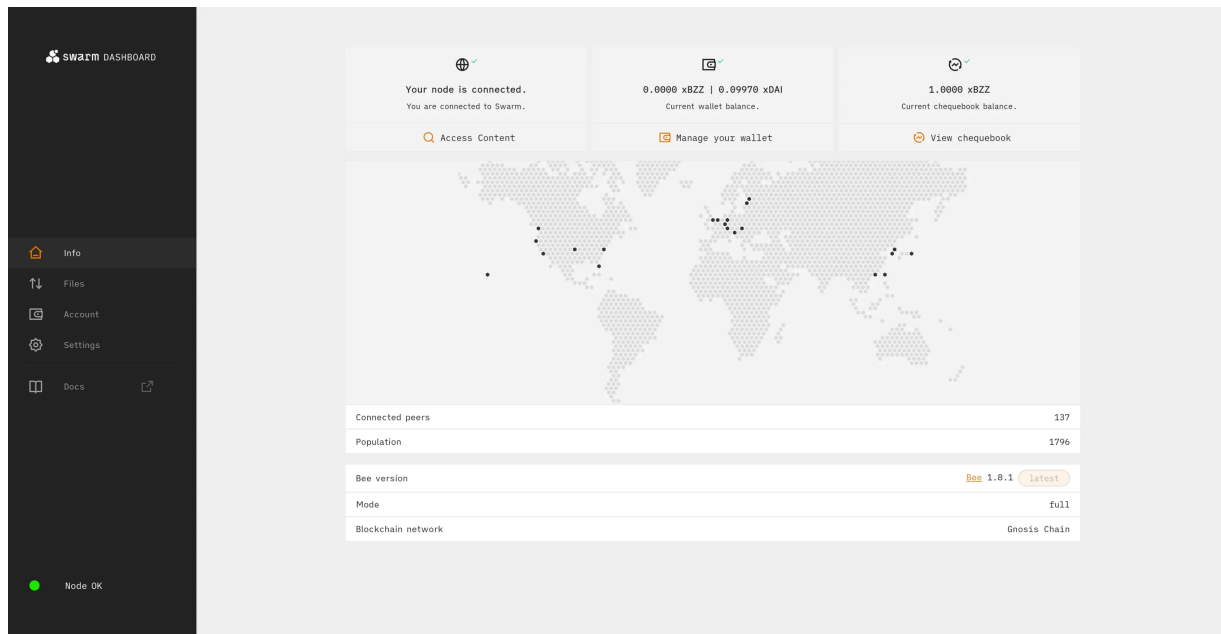


Figure 2.15: Ethereum Swarm Bee gateway

In order to upload the files and access it, we need to connect our wallet to fund cryptocurrency xBZZ or DAI and use it. IMG

If you go to **Settings**, we can find out the blockchain it is connected to and this can be changed to custom blockchain nodes as well:



Figure 2.16: Swarm Bee configuration

Swarm Bee is production ready and can be used as a decentralized file storage system.

Alright, let us visit the last part in the DApps stack.

Decentralized message communications protocols

One of the crucial components in any distributed application is inter-process communication. The application must be able to communicate with other instances via some medium to exchange data back and forth while maintaining the state.

In conventional applications, message queues and inter-process channels were the way to handle such communications. These message queues are central in nature and introduce another single point of failure in the system.

In DApps, communication is done via the P2P messaging protocol. At the time of writing this book, the most notable and widely accepted communication protocol for DApps is the Waku - <https://waku.org/about/>.

Ethereum also provides a service called **Ethereum Name Service (ENS)**. Developers can use ENS to point the hash-based resources to more friendly human-readable names like how we do in DNS services.

Conclusion

In this chapter, we have explored Ethereum in detail. We studied the vision and platform and how one can use Ethereum. We also studied various parts of Ethereum, such as EVM and smart contracts, and we built a simple application to see how smart contracts work. We looked at building DApps using Ethereum, and we explored the various protocols that are required to build a DApp.

In the next chapter, we will learn about IPFS in-depth. We will build an application that uses IPFS and deploy it to see how it works in real-time.

Join our book's Discord space

Join the book's Discord Workspace for Latest updates, Offers, Tech happenings around the world, New Release and Sessions with the Authors:

[**https://discord.bpbonline.com**](https://discord.bpbonline.com)



CHAPTER 3

Blockchain Consensus Algorithms

Introduction

In this chapter, we learn about consensus in blockchain which refers to the process in which all participants in a distributed network agree on the validity of a transaction or a set of transactions.

The primary goal of consensus in blockchain is to achieve a common and unambiguous view of the distributed ledger, ensuring that all nodes in the network have the same information about the transactions that have occurred. This is crucial for maintaining the integrity and security of the blockchain.

Consensus mechanisms play a critical role in addressing the following challenges in a decentralized environment:

- **Double spending:** Preventing the same digital asset from being spent more than once.

- **Security:** Ensuring that malicious actors cannot manipulate or compromise the integrity of the blockchain.
- **Decentralization:** Distributing decision-making power across the network, eliminating the need for a central authority.
- **Trust lessness:** Allowing participants to transact with each other without the need for trust, as the consensus mechanism ensures the validity of transactions.

Structure

In this chapter, we will cover the following topics related to consensus algorithms:

- Proof of Work consensus
- Proof of Stake algorithm
- Delegated Proof of Stake
- Fault tolerance algorithms
- Practical Byzantine Fault Tolerance consensus algorithm
- Tendermint Core

Objectives

After studying this chapter, you should be able to understand consensus algorithm(s) and its usage and different types of Algorithms and their applications.

Let us start with the first consensus algorithm called **Proof of Work (PoW)**.

Proof of Work consensus

PoW is one of the fundamental consensus algorithms to perform validation in the decentralized network. We have briefly covered the PoW consensus algorithm in *Chapter 1, Introduction to Blockchain and Decentralized Network*, and in this section, we will cover it in detail.

As the name suggests, PoW requires proof of the computational effort to prove to other participants that the work is legitimate. PoW was invented by *Moni Naor* and *Cynthia Dwork* in 1993 to prevent **denial of service (DoS)** attacks by validating and requiring some work from the service requester (work means computational effort).

PoW algorithm was re-invented by *Satoshi Nakamoto* in the Bitcoin whitepaper where *Satoshi* explained the concept of consensus and how we can validate transactions in the decentralized network.

Let us begin with the working of PoW consensus algorithm in detail. At this point, we assume you have read *Chapter 1, Introduction to Blockchain and Decentralized Network*, and understood the basic terminologies of decentralized networks and cryptography.

Bitcoin transactions are stored and assembled in an entity called **block**. Each block contains transactions, and every node contains a copy of the block. Now, to submit the block to the main chain, each node must perform a mining activity, where they need to find the hash lower than the target hash set by the network.

One finds the hash through a random algorithm, which is executed by the Bitcoin program. The key role in this process is played by hardware strength, as a faster CPU or GPU can randomize quicker than a normal computer.

The problem is directly related to the hardware strength of the network. The higher the power of the network, the more difficult the problem, thus ensuring fair participation in the network.

The minted block in the Bitcoin blockchain network is shown in the following figure. Notice the nonce and the difficulty. Due to the powerful network, the difficulty is also increased proportionally:

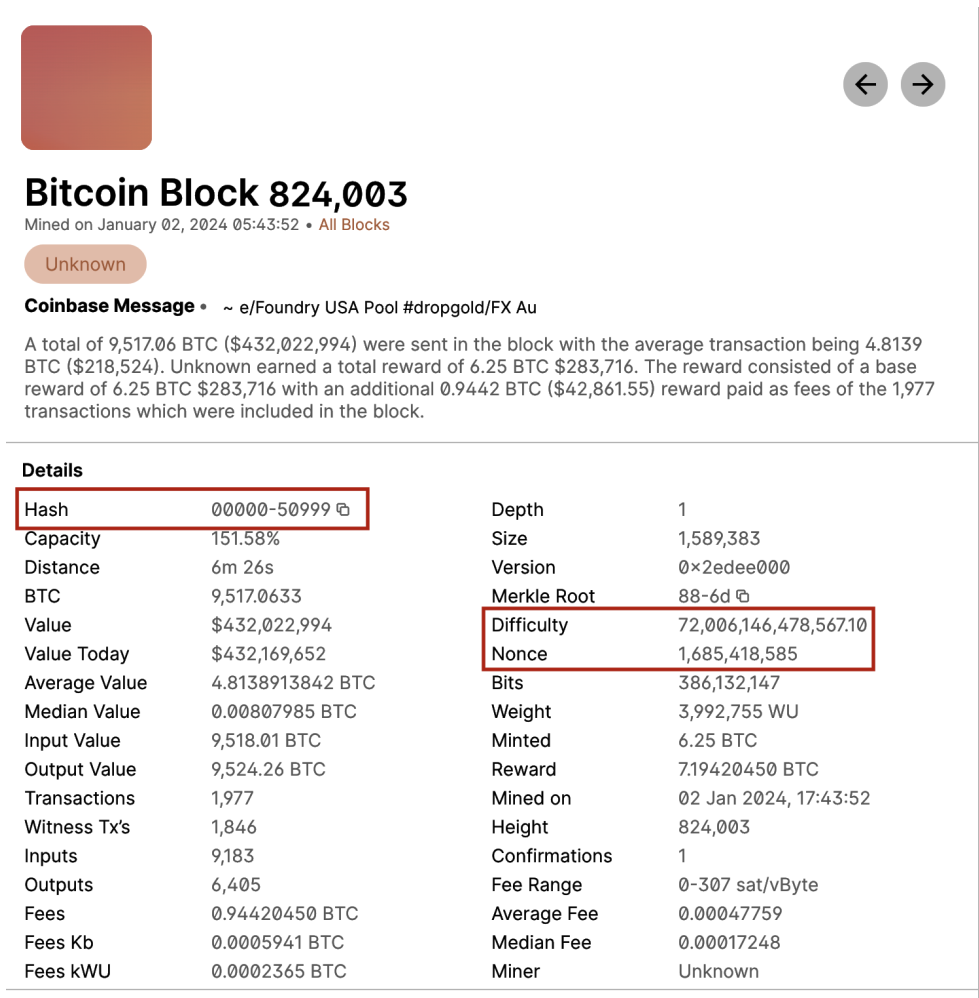


Figure 3.1: Blockchain block information

Upon successful mining, the miner will produce a number using which other nodes can verify if the hash is correct or not. Once it is validated, the block is submitted to the

network, and miner earns some bitcoin for their effort. When writing this, 3.125 Bitcoin (~68000 US dollars) was the reward for successful mining.

Here is one sample of the GPU lined together and the Bitcoin mining program running on the computer:

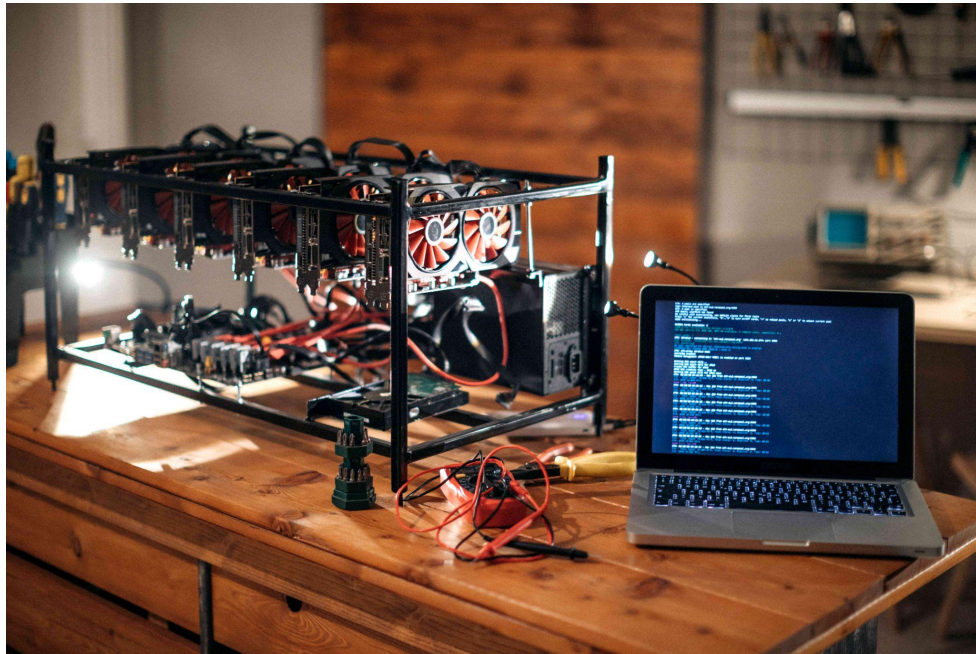


Figure 3.2: GPUs used in the mining

While PoW consensus algorithm ensures a strict validation of the transactions and blocks, it also presents some serious challenges.

Proof of Work challenges and limitations

PoW has the following challenges:

- **51% attack:** This is the only security risk Bitcoin and PoW possess, where if one owns 51% of the nodes, they can control the network. However, it becomes a challenging and cost-intensive operation to control 51% of the network.

- **Efficiency:** PoW is inefficient and takes time to compute the PoW hash. Generally, it takes around 10 minutes for blocks to submit to the network, and it can take longer based on the congestion.
- **Hardware and resource consumption:** PoW requires efficient hardware to calculate the number to generate a hash. Due to heavy usage, this hardware needs to be replaced very often, resulting in huge e-waste and higher demand for hardware such as GPUs.

To overcome these challenges and provide faster transaction settlement, a **Proof of Stake (PoS)** consensus algorithm was developed.

Proof of Stake algorithm

PoW requires miners to solve a problem and present proof as a hash to submit a block. PoS, however, requires nodes to stake some value to take the role of a validator to submit a new block.

The PoS algorithm bypasses the mining effort and focuses on providing validators transaction fees as a way to earn incentives for the effort. This ensures faster transaction throughput and less computational effort.

Let us understand more about the POS consensus algorithm.

The blockchain networks consist of nodes, which can become validators after offering their coins as collateral or stake. The collateral acts as security and ensures the validator will not harm the network. Once the node performs this activity, it becomes a validator.

Now, the validator can submit a block to the network, and every other node will agree. The validator will earn the

transaction fee for each transaction submitted in the block, ensuring a steady flow of currency in the network.

Validators are also selected on a random basis using Randao, so there is no partiality in the network. Based on the stake, the network randomizes who gets to collect the transaction fees rather than a competitive reward system like PoW in Bitcoin.

The amount of coin to be deposited to become a validator depends on the network. Ethereum requires 32 Ethereum coins to be staked to take the role of a validator.

This approach of validations solves some of the key issues which the PoW consensus algorithm has such as efficiency, less resource consumption, and security.

Ethereum is one of the major blockchain technologies and networks that uses a PoS consensus algorithm.

PoS is great in solving the peer-to-peer consensus problem but also introduces some challenges and drawbacks, some of which are listed in the coming section.

The challenges and limitations of PoS are:

- **Security:** PoS is not as secure as PoW because the validator can corrupt the network with malicious intentions. Networks and other validators can stop it, although it is not that easy or possible in PoW consensus.
- **Adaptability and testing:** PoS is not widely used or tested with scale. It sometimes fails when there is network congestion, and transaction fees become way higher to submit a block; sometimes fees are even higher than the transaction value.

- **Centralization of validators:** There is a huge risk involved with malicious validators to form a group and control the network. This leads to centralized control and manipulation of the network.

Due to the randomness of validator selection and centralization problems that may arise, we have a tweaked version of the algorithm called **Delegated Proof of Stake (DPoS)**. Let us learn about it in the following section.

Delegated Proof of Stake

DPoS is the tweaked version of PoS consensus where the validator is selected based on the voting and election combined with the collateral stake mechanism.

The voting and election mechanism ensure there is a democratic process to choose validators and is not done on a random basis as in the traditional PoS consensus.

Let us understand how the voting and election mechanism works in the DPoS. Refer to the working diagram given in the following figure:

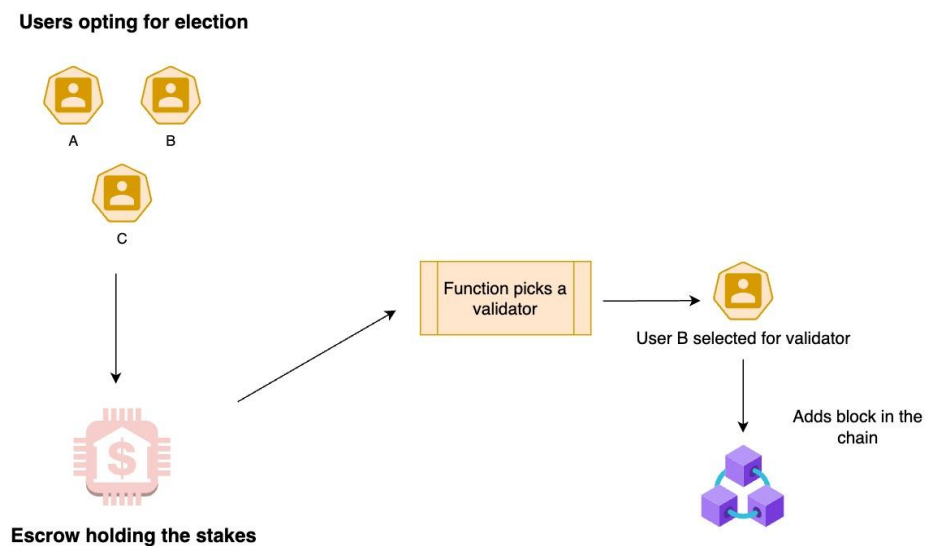


Figure 3.3: How PoS works

To select a validator who is going to validate the next block in the network, the nodes in the network stake their coins to nodes whom they want to become validators.

The node with the highest number of coins staked by the users wins the election and validates the next block as well as earns transaction fees. The transaction fees earned by the validator are then distributed to the nodes who supported it based on the stake coins they deposited during the election process.

DPoS solves some critical problem(s) but also offers some limitation and challenges, we have listed some of the crucial challenges in the next section.

The limitations of DPoS are:

- **Centralized nature:** Due to the user's participation in the validator election, staking their coins to pick new validator is a technical concern of centralized approval. A group of users can and always select a set of validators of their choice.
- **Security:** The limitation listed previously becomes a security concern as a malicious delegate can perform a security breach. The validator can also perform 51% attack and take over network.

Fault tolerant algorithms

Fault tolerance is the key aspect of blockchain and distributed systems. In order to ensure robustness and reliability of the system in the event of failure, the system is able to handle it gracefully.

In the context of the blockchain and decentralized system, the fault tolerance is the responsibility of the consensus

algorithm to ensure that the system is up and running without data corruption.

There are two widely adopted consensus algorithms used for fault tolerance mechanism:

- **Practical Byzantine Fault Tolerance (PBFT)** consensus algorithm
- Tendermint Core consensus algorithm

Before we discuss these two algorithms, let us first understand the Byzantine Generals problem.

The Byzantine Generals problem was introduced in 1982 as a logical dilemma that explains how a group of Byzantine Generals may have communication problems when trying to agree on their next move in the war. Refer to the following figure:

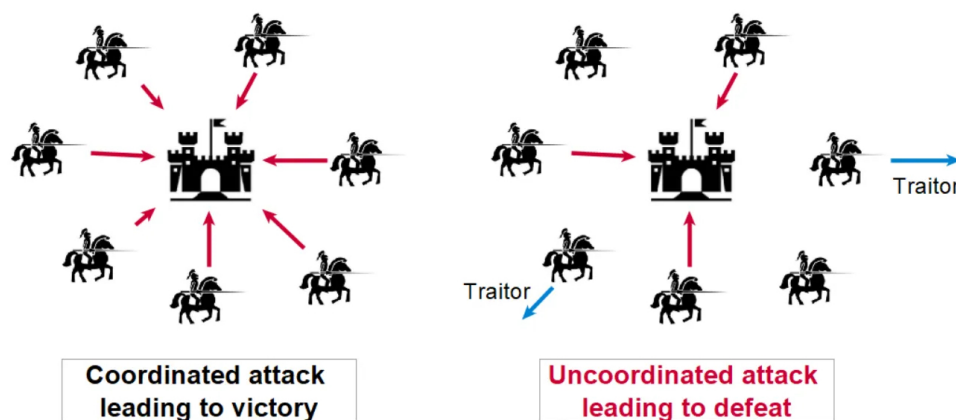


Figure 3.4: Byzantine Generals problem

The problem assumes that each general has its own army and that each army group is situated in different locations around the location they intend to attack. The generals need to agree on either attacking or retreating, depending upon the assessment of the situation.

The generals need to agree on the consensus in order to execute a common decision so that there is no dispute and army is not confused.

Therefore, we may consider the following requirements:

- Each general has to decide: attack or retreat (yes or no).
- After the decision is made, it cannot be changed.
- All generals have to agree on the same decision and execute it in a synchronized manner.

The aforementioned communication problems are related to the fact that one general is only able to communicate with another through messages, which are forwarded by a courier. Consequently, the central challenge of the Byzantine Generals problem is that the messages can get somehow delayed, destroyed or lost.

In addition, even if a message is successfully delivered, one or more generals may choose (for whatever reason) to act maliciously and send a fraudulent message to confuse the other generals, leading to a total failure.

If we apply the dilemma to the context of blockchains, each general represents a network node, and the nodes need to reach a consensus on the current state of the system. To put it in another way, the majority of participants within a distributed network have to agree and execute the same action to avoid complete failure.

Therefore, the only way to achieve consensus in these types of distributed systems is by having at least two third or more reliable and honest network nodes. This means that if the majority of the network decides to act maliciously, the system is susceptible to failures and attacks.

The **Byzantine Fault Tolerance (BFT)** is a system's ability to continue the operation in the event of a node failure or if some of the nodes act maliciously.

There are different ways to achieve BFT in your system. In the blockchain system, consensus algorithm such as PoW achieve BFT using mining and synchronization of nodes with the full data after a block is submitted successfully. PoS achieve the BFT by locking the wallet of validator with the stake coins and ensuring it is not released until the operation is completed.

Practical Byzantine Fault Tolerance consensus algorithm

One of the key requirements of the BFT is that system or nodes of the network needs to be in the synchronous state. This leads to performance issues especially in the decentralized system where nodes are distributed across geolocations and latency of the network is still high, resulting in the slower response compared to traditional web2.0 client/server model.

The other key requirement is that in the event of the byzantine fault present in the system, the system needs to figure out a way to work. PBFT algorithm is designed for such a use case, and we are going to learn it in this section.

Working of Practical Byzantine Fault Tolerance Algorithm

The PBFT uses machine state replication by assigning one node as primary and another node as secondary. PBFT is designed to work in the asynchronous state and can work

even when nodes are down or there is a malicious node in the network.

Nodes in a network running PBFT algorithm are assigned primary or leader node and secondary or backup nodes. Here, any node can become primary or secondary, and all of it works in a sequential manner.

The idea behind this node assignment is to achieve a consensus in the network by using the majority node. The PBFT can function as long as the count of malicious nodes or nodes that are not responding is lower than honest nodes or nodes working in the system. Following is the PBFT sequence figure:

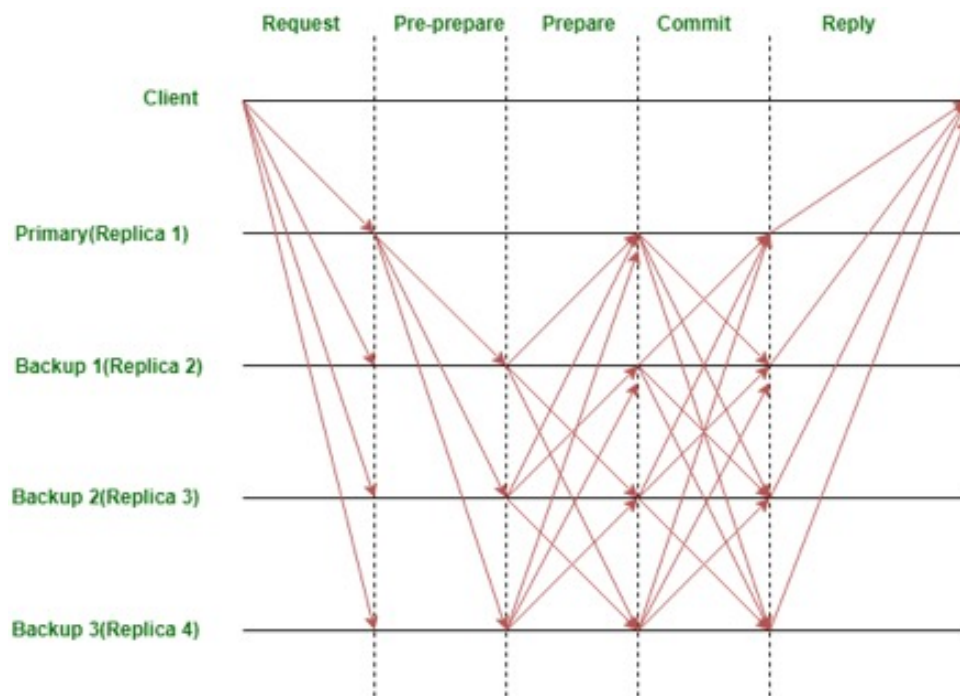


Figure 3.5: PBFT sequence diagram

Here are the four phases in which the PBFT assigns leader and backup nodes:

1. The client sends a request to the primary or leader node.

2. The leader node broadcasts the request to all the secondary nodes.
3. The primary and secondary nodes perform the service requested by the client and sends a response.
4. The request is served when client receives the response from different nodes with the same result. The replies should be $m+1$ where m is the maximum number of malicious nodes allowed.

One of the key limitations with the PBFT is the efficiency issue. The PBFT is efficient when the network is small because it uses lots of communication to perform the leader selection and maintain the system's reliability. Once the network is large enough, the latency increases along with it. The communication overhead problem is a key scaling challenge when implementing PBFT in the network.

One of the famous platforms that uses PBFT is Tendermint. We are going to learn about it in the next section.

Tendermint Core

Tendermint Core is the PBFT consensus engine that powers many blockchain networks and projects. Tendermint provides secure and consistent replication of an application state across multiple nodes in a network. Tendermint Core is often used as the underlying consensus layer for blockchain platforms and distributed ledger systems. Cosmos network is one of the popular blockchain platform network that uses Tendermint Core as the consensus engine.

Tendermint Core uses a PBFT inspired consensus algorithm to reach an agreement among a set of nodes in a distributed network. This consensus algorithm helps

maintain a consistent and secure state across all participating nodes. Tendermint consensus algorithm maintains the network upstate till one third of the nodes are honest.

Tendermint Core mandates that all honest nodes in the network agree on the same sequence of transactions. It achieves this through a process known as **atomic broadcast**, where each block of transactions is agreed upon by the validator nodes in a deterministic manner.

The validator nodes changes over time and the network allows for this to happen. The validators can be added or removed, and the system can adapt to changes in the network's composition based on the consensus.

Tendermint Core uses an **Application Blockchain Interface (ABCI)** to separate the consensus engine logic from the application logic. This allows developers to implement their blockchain application logic while using Tendermint Core for consensus. Due to this design choice, several blockchain projects use the Tendermint Core as the blockchain engine.

Due to ABCI, Tendermint Core is language agnostic, and it supports state machines in any programming language.

Tendermint Core can be used in the variety of applications area such as identity services, gaming, asset management, etc.

Setting up Tendermint Core

Let us set up a simple Tendermint Core in your machine. We are assuming that you have an Ubuntu Linux machine running with you, if not, you can download the pre-

compiled binaries directly from the GitHub repository with your choice of operating system. Tendermint is also available in the Docker image. Please refer to the following link for the same:

<https://github.com/tendermint/tendermint/releases>

Now, open the terminal in the Ubuntu and paste the following command to install Tendermint Core:

curl -L https://git.io/fFfOR | bash

source ~/.profile

The link in the command contains the installation commands and instructions. Wait for the installation to complete.

Once the installation is complete, we can initiate the Tendermint Core using the following command:

tendermint init

This will create some files in the **\$HOME** location. Please note these files will support single machine node network.

Now, let us start the Tendermint server:

tendermint node --proxy_app=kvstore

This command will initialize the server and you will see blocks popping up in the terminal.

Limitations of Tendermint Core

Some of the key limitations of Tendermint Core are:

- **Centralization:** Tendermint Core works with a fixed set of validators and changing the validators requires

the hard fork. This leads to not true but partial centralization of the network.

- **Scalability:** Tendermint Core uses PBFT, hence it inherits the communication overhaul problem which would come up whenever we are scaling the network. More number of nodes equals more communications which results in the high latency.
- **Resource requirements:** Tendermint Core uses more computation power and network bandwidth due to the use of PBFT consensus. This can add up in the cost of the running the operation.

Tendermint Core is improving and the community is trying to solve problems as they go. This is one of the key exciting projects in the blockchain and decentralized space.

Conclusion

In this chapter, we studied various consensus algorithms and their limitations. We studied how different consensus algorithms are designed for different problems. The best takeaway is that we need to identify the problem in the network and use the consensus algorithm which best fits the problem.

In the next chapter, we are going to study the scaling challenges blockchain and decentralized applications are facing and how we can solve them using different techniques.

Join our book's Discord space

Join the book's Discord Workspace for Latest updates, Offers, Tech happenings around the world, New Release and Sessions with the Authors:

[**https://discord.bpbonline.com**](https://discord.bpbonline.com)



CHAPTER 4

Blockchain Scaling Challenges

Introduction

Blockchain was built to accommodate the world's first cryptocurrency, i.e. Bitcoin. Bitcoin aimed to become an alternative mode of payment owned by the people for the people.

The sole purpose was to introduce an alternative to the current centralized mode of payment. However, the minute Bitcoin and other cryptocurrencies got together on a centralized exchange, and people started trading and making money, things started to change and attracted tons of people. This led to the crypto boom and the user size of the crypto grew by more than 100x.

Despite its innovative potential, blockchain encounters significant challenges in efficiently handling large-scale loads. These challenges manifest in various issues, notably heavy transaction fees, prolonged transaction durations, and systemic performance degradation.

The decentralized nature of blockchain nodes, operating within peer-to-peer networks distributed globally, introduces complexities such as heightened latency due to geographic dispersion, extended transaction processing times attributable to consensus algorithms like **Proof of Work (PoW)**, and escalating network propagation delays with the expansion of the peer-to-peer network.

As engineers, addressing these scaling challenges becomes imperative, necessitating the development of tailored solutions to enhance blockchain's operational efficacy.

In this interesting and important chapter, we are going to discuss and learn about various scaling techniques in Blockchain technology.

Structure

In this chapter, we will address the following topics:

- Blockchain scalability challenges
- Sharding
- Layer 2 approach
- Lightning networks
- Use cases and applications

Objectives

The objective is to address and learn the scalability challenges inherent in blockchain technology, aiming to enhance its utility and accessibility. Leveraging scaling techniques like sharding and layer 2 approaches such as Lightning Networks are pivotal strategies. Sharding entails the segmentation of the network into smaller, manageable segments, bolstering transaction throughput and

diminishing latency. Meanwhile, layer 2 solutions like Lightning Networks facilitate off-chain transactions, effectively reducing congestion on the primary blockchain. These initiatives pave the way for a myriad of applications spanning **decentralized finance (DeFi)**, supply chain management, digital identity verification, and voting systems, fostering innovation and operational efficiency across diverse sectors.

Blockchain scalability challenges

The consensus algorithms such as PoW became a bottleneck in the rapidly growing blockchain industry. PoW consensus algorithms could not handle the transaction demand, resulting in slow transactions and high transaction fees.

Peer-to-peer network growth also resulted in slower transaction and response time because the entire network and chain had to be updated, and propagation took time as every peer differed in latency, bandwidth, and network strength.

Each blockchain platform operates in isolation resulting in creating data silos limiting their potential. Blockchain platforms lack standardization and compatibility. Thus, interoperability between blockchain networks is one of the key problem areas.

Malicious code detection is one of the key challenges in the smart contract platform. Smart contracts are self-executing code, and malicious code results in serious issues such as loss of funds and theft.

Let us study some of the scaling challenges:

- **Network latency:** Transactions in decentralized systems require consensus from multiple nodes, which can lead to longer confirmation times and slower transaction processing.
- **Limited throughput:** Decentralized networks have a limited capacity for the number of transactions that can be processed per second, resulting in slow confirmation times during periods of high network usage.
- **Gas fees:** As the network becomes congested, the cost of executing smart contracts (in the form of gas fees) can become prohibitively expensive, making the platform less accessible for small transactions.
- **Resource intensity:** Decentralized systems often require high computational power and storage resources, which can be a barrier to scalability.
- **Scalability bottlenecks:** Some smart contract platforms may be limited by scalability bottlenecks in their underlying infrastructure, such as the number of nodes participating in the network.

To address scalability, these are some of the widely adopted approaches:

- **Sharding:** A sharding solution would divide the network into smaller sub-networks, allowing each sub-network to process transactions in parallel, improving overall throughput.
- **Layer 2 scaling solutions:** This involves adding a layer on top of the main blockchain network to handle a larger volume of transactions, such as state channels and plasma.
- **Lightning Network:** Lightning Network is a layer 2 payment protocol built on top of the Bitcoin

blockchain. It enables faster and cheaper transactions by allowing users to make off-chain payments directly between each other without broadcasting every transaction to the network.

- **Optimizing consensus algorithms:** This involves improving the existing consensus algorithms or developing new ones that are more efficient and scalable. We studied about consensus algorithms in detail in the *Chapter 3, Blockchain Consensus Algorithm*.
- **Hybrid systems:** Combining centralized and decentralized systems to take advantage of the strengths of both, thereby improving scalability and security, for example cryptocurrency exchange.
- **Interoperability:** Developing protocols and standards for decentralized systems to exchange information and assets, enabling them to work together seamlessly.

Sharding

Sharding is a scaling technique used in blockchain networks to improve scalability and performance by dividing the network into smaller, more manageable parts called **shards**.

Each shard operates independently, processing its own transactions and smart contracts. Sharding is designed to address the limitations of traditional blockchain networks that process all transactions in a linear manner, leading to potential bottlenecks.

As you can see in the following figure, the network consists of 1000 nodes and is trying to process the transactions. In sharding, the transactions are divided into multiple sets of

smaller units (as shown in the figure, each shard consists of 100 nodes) and they manage the transaction assigned to them:

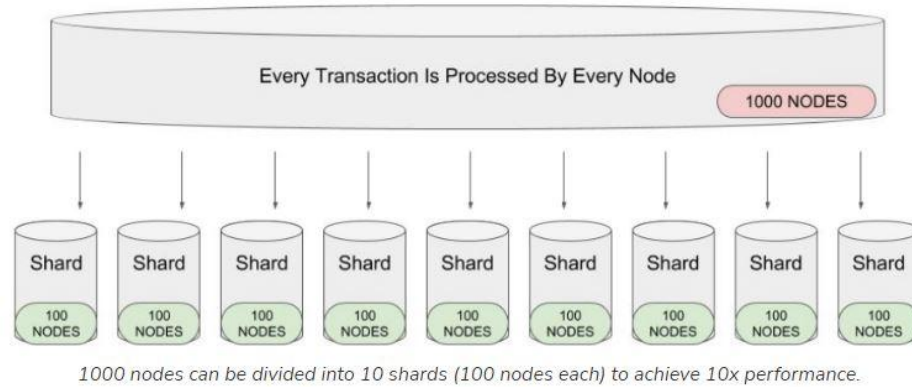


Figure 4.1: Sharding of nodes

This improves the efficiency of the network. If done properly, it can be improved to achieve more than 10x performance in transaction handling.

Shards process transactions independently, reducing the contention for resources often occurring in non-sharded networks. Nodes within a shard are responsible for validating and reaching consensus on transactions within that shard.

Shards need to communicate with each other when a transaction involves parties from different shards. This requires mechanisms for cross-shard communication. Inter-shard communication can be achieved through protocols that enable secure and efficient transfer of information between shards.

Advantages of sharding

Let us learn and understand the advantages of using sharding in the blockchain network:

- **Optimal throughput increase:** Sharding significantly increases the throughput of the blockchain network. With multiple shards processing transactions simultaneously, the overall capacity of the network is enhanced. Throughput also scales with the number of shards, allowing the network to handle a larger number of transactions per second.
- **Reduced latency:** Sharding reduces transaction confirmation time by distributing the workload across multiple shards. Transactions processed in different shards can occur simultaneously, leading to lower latency as compared to non-sharded networks.
- **Horizontal scaling:** Sharding is a key technique for achieving horizontal scalability in blockchain networks. As the network grows, additional shards can be added to accommodate the increased transaction volume. The ability to scale horizontally is crucial for blockchain networks aiming to support a large and diverse user base.

Sharding offers some crucial advantages, but it comes with its own challenges. Let us have a look at it:

- **Security:** Ensuring the data is secure across different shards is a crucial challenge. As the network is spanned across multiple geographical locations, ensuring all shards follow the same protocol while doing the cross-shard communication is a challenge.
- **Consensus across shards:** As the network size increases, the shards count will increase as well. To ensure data integrity and consistency, the shards need to ensure proper and smooth data synchronization.
- **Cross-shard communication:** Shards need to communicate with each other to ensure consensus

and data consistency. This requires communication, and it must be done in a way where the system does not become heavily dependent on each other as it will contradict with the nature of blockchain systems.

Ethereum 2.0 blockchain network uses sharding heavily to handle the scaling challenges Ethereum network is facing currently.

Now, let us explore the layer 2 approach and how it helps to solve scaling challenges.

Layer 2 approach

Layer 2 scaling solutions are approaches designed to address scalability challenges in blockchain networks by moving some transaction processing or data off the main blockchain.

These solutions aim to improve transaction throughput, reduce latency, and minimize congestion on the primary blockchain. By handling transactions off-chain or through a secondary layer, layer 2 solutions can significantly enhance overall performance and efficiency of blockchain networks.

Please refer to the following figure:

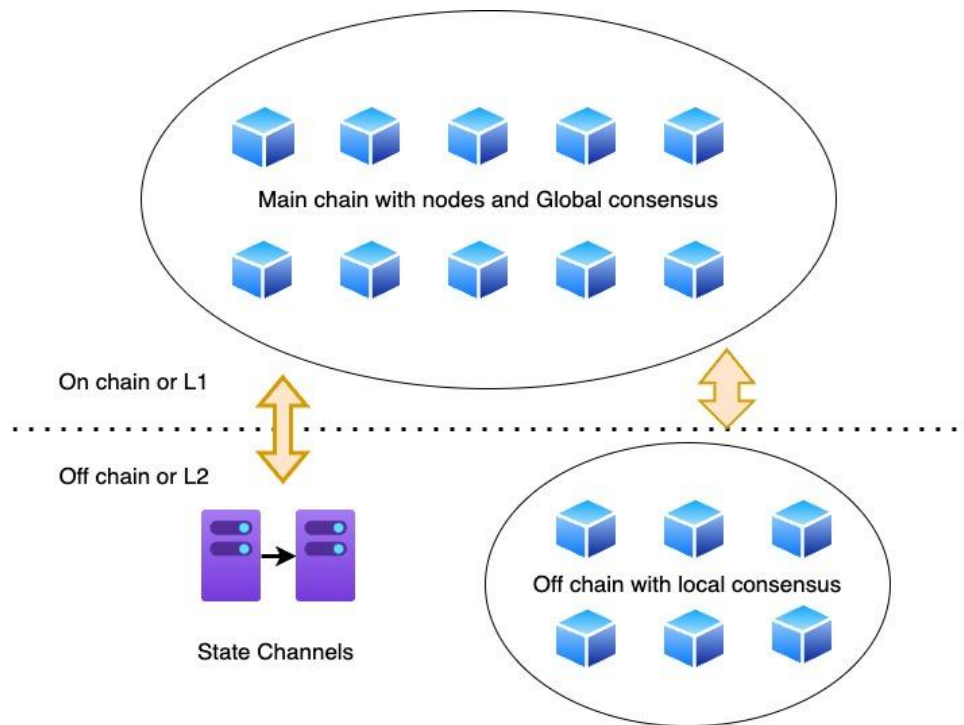


Figure 4.2: Layer 2 chain segregation

As you can observe in the *Figure 4.2*, layer 2 system works by performing settlement of transactions offline, and there are different ways we can achieve the same. Some of them are as follows:

- **State channels:** They are off-chain channels where multiple transactions can be settled without involving the mainchain.
- **Payment channels:** They are similar to state channels but exclusively designed for payments.
- **Sidechains:** These are separate chains with interoperability with the mainchain, and assets can move between chains.
- **Plasma:** It is a framework for creating a scalable blockchain, often referred to as **child chains**, which are connected to the mainchain.

- **Rollups:** It is a concept where summaries of all transactions are submitted to the blockchain and not all the transactions.

Let us discuss these layer 2 techniques in detail.

State channels

State channels are a layer 2 scaling solution designed to address scalability challenges in blockchain networks by enabling off-chain transactions.

These channels allow participants to conduct multiple transactions privately, off the main blockchain, and only settle the final state on the blockchain when necessary.

State channels are particularly useful for use cases involving frequent and rapid transactions between a set of participants.

Please refer the following figure:

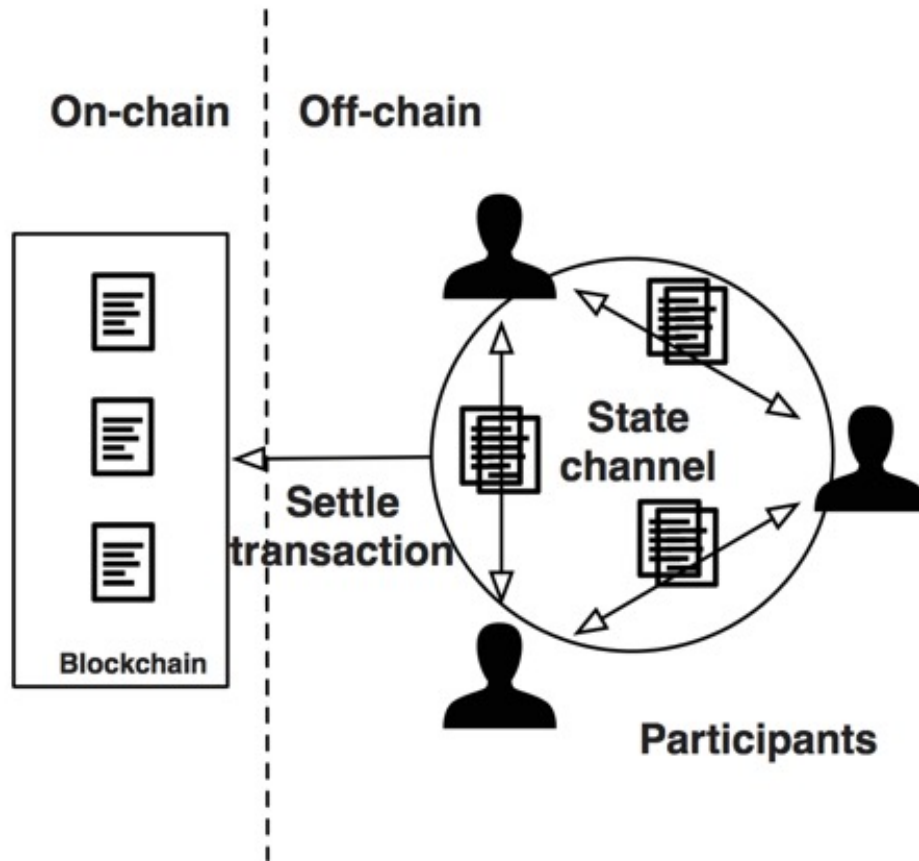


Figure 4.3: State channels

State channels operate in the following manner:

1. **Opening a state channel:** The network peers who wants to ensure collaboration and share data with each other needs to first open a state channel. The process to open a state channel is by creating a multi-signatory wallet between the participants.

Once the state channel is created, it is recorded in the blockchain.

2. **Off-chain transactions:** Once the state channel is open, participants can conduct a series of off-chain transactions between themselves without involving the main blockchain.

Each off-chain transaction updates the state of the channel and is recorded similarly to how it is recorded in the mainchain.

3. **State channel updates:** As the transactions get settled in the off-chain, the state channel created by the participants is updated. These updates are off-chain updates and do not sync with the mainchain yet.
4. **Closing/settling the channel:** Participants can close the state channel at any time by submitting the latest agreed-upon state to the blockchain. The blockchain verifies the final state, and the distribution of assets is settled according to the latest state agreed-upon by the participants. This gets synchronized with the mainchain as well.

State channels are used in the micropayment where payments are not high in volume. Since state channels are often used in payments, there is a special design of state channels exclusively for payments.

We will discuss about payment channels in the coming section.

Payment channels

Payment channels are a specific type of state channel used in blockchain networks to enable fast and scalable transactions, particularly for micropayments.

These channels allow participants to conduct a series of off-chain transactions between themselves, reducing the need for every transaction to be recorded on the main blockchain.

Payment channels offer benefits such as low transaction fees, increased transaction throughput, and reduced latency.

Here is how a payment channel works:

1. **Opening a payment channel:** Two parties decide to transact with each other frequently and opt to open a payment channel. This involves creating a multi-signature wallet (usually a 2-of-2 or 2-of-3 multi-sig address) on the blockchain. The opening transaction is recorded on the blockchain, establishing the initial state of the channel.
2. **Off-chain transactions:** Once the payment channel is open, the two parties can conduct off-chain transactions between themselves without broadcasting each transaction to the blockchain. Each off-chain transaction is signed by both parties involved in the channel.
3. **Updating the channel state:** Over time, the participants may engage in multiple transactions within the payment channel. The net effect of these transactions is continuously updated in real-time within the channel. Both parties must agree on the new state of the channel after each transaction. The latest agreed-upon state is reflected in the most recent signed transaction.
4. **Closing the payment channel:** At any point, either party can choose to close the payment channel. To do this, they broadcast the latest signed transaction to the blockchain. The closing transaction reflects the final state of the payment channel, including the net outcome of all the off-chain transactions. It is then confirmed on the blockchain, and the channel is considered closed.

5. **Settling balances:** Once the closing transaction is confirmed on the blockchain, the final balances are settled. The blockchain's smart contract ensures the funds are distributed according to the most recent agreed-upon state within the payment channel.

This is how payment channels operate in the blockchain. They inherit majority of the working methodology from state channel but exclusively for payments.

In the next section, we are going to learn about the sidechains.

Sidechains

As the name suggests, sidechains are a concept in blockchain technology that involves the creation of separate blockchains that can operate alongside the main blockchain (mainchain), and both chains are connected with each other.

Please refer to the following figure:

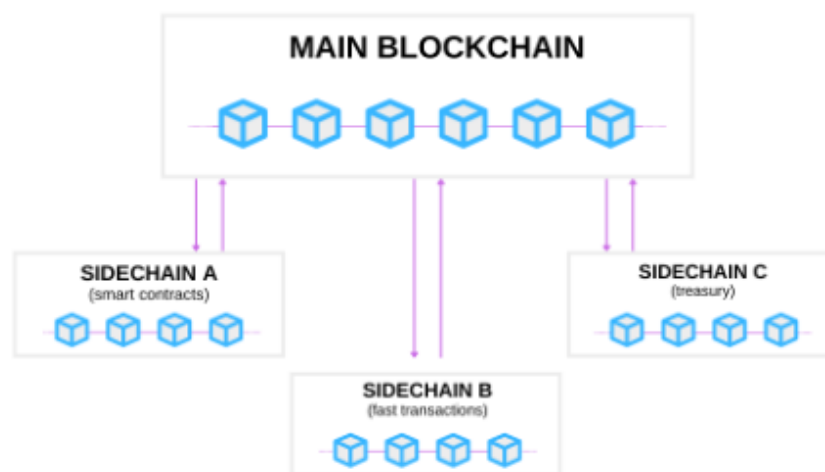


Figure 4.4: *Sidechains working with main blockchain*

The idea is to enable the efficient transfer of assets or data between the mainchain and the sidechain, making the transfer secure, interoperable, and secure.

Let us discuss some important properties of sidechains:

- **Parallel chains:** Sidechains run in parallel to the main blockchain but have their own set of rules and features. They are independent blockchains, as you can see in the preceding figure, and can have different consensus mechanisms, governance structures, and block generation times.
- **Two-way pegging:** This is a mechanism by which sidechains are often connected to the mainchain. It allows assets to be transferred from the mainchain to the sidechain and vice versa.
- **Interoperability:** Sidechains aim to bring interoperability between multiple blockchain networks using different mechanisms. This is one of the key design and architecture goals of sidechains. Users should be able to move assets or data seamlessly between the mainchain and various sidechains, enabling more flexibility and scalability.

When different systems collaborate, there is always a chance of security risks. This emphasizes the importance of closely examining security details. Due to interconnectivity and data exchange, security of sidechains is crucial, as any vulnerabilities could impact the overall security of the entire network.

Two-way pegging and consensus mechanisms must be carefully designed to prevent attacks and ensure the integrity of the interconnected blockchains.

Bitcoin Liquid Network is an example of a sidechain designed to facilitate faster and confidential transactions between exchanges. Another example is the **Rootstock (RSK)** sidechain for the Bitcoin network, which enables the execution of smart contracts on the Bitcoin blockchain.

In summary, sidechains offer a way to extend the functionality of a blockchain network by creating separate chains with specific features, allowing for experimentation, scalability, and interoperability while maintaining a connection to the main blockchain.

Plasma framework

Plasma is a framework for scaling blockchain networks by creating hierarchical structures of interconnected sidechains, also known as **child chains**. It was proposed by *Joseph Poon* and *Vitalik Buterin* in a whitepaper in 2017.

The primary goal of the plasma framework is to enhance the scalability and efficiency of blockchain systems.

Please refer to the following figure:

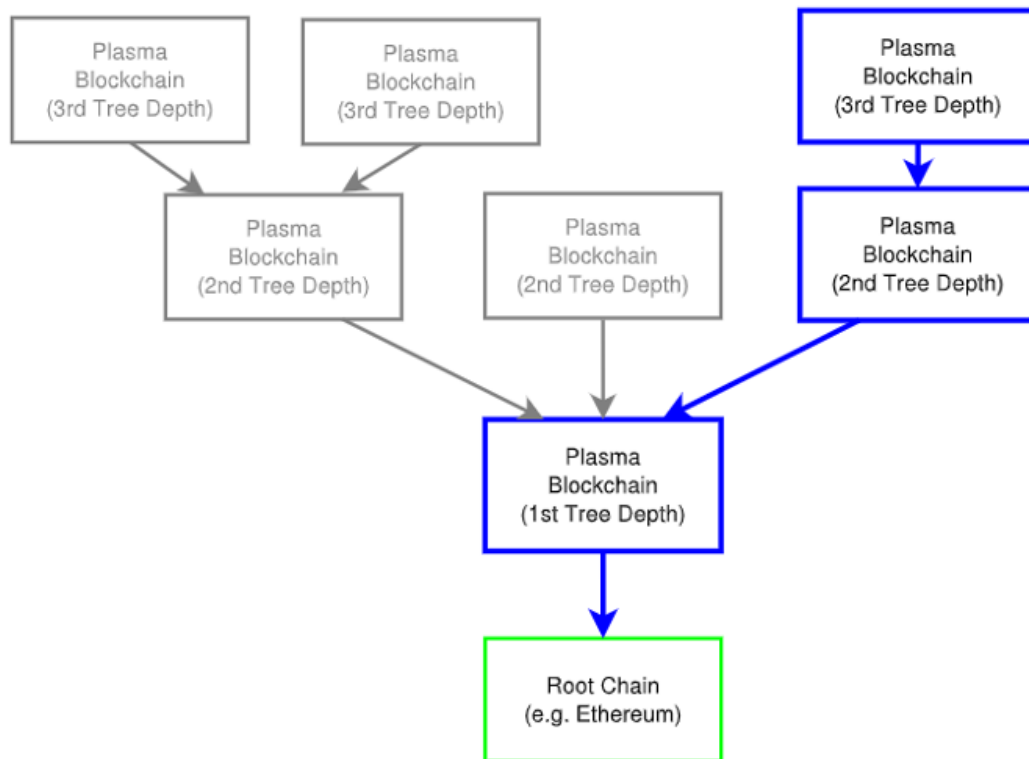


Figure 4.5: Plasma chain hierarchy

Here are the key components and concepts of the plasma framework:

- **Root chain (Mainchain):** The mainchain, also referred to as the **root chain**, serves as the primary blockchain with its consensus mechanism (e.g., PoW or **Proof of Stake (PoS)**). It is responsible for the overall security of the plasma framework.
- **Child chains (Plasma chains):** Child chains are individual blockchains that operate independently but are connected to the root chain. Each child chain can have its own rules, consensus mechanism, and features. These chains handle transactions and smart contracts, providing a way to scale the transaction throughput.

- **Plasma operator:** The entity responsible for managing the child chain is known as the **plasma operator**. The operator creates the child chain, validates transactions, and periodically submits a summary of the child chain's state to the root chain.
- **Plasma smart contracts:** Plasma introduces the concept of smart contracts that govern the relationship between the root chain and the child chains. These smart contracts define the rules for depositing assets into the child chain, transferring assets within the child chain, and exiting assets from the child chain back to the root chain.
- **Exit mechanism:** Participants in the child chain can exit and withdraw their assets to the root chain if they believe that the child chain is misbehaving. The exit mechanism ensures that users can reclaim their assets on the root chain in case of disputes or malfunctions on the child chain.
- **Challenge period:** Plasma introduces a challenging period during which participants can challenge any malicious or incorrect behavior on the child chain. If a challenge is successful, the malicious actor may lose their deposited funds.
- **Mass exits:** To prevent **denial of service (Dos)** attacks, plasma allows for mass exits where multiple users can exit the child chain simultaneously in the event of a significant issue.

Plasma framework is still evolving and going through incremental changes. Scalability is a tough problem to solve and require lots of experimentation. Plasma framework carry a potential to become a good solution to the blockchain scalability problem in the near future.

In the coming section, we will understand about rollups and how it works.

Rollups

Rollups is an aggregation technique where multiple transactions are bundled or rollup in a single batch to process on the mainchain. Rollups are a scaling solution designed to enhance scalability and reduce transaction costs by moving some processing off-chain while maintaining the security and decentralization of the mainchain.

There are two main types of rollups:

- Optimistic rollups
- **Zero-knowledge (ZK)** rollups

Let us discuss them in detail.

Optimistic rollups

In optimistic rollups, the transaction data and computation are conducted off-chain in a separate layer called the **rollup chain**. Only the transaction's outcome, along with cryptographic proof, is submitted to the mainchain.

Optimistic rollups assume that the off-chain computation is correct. The mainchain validators initially accept the transaction without validating the entire computation. This speeds up the process and reduces on-chain congestion.

However, there is a challenge period during which anyone can raise a dispute if they believe that a transaction is fraudulent. Validators then need to validate the entire computation. If the transaction is found to be correct, the

challenger loses a security deposit. If it is fraudulent, the submitter loses their deposit.

Optimistic Ethereum is an example of optimistic rollups implemented on the Ethereum blockchain. It aims to improve scalability by handling most transactions off-chain and submitting only the essential information to the Ethereum mainchain.

Zero-knowledge rollups

In ZK-rollups, the transaction data and the computation are kept off-chain, but the cryptographic proof is submitted to the mainchain. This proof provides a zero-knowledge succinct argument that the off-chain computation was done correctly without revealing any details.

Unlike optimistic rollups, ZK-rollups do not require a challenge period because the cryptographic proof ensures the correctness of the computation. This leads to faster finality and reduces the potential for disputes. They provide privacy benefits as the off-chain computation details are not exposed. Users can retain a higher level of privacy while still benefiting from the security and scalability advantages. Loopring and ZKsync are examples of projects implementing ZK-rollups on Ethereum, aiming to improve scalability and reduce transaction fees.

In summary, both optimistic rollups and ZK-rollups are solutions that leverage off-chain processing to enhance blockchain scalability. They strike a balance between efficiency and security, offering different trade-offs in terms of assumptions, dispute resolution mechanisms, and privacy features. These rollup solutions play a crucial role

in addressing the scalability challenges faced by blockchain networks.

Lightning Network

The Bitcoin Lightning Network is a layer 2 scaling solution as we studied in the earlier section. It is designed to address the scalability issues of the Bitcoin blockchain. It enables faster and cheaper transactions by facilitating off-chain payment channels.

The Lightning Network builds on top of the existing Bitcoin blockchain and allows users to conduct transactions off-chain, reducing congestion and fees on the mainchain. Participants open payment channels between each other. These channels are like off-chain channels where users can transact without interacting with the main Bitcoin blockchain.

Opening a payment channel involves creating a multi-signature wallet (2-of-2 or 2-of-3) between the participants, and the channel's initial state is recorded on the Bitcoin blockchain. Once a payment channel is established, users can transact with each other off-chain. They can send and receive payments without needing to broadcast every transaction to the mainchain.

Lightning Network channels support bi-directional payments. Users can send payments back and forth as long as the channel's balance allows it. The net effect of these transactions is updated in real-time within the payment channel.

Users can close the payment channel at any time by broadcasting the latest transaction state to the Bitcoin blockchain. This finalizes the transactions and settles the

balances. The final state is enforced by the underlying smart contract on the Bitcoin blockchain.

The benefits of the lightning network are as follows:

- **Scalability:** The Lightning Network significantly increases the transaction throughput by allowing users to conduct numerous off-chain transactions without congesting the main Bitcoin blockchain.
- **Reduced fees:** Off-chain transactions on the Lightning Network are faster and incur lower fees compared to on-chain transactions.
- **Instant transactions:** Transactions within the Lightning Network are nearly instant, providing a quick and efficient way for users to transact.
- **Microtransactions:** The Lightning Network enables microtransactions, allowing users to send and receive small amounts of Bitcoin economically.

Note: Lightning Network is an opt-in solution, and users can choose whether to use it for their transactions. While it offers scalability benefits, it also introduces new considerations, such as the need for users to manage channels and liquidity effectively. The Lightning Network continues to evolve, contributing to the ongoing development of the Bitcoin ecosystem.

Use cases and applications

Now, let us understand the use cases of fast and scalable blockchain:

- **Payments and microtransactions:** Decentralized payment networks can handle a large number of

transactions per second, making them suitable for micropayments and other low-value transactions.

- **Supply chain management:** Decentralized systems can track the origin, movement, and destination of goods in a supply chain, reducing the time and costs associated with traditional supply chain management.
- **Digital identity:** Decentralized systems can provide secure and scalable digital identity solutions, enabling individuals to control and manage their personal information.
- **DeFi:** DeFi applications such as exchanges, lending platforms, and stablecoins can benefit from scalability to handle the large volume of transactions in the DeFi ecosystem.
- **Gaming:** Decentralized gaming platforms can use scalability solutions to handle large numbers of players and transactions, enabling fast and secure gaming experiences.
- **Data management:** Decentralized systems can be used to store and manage large amounts of data, such as medical records, in a secure and scalable manner.
- **Decentralized autonomous organizations:** **Decentralized autonomous organizations (DAO)** can use scalability solutions to handle a large number of members, votes, and transactions in a secure and transparent manner.

Conclusion

We studied various blockchain scaling challenges and problems blockchain systems are facing right now. We also studied various techniques and approaches both matured

and experimental and how they can be applied to handle scalability issues. Blockchain systems are evolving and getting more and more load as we write this chapter, without scaling solutions in place, Blockchain systems will not be able to cope up with the demand. Hence, solving scalability problem is the key issue in the blockchain community.

In the next chapter, we are going to learn blockchain interoperability and how it can be useful in scaling blockchain systems.

Join our book's Discord space

Join the book's Discord Workspace for Latest updates, Offers, Tech happenings around the world, New Release and Sessions with the Authors:

[**https://discord.bpbonline.com**](https://discord.bpbonline.com)



CHAPTER 5

Blockchain and Smart Contracts Interoperability

Introduction

Making different blockchain systems work together smoothly is important for making them bigger and better. When these systems can talk to each other and share information easily, it helps create stronger and more reliable solutions. It also makes it easy to connect with other outside tools and systems.

We can make this happen by introducing techniques like direct trades between different blockchains and setting up special communication networks. When blockchains can work together like this, it opens up many possibilities for new and exciting ideas. In this chapter, we learn how developers can build apps that use the best parts of different blockchains, creating new and useful things that were not possible before.

Structure

In this chapter, we will cover the following topics related to interoperability:

- Inter-Blockchain Communication protocol
- Cross-chain atomic swaps
- Hubs for interoperability in blockchain
- Relays for blockchain
- Security concerns for interoperable blockchain

Objectives

In this chapter, we are going to learn inter-blockchain protocol and how we can enable this using various techniques such as hubs, relays, etc. At the end of the chapter, you should be well versed with the blockchain interoperability technology.

Inter-Blockchain Communication protocol

The **Inter-Blockchain Communication (IBC)** protocol is a groundbreaking standard protocol that facilitates communication and interoperability between independent blockchain networks. It emerged as a vital solution to address the growing need for seamless interaction and data transfer across various blockchain ecosystems. Please refer to the following figure:

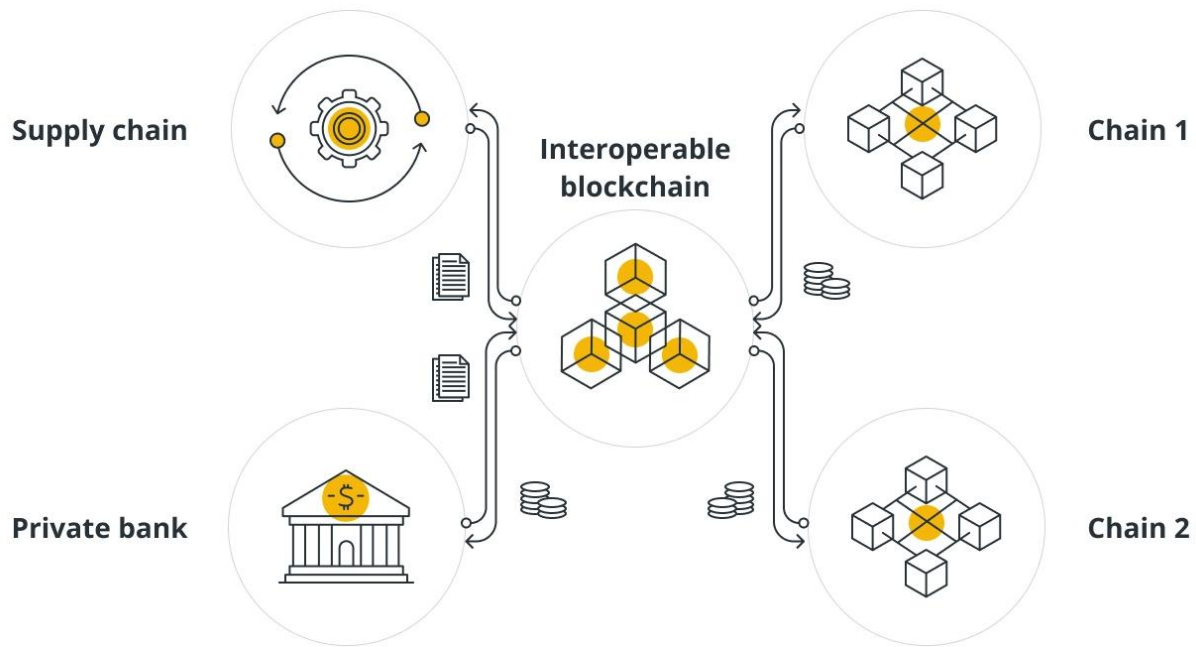


Figure 5.1: *Interoperable blockchain*

In this section, we will study a comprehensive understanding of IBC, including its underlying principles, technical components, use cases, and potential implications for the blockchain industry.

Overview of Inter-Blockchain Communication protocol

IBC is a standardized protocol that enables different blockchain networks to communicate, share data, and execute transactions with each other.

The primary purpose of IBC is to break down the barriers between isolated blockchain ecosystems, fostering collaboration and interoperability within the decentralized landscape.

Here are some of the key features of IBC:

- **Packet-based communication:** IBC relies on a packet-based communication model, transferring data

between blockchains in discrete packets.

- **Connection establishment:** Before two blockchains can communicate using IBC, they establish a secure and reliable connection.
- **Channels:** Within a connection, blockchains can create multiple channels for different types of communication, including token transfers and informational data exchange.
- **Packet relay:** Packets of data are relayed between blockchains through intermediate relayer called **packet relayer**, ensuring secure and reliable data transfer.
- **Acknowledgements and timeouts:** IBC includes mechanisms for acknowledgments and timeouts to ensure the integrity of data transfer and maintain consistency across blockchains.
- **Atomicity and consistency:** Transactions executed using IBC are atomic and consistent, ensuring that either the entire transaction is executed successfully or none of it is.

Let us understand the important technical components of IBC.

Technical components of IBC protocol

IBC protocol consists of the following four core components, as shown in *Figure 5.2*:

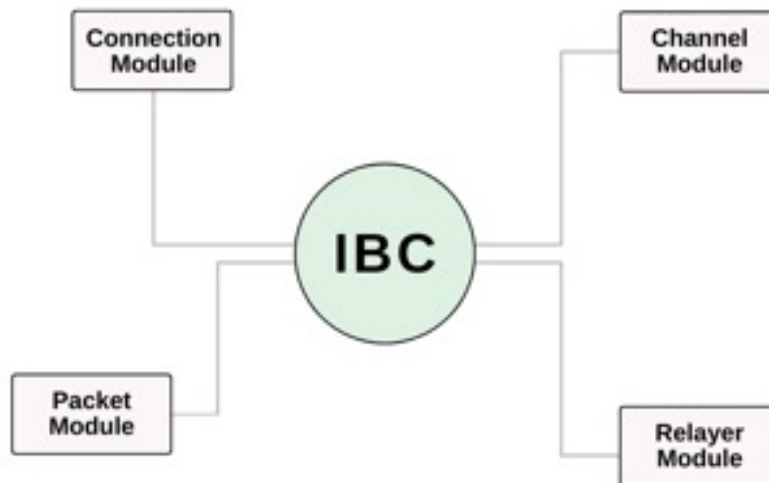


Figure 5.2: IBC modules

Let us learn about the IBC modules in brief:

- **Connection Module:** The Connection Module handles the establishment and maintenance of connections between different blockchains. It verifies the authenticity of connection requests and ensures the security of communication channels.
- **Channel Module:** The Channel Module enables the creation and management of channels within established connections. It defines the rules and protocols for data transfer and ensures the integrity and reliability of channel-based communication.
- **Packet Module:** The Packet Module facilitates the transfer of data between blockchains as discrete packets. It includes mechanisms for packet encoding, encryption, and validation to ensure secure and reliable data transfer.

- **Relayer Module:** The Relayer Module serves as an intermediary between blockchains, relaying packets of data from the source blockchain to the destination blockchain. It ensures that packets are delivered securely and reliably, even in the presence of network delays or failures.

So far, we have studied IBC's overview and its technical components. Now, let us try to understand its various use cases.

Use cases and applications of IBC protocol

IBC communication protocols open up the possibility of various applications which were not possible before. Here, we have listed some use cases and applications that can take advantage of the IBC communication protocol:

- **Cross-chain asset transfers:** IBC enables the seamless transfer of digital assets between different blockchain networks, allowing users to exchange tokens and assets across disparate ecosystems.
- **Decentralized exchange (DEX) platforms:** IBC facilitates the creation of DEX platforms that enable cross-chain trading of digital assets without intermediaries.
- **Multi-chain smart contracts:** IBC opens up new possibilities for multi-chain smart contracts, where smart contracts can interact with and execute transactions across multiple blockchain networks.
- **Interoperable applications and services:** IBC promotes the development of interoperable applications and services that can leverage the

strengths of multiple blockchain networks to create innovative solutions.

These use cases and applications can have a significant impact on the blockchain industry. Let us understand some of the implications.

Implications for the blockchain industry

IBC enhances scalability and efficiency within the blockchain industry by breaking down the barriers between isolated blockchain ecosystems, enabling greater collaboration and resource sharing.

IBC fosters innovation and experimentation by empowering developers to create **decentralized applications (DApps)** that can seamlessly interact with multiple blockchain networks, driving progress and evolution within the industry.

IBC enhances the user experience by enabling seamless cross-chain asset transfers, DEX platforms, and interoperable applications, providing users with greater flexibility and control over their digital assets and interactions.

The IBC represents a significant milestone in the evolution of blockchain technology, paving the way for greater collaboration, interoperability, and innovation within the decentralized landscape.

By enabling different blockchain networks to communicate and interact with each other seamlessly, IBC unlocks new possibilities for cross-chain asset transfers, DEX platforms,

multi-chain smart contracts, and interoperable applications and services.

As the blockchain industry continues to evolve and mature, IBC will play a crucial role in driving progress, scalability, and efficiency, ultimately shaping the future of decentralized technology.

Here are some of the platforms that implement and support IBC:

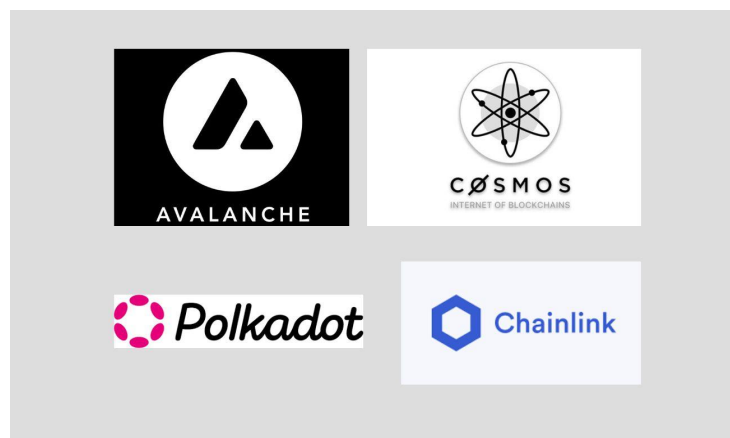


Figure 5.3: IBC platforms

In the next section, we will learn and understand how asset transfer can be done using an interoperable blockchain.

Cross-chain atomic swaps

Cross-chain atomic swaps are a mechanism for exchanging digital assets across different blockchain networks.

At its core, an atomic swap is akin to a sophisticated handshake between blockchain participants, ensuring a secure and trustless exchange of assets. Here is a simplified breakdown:

- **Agreement:** Two parties, let us call them *Alice* and *Bob*, agree to swap their digital assets, such as cryptocurrencies, under certain conditions.

- **Smart contracts:** Utilizing smart contracts, *Alice* and *Bob* lock their respective assets in escrow. These contracts enforce the swap's terms and conditions, ensuring fairness and security.
- **Execution:** The swap is executed atomically through cryptographic techniques like **hash time locked contracts (HTLCs)**. This means that either both parties receive the agreed-upon assets, or the transaction is cancelled, preventing any form of partial fulfilment or loss.

Now, let us elevate the complexity by considering assets residing on different blockchain networks. Cross-chain atomic swaps enable interoperability between these networks, facilitating seamless asset exchanges. Here is how it works:

- **Compatibility:** For cross-chain swaps to occur, the involved blockchains must support compatible scripting languages and cryptographic primitives. This ensures that smart contracts across different networks can communicate and interact effectively.
- **HTLCs:** The backbone of cross-chain swaps lies in HTLCs, which enable secure and trustless transactions across disparate blockchains. By leveraging cryptographic hash functions and time locks, HTLCs ensure that asset transfers occur only when predetermined conditions are met, safeguarding the integrity of the swap process.
- **Interoperability solutions:** Various projects and protocols, such as atomic swaps, Lightning Network, and **Interledger Protocol (ILP)**, offer frameworks for cross-chain interoperability. These solutions enhance the seamless transfer of assets between different

blockchain ecosystems, fostering greater decentralization, and utility.

Let us understand another concept of interoperability and communication using a hub technology.

Hubs for interoperability in blockchain

Hubs play a pivotal role in enabling seamless communication and interoperability between various blockchain networks. Let us understand the concept of hubs and how they are useful in blockchain interoperability.

Imagine hubs as central meeting points or bridges that connect different blockchain networks, allowing them to exchange information and assets effortlessly. Here is a simplified breakdown of how hubs work:

- **Centralized points of connectivity:** Hubs serve as centralized entities that act as intermediaries between multiple blockchain networks. They establish connections and facilitate communication channels, enabling interoperability among otherwise isolated networks.
- **Routing:** Similar to how routers direct internet traffic, hubs route data and transactions between connected blockchains. They ensure that information flows smoothly from one network to another, maintaining integrity and security throughout the process.
- **Protocol translation:** Hubs often perform protocol translation, converting data and transactions from one blockchain's format to another. This ensures compatibility and seamless interaction between diverse blockchain ecosystems.

Refer to the following figure. Hubs and zone communicate with each other and cross hub communication happens only by hubs:

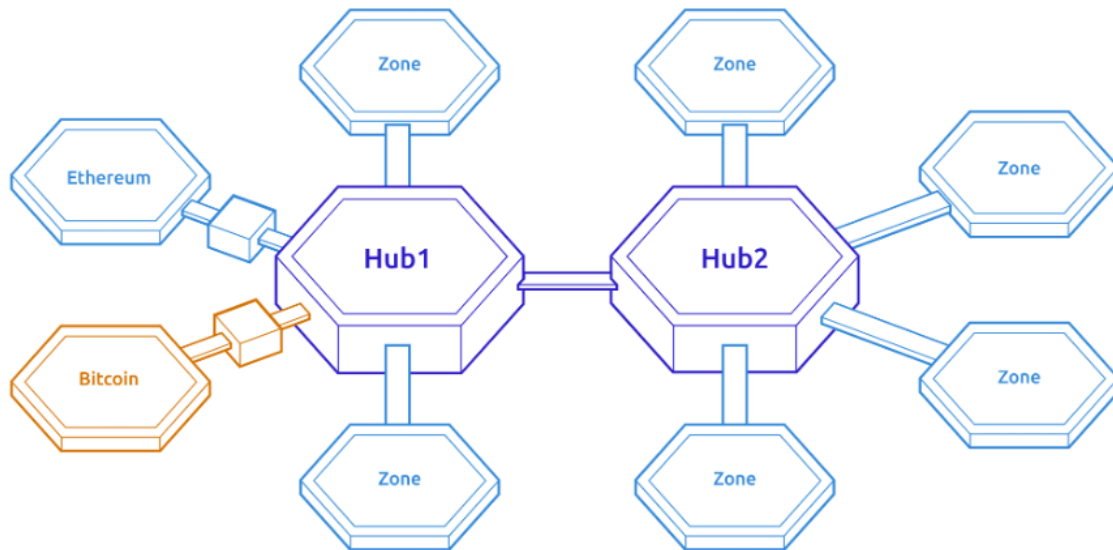


Figure 5.4: Hub communication

Key components of hubs

To grasp the inner workings of hubs, it is essential to understand their key components and functionalities:

- **Connection establishment:** Hubs establish secure and reliable connections with participating blockchain networks. This involves negotiating communication protocols, verifying identities, and ensuring trust among network participants.
- **Data routing:** Once connections are established, hubs route data and transactions between interconnected blockchains. They act as intermediaries, relaying information while maintaining data integrity and security.

- **Protocol conversion:** Hubs facilitate protocol conversion, translating data and transactions between different blockchain protocols. This ensures seamless interoperability, allowing diverse networks to communicate and transact effectively.
- **Transaction verification:** Hubs are crucial in transaction verification and validation. They ensure that transactions comply with predefined rules and protocols, preventing double spending and other fraudulent activities.

The versatility of hubs extends to various use cases and applications within the blockchain ecosystem:

- **Cross-chain asset transfers:** Hubs enable frictionless asset transfers between different blockchain networks. Users can exchange digital assets seamlessly, leveraging the connectivity provided by hubs.
- **Decentralized finance (DeFi):** In DeFi, hubs facilitate interoperability between DApps and financial protocols across multiple blockchains. This enables novel financial products and services to thrive in a decentralized ecosystem.
- **Multi-chain smart contracts:** Hubs empower developers to create multi-chain smart contracts that interact with multiple blockchain networks. This opens up new possibilities for DApps and autonomous systems that transcend individual blockchains.

Hubs offer many advantages to facilitate interoperability, but there is one issue: They operate in a centralized way to facilitate communication. Let us learn about relays and how they offer decentralized communication.

Relays for blockchain interoperability

Relay is a bridge that connects two or more blockchain networks, enabling the transfer of assets and data between them. It does not act as a central point of communication. Relay establishes direct connections between blockchain networks and facilitates the transfer of assets and data between them. Relays can help to improve the speed and efficiency of the network by reducing latency and increasing transaction throughput.

Please refer to the following figure for relays:

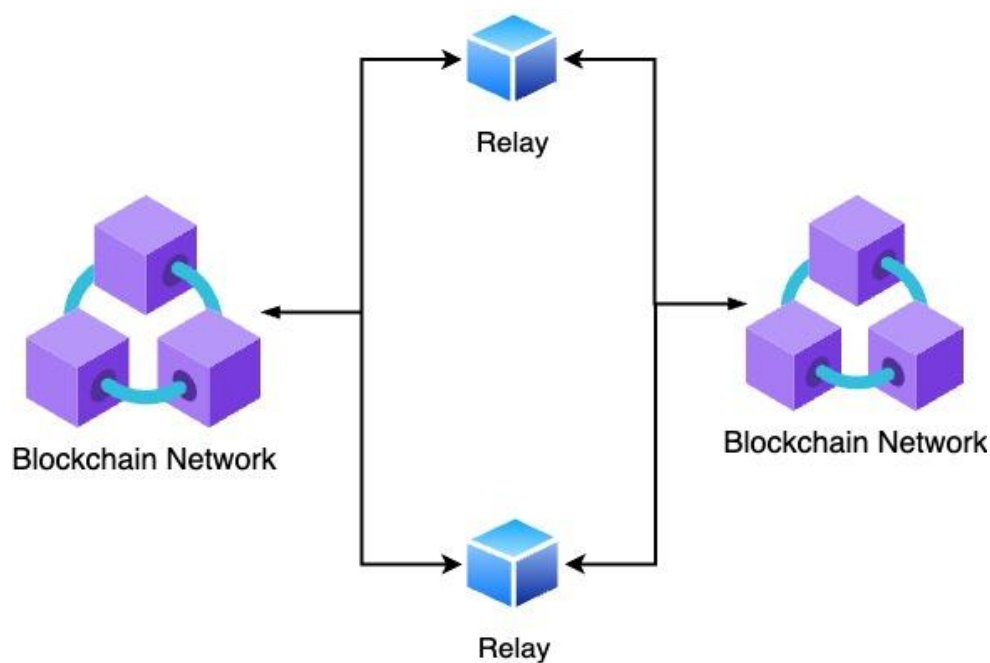


Figure 5.5: Relays allowing communication between networks

Relays act as intermediaries or bridges between different blockchain networks, enabling them to exchange data, transactions, and assets. They serve as communication channels that facilitate interoperability by relaying information between otherwise isolated ecosystems. To grasp the significance of relays, let us explore their key functionalities:

- **Message routing:** Relays receive messages or transactions from one blockchain network and forward them to their intended destinations on another network. This involves routing data packets through secure channels while ensuring timely and reliable delivery.
- **Protocol translation:** Interoperability often requires translating data and transactions between different blockchain protocols or formats. Relays facilitate this process by converting messages into compatible formats that can be understood and processed by the receiving network.
- **Data verification:** Before relaying messages, relays validate the authenticity and integrity of the data to prevent tampering or manipulation. They employ cryptographic techniques and consensus mechanisms to verify transactions and ensure their validity across interconnected networks.
- **Network monitoring:** Relays continuously monitor the health and performance of interconnected blockchain networks to ensure smooth communication and operation. They detect and address issues such as network congestion, latency, and downtime to maintain optimal reliability and efficiency.

Relays can be categorized based on their operational characteristics and functionalities. Some common types of relays include:

- **Cross-chain relays:** These relays specialize in facilitating interoperability between different blockchain networks, allowing for seamless data and asset transfers across disparate ecosystems.
- **Data relays:** Data relays focus on transmitting non-financial data, such as smart contract interactions,

oracle updates, or IoT sensor readings, between interconnected networks.

- **Asset relays:** Asset relays enable the exchange of digital assets, such as cryptocurrencies or tokenized assets, between different blockchain networks, powering DEXs and liquidity pools.

Use cases and applications

The versatility of relays extends to various use cases and applications within the blockchain ecosystem:

- **Cross-chain asset transfers:** Relays enable users to transfer digital assets across different blockchain networks, facilitating decentralized trading, lending, and borrowing activities.
- **DeFi:** In the rapidly evolving landscape of DeFi, relays power DEXs, liquidity pools, and lending protocols, allowing users to access financial services across multiple blockchain networks.
- **Interoperable applications:** Relays empower developers to build interoperable applications and dApps that leverage data and services from multiple blockchain networks. These applications can seamlessly interact with diverse ecosystems, unlocking new use cases and functionalities.

Security concerns for interoperable blockchain

Interoperable blockchains hold tremendous promise for revolutionizing various industries by enabling seamless data and asset exchange between disparate networks. However, with this increased connectivity comes a myriad of security challenges and concerns that must be addressed

to ensure the integrity, confidentiality, and trustworthiness IBC.

Let us understand the security considerations associated with interoperable blockchains:

- **Attack surface expansion:** Interoperable blockchains create a larger attack surface, as malicious actors may exploit vulnerabilities in interconnected networks to compromise multiple systems simultaneously.
- **Protocol incompatibility:** Incompatible protocols between interconnected blockchains can introduce vulnerabilities and attack vectors, leading to unauthorized data access, transaction manipulation, or smart contract exploits.
- **Cross-chain attacks:** Malicious actors may launch cross-chain attacks targeting vulnerabilities in the interoperability mechanisms, such as relays or bridges, to manipulate data, steal assets, or disrupt network operations across multiple blockchains.
- **Data privacy concerns:** Interoperable blockchains raise concerns about data privacy and confidentiality, as sensitive information transmitted between networks may be exposed to unauthorized parties during the communication process.
- **Smart contract vulnerabilities:** Smart contracts facilitating cross-chain transactions are susceptible to vulnerabilities such as reentrancy attacks, integer overflow, or unauthorized access, which can lead to financial losses and system exploitation.
- **Centralization risks:** Centralized components, such as relays or bridges used for interoperability may introduce centralization risks, as they become single

points of failure or control susceptible to regulatory intervention or malicious attacks.

- **Trust dependencies:** Interoperable blockchains rely on trust assumptions between network participants, including relay operators, bridge validators, or cross-chain service providers, raising concerns about trustworthiness, transparency, and accountability.
- **Verification challenges:** Verifying the authenticity and integrity of data and transactions transmitted between interconnected blockchains presents technical challenges, as cryptographic proofs and consensus mechanisms may vary across different networks.
- **Oracles and data feeds:** Integration with external data sources or oracles introduces risks related to data accuracy, reliability, and manipulation, potentially leading to misinformation, oracle attacks, or invalid smart contract outcomes.

Conclusion

In this chapter, we have studied different ways we can connect blockchain running on unique protocols and designs, thus achieving interoperability. Interoperability provides blockchain integration and smooth flow of data between integrations. We also studied security concerns that come with interoperability.

In the next chapter, we are going to study about interplanetary file system and how it is evolving and helping Web 3.0 ecosystem.

Join our book's Discord space

Join the book's Discord Workspace for Latest updates, Offers, Tech happenings around the world, New Release and Sessions with the Authors:

[**https://discord.bpbonline.com**](https://discord.bpbonline.com)



CHAPTER 6

Interplanetary File System

Introduction

In this chapter, we will learn about the **Interplanetary File System (IPFS)**, as a distributed system to store, access, and share files. IPFS has an ambitious goal to turn the existing centralized internet to distributed internet. it aims to replace HTTP as a default protocol to share files over the internet. We will also learn about how to install and configure IPFS and build a full-stack application using IPFS.

Structure

In this chapter, we will learn the following:

- Introduction to Interplanetary File System
- Working of Interplanetary File System
- Installing and configuring Interplanetary File System
- Hosting a static website in IPFS

- Building a full-stack application using IPFS
- IPFS Cluster

Objectives

After studying this chapter, you should be able to understand IPFS core concepts and working of IPFS protocol, and the use of IPFS to build the application. Let us get started.

Introduction to Interplanetary File System

According to the IPFS official website:

IPFS is a distributed system for storing and accessing files, websites, applications, and data.

IPFS provides a platform for developers to store data on distributed platforms. There are projects such as OrbitDB (we will learn about it in the upcoming chapters), that provide database solutions on top of IPFS. Due to the distributed nature of IPFS, it is the first choice for **decentralized application (DApp)** development. In the previous chapter, we studied building applications using Ethereum and used Swarm as a storage solution. IPFS uses content-based addressing instead of location-based addressing to access files. So, location-based addressing is what we will use; currently, we use domain names or IP addresses to access the content.

Consider this example:

`https://abc.com/hello.html`

Here, we are accessing the **hello.html** file using the domain name. The domain name, in turn, looks out for the

IP address of the server and picks up the file from the server.

In the IPFS, we use content addressing. Let us look at an example:

<https://ipfs.io/QmXoypizjW3WknFiJnKLwHCnL72vedxjQkDDP1mXWo6uco/hello.html>

As you can see here, we access the content by the hash. Hash represents the file and not the location where the file is stored. It is the responsibility of the IPFS network to collect the bits and pieces of the file and deliver them to the user.

Let us understand how IPFS works in the following sections.

Working of Interplanetary File System

IPFS is a distributed peer-to-peer storage network. In IPFS, content is accessed from peers spread across the world. Peers in IPFS can relay information, store information, or do both.

IPFS works by forming a network using independent peers. These peers store and relay files information asked by the users.

IPFS uses the following important modules to make the network work:

- Content-based addressing
- **Directed acyclic graph (DAG)**
- **Distributed hash table (DHT)**

Let us explore these modules in detail.

Content-based addressing

IPFS uses content-based addressing to identify data by what is in it, rather than by where it is located.

Looking for an item by content is something we already do. For example, when you look for a piece of cloth, you often ask for it by the color or type; that is content addressing because you are asking for what it is.

If you were using location addressing to find that cloth, you would ask for it by where it is: I want the t-shirt that is on the second rack, three pieces of cloth away from the left. If someone moved that cloth, you would be out of luck!

This is how we access content on the internet. For example,

- **/Users/Alice/Documents/project_paper.doc**
- **C:/Users/Joe/My/Documents/presentation.ppt**

IPFS, in turn, generates the **Content Identifier (CID)**, which is also referred to as the **hash**.

Content addressing by using a hash is widely used to connect data in distributed systems. You can see the hash-based content addressed in the Git to the cryptocurrencies. It is everywhere.

However, the underlying data structures in these systems are not always interoperable. Every system uses a different set of data sets and structures. To tackle such issues, IPFS developed **Interplanetary Linked Data (IPLD)**, which acts as a data model for content addressing. IPLD acts as a translation layer between different types of data sets used by the different systems. So, IPFS uses content addressing using the hash, and IPLD performs the data sets

translation, if required, to enable smooth transition of data from one system to another.

Directed acyclic graph

IPFS uses a DAG to store the content. It uses Merkle DAG, which is hashing together leaf nodes to form a parent node. We studied the Merkle tree in *Chapter 1, Introduction to Blockchain and Decentralized Network*.

DAG is a graph in which edges have direction and cycles are not allowed.

For example, a linked list like $A|B|C$ is an instance of a DAG where A references B , and so on. We say that B is a child or a descendant of A , and node A has a link to B . Similarly, A is a parent of B .

In DAG, nodes that are not children to any other node are referred to as **root nodes**.

A Merkle DAG is a DAG where each node has an identifier (hash or CID), and this is the result of hashing the node's contents with the SHA-256 function. Merkle DAG node is immutable. Any change in the node, in turn, creates a different Merkle DAG.

IPFS builds the Merkle DAG tree of the content by splitting the content into blocks. Each of these blocks is then associated with CID and then hashed together from the leaf node to form a Merkle DAG tree.

So, IPFS provides content-based addressing using hash and Merkle DAG links, which hash together to form content. Let us learn about the next and last module, **DHT**.

Distributed hash table

A hash table is a set of key-value pairs, and a distributed hash table is where the table is distributed across the different peers in the network.

The main advantages of DHTs are decentralization, fault tolerance, and scalability. Due to this, the hash table can span across millions of nodes, and data availability is higher due to the distributed nature of data.

IPFS has a separate library called **libp2p**, which deals with distributed hash tables and peer networking.

IPFS uses DHT to find the current location of the content and peers. Once the content is found, IPFS connects to the content and downloads it to your system. Once these blocks are downloaded, you can calculate the hash and verify the authenticity of the content.

libp2p also handles peer-to-peer networking and connection multiplexing. Connection multiplexing allows **libp2p** to use a single connection for multiple services.

We will learn more about DHT in the coming section. Now, let us install IPFS in our system in the following sections.

Installing and configuring Interplanetary File System

Let us install IPFS in our system and become a peer through the following steps:

1. Navigate to the official IPFS website at **<https://dist.ipfs.io/-go-ipfs>** and download the IPFS package for your system. As shown in the following figure, select your operating system to download the correct version of IPFS:

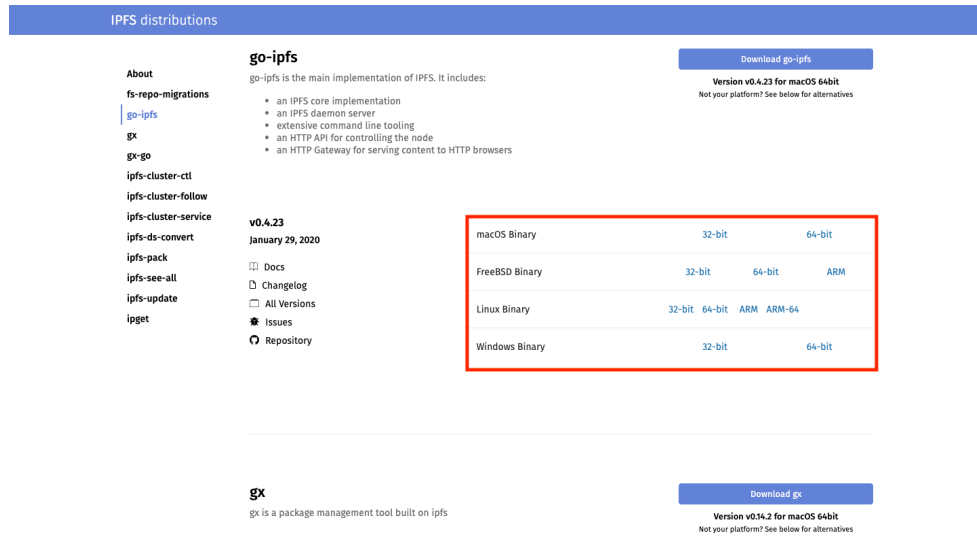


Figure 6.1: IPFS website

2. Once you have downloaded the package, extract or unzip it and switch to the extracted folder. The following command installs the IPFS in your system:

```
cd go-ipfs  
./install.sh
```

3. Once IPFS installed, you can verify it by running the following command:

```
$ ipfs help
```

It should print the IPFS commands in the Terminal, as shown in the following screenshot:

```
UnixRoot@Shahids-MacBook-Air: ~  
~ ipfs help  
USAGE  
  ipfs - Global p2p merkle-dag filesystem.  
  
  ipfs [--config=<config> | -c] [--debug=<debug> | -D] [--help=<help>] [-h=<h>]  
  [--local=<local> | -L] [--api=<api>] <command> ...  
  
SUBCOMMANDS  
BASIC COMMANDS  
  init      Initialize ipfs local configuration  
  add <path> Add a file to IPFS  
  cat <ref>  Show IPFS object data  
  get <ref>  Download IPFS objects  
  ls <ref>   List links from an object  
  refs <ref> List hashes of links from an object  
  
DATA STRUCTURE COMMANDS  
  block      Interact with raw blocks in the datastore  
  object      Interact with raw dag nodes  
  files      Interact with objects as if they were a unix filesystem  
  dag        Interact with IPLD documents (experimental)  
  
ADVANCED COMMANDS  
  daemon      Start a long-running daemon process  
  mount       Mount an IPFS read-only mountpoint  
  resolve     Resolve any type of name  
  name        Publish and resolve IPNS names  
  key         Create and list IPNS name keypairs  
  dns         Resolve DNS links  
  pin         Pin objects to local storage  
  repo        Manipulate the IPFS repository  
  stats       Various operational stats
```

Figure 6.2: IPFS help command

Now, let us run IPFS in our system:

4. Run this command on your Terminal:

\$ ipfs init

It will initialize the repository and create the required files in your system to run the IPFS.

5. Start the IPFS daemon. To do so, run this command:

\$ ipfs daemon

This command will start the IPFS in a daemon (background process) mode. You should see the message in your Terminal, as follows:

```
ipfs daemon
Initializing daemon...
go-ipfs version: 0.4.18-
Repo version: 7
System version: amd64/darwin
Golang version: go1.11.1
Successfully raised file descriptor limit to 2048.
Swarm listening on /ip4/127.0.0.1/tcp/4001
Swarm listening on /ip4/192.168.1.6/tcp/4001
Swarm listening on /ip6:::1/tcp/4001
Swarm listening on /p2p-circuit
Swarm announcing /ip4/127.0.0.1/tcp/4001
Swarm announcing /ip4/192.168.1.6/tcp/4001
Swarm announcing /ip6:::1/tcp/4001
API server listening on /ip4/127.0.0.1/tcp/5001
Gateway (readonly) server listening on /ip4/127.0.0.1/tcp/8080
Daemon is ready
```

Figure 6.3: Running IPFS daemon server

- Now, navigate to **<http://localhost:5001/webui>**; you should see the admin screen, as follows:

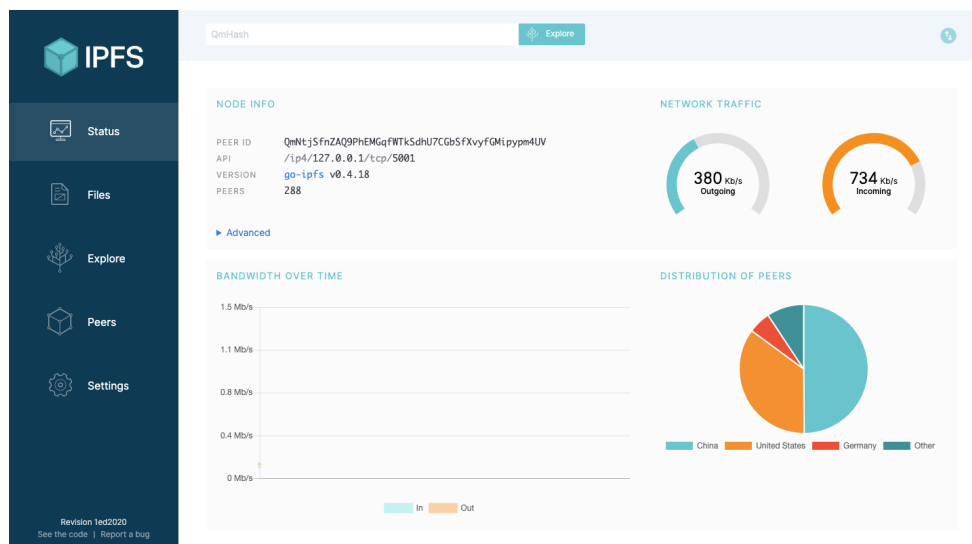


Figure 6.4: IPFS web interface

In the admin screen, you can see the **NODE INFO**, such as **Peer ID**, **VERSION**, **NETWORK TRAFFIC** charts, and **DISTRIBUTION OF PEERS**.

7. Now, visit the **Peers** page; you should be able to see the peer distribution across the world. It should look like the following screenshot, but the peer's number may vary:

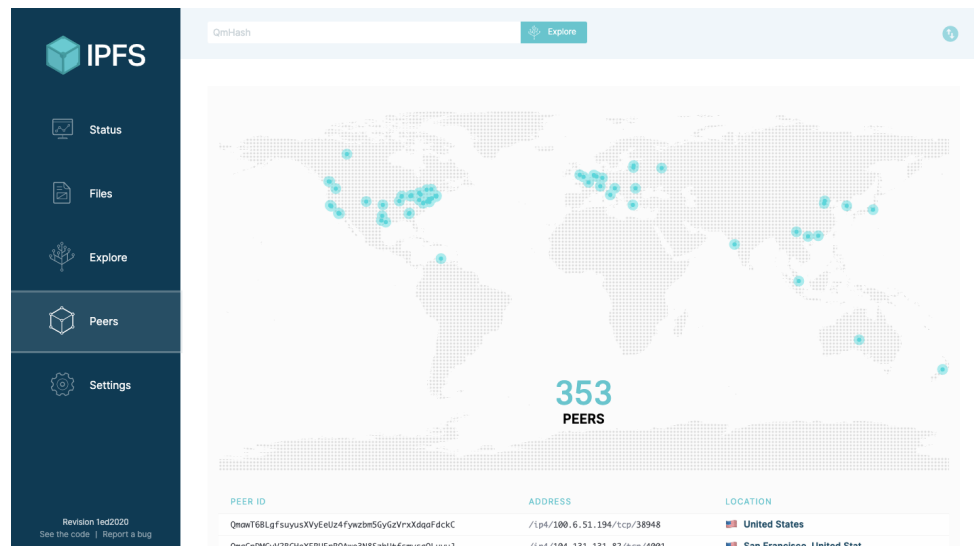


Figure 6.5: IPFS peers

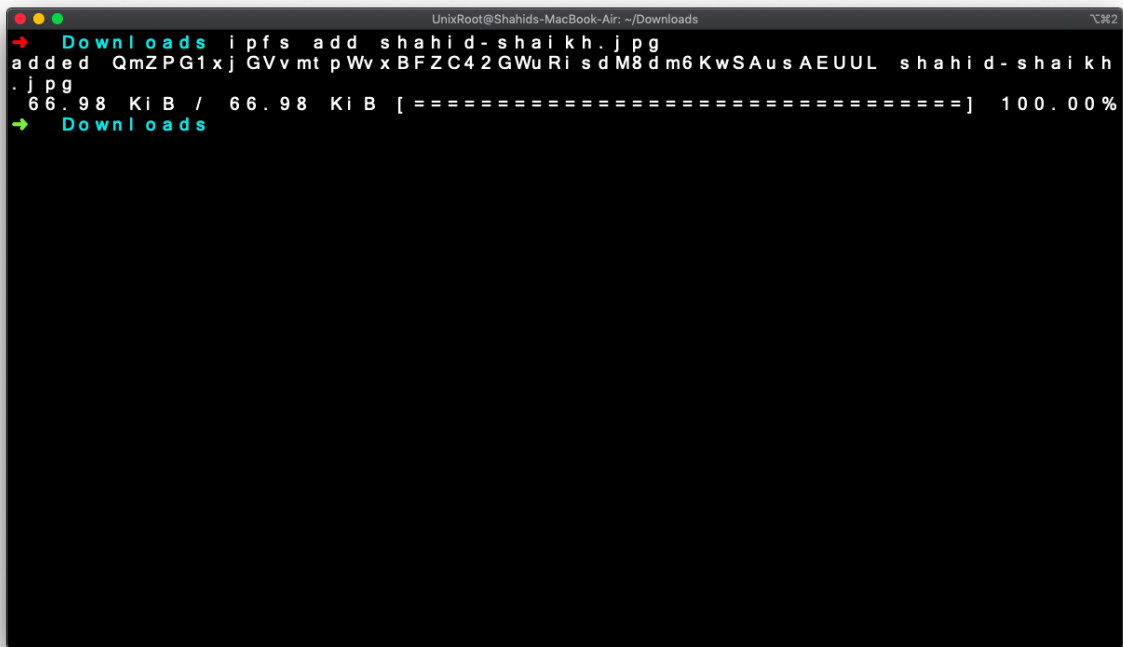
You can see the peers and their IDs scattered across the world. Now you are also a peer, and you can access any documents you like (only if you have the CID or hash of the file).

Let us quickly add a file and try to access it using the IPFS commands.

8. Run this command to add a file in the IPFS network:

\$ ipfs add <path of the file>

This command will generate the hash of the content on the screen if executed successfully:

A terminal window titled 'UnixRoot@Shahids-MacBook-Air: ~/Downloads' showing the execution of the 'ipfs add' command. The command 'ipfs add shahid-shaikh.jpg' has been entered. The output shows the file being added with its hash 'QmZPG1xjGVvmt pWvxBFZC42GWuRi sdM8dm6KwSAusAEUUL' and the filename 'shahid-shaikh.jpg'. A progress bar indicates that 66.98 KiB of the 66.98 KiB file has been added, reaching 100.00% completion. The prompt 'Downloads' is visible at the bottom left of the terminal window.

```
UnixRoot@Shahids-MacBook-Air: ~/Downloads
→ Downloads ipfs add shahid-shaikh.jpg
added QmZPG1xjGVvmt pWvxBFZC42GWuRi sdM8dm6KwSAusAEUUL shahid-shaikh
.jpg
66.98 KiB / 66.98 KiB [=====] 100.00%
→ Downloads
```

Figure 6.6: Adding objects in IPFS

9. Now, copy the hash and access it using the browser:

localhost:8080/ipfs/<hash of the file>

You should see the file in your browser like the following figure:

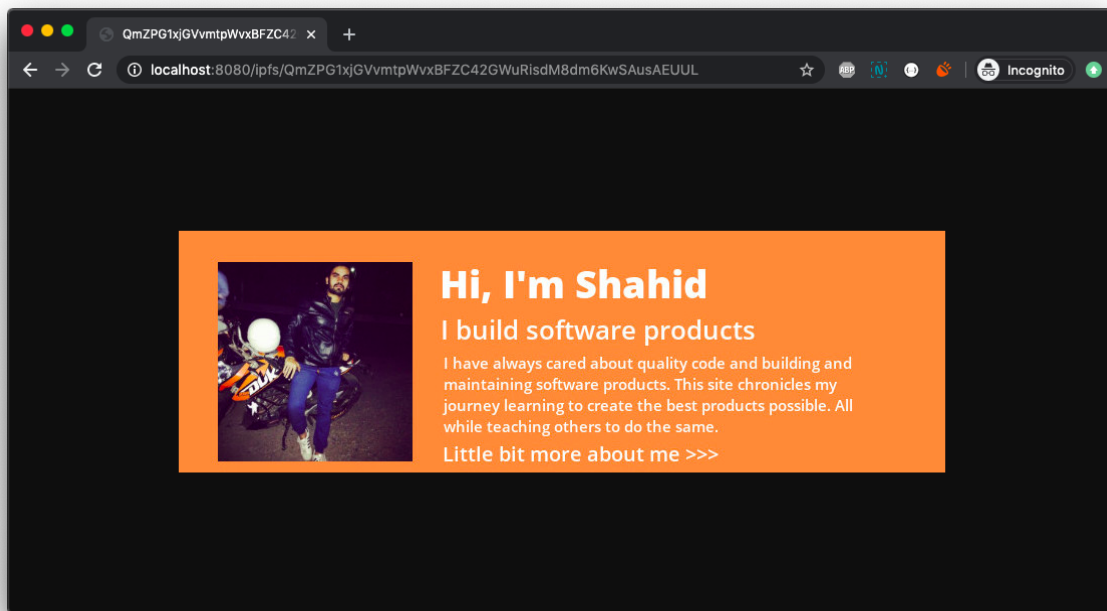


Figure 6.7: *Fetching Objects from IPFS*

You can also access it from anywhere using the global IPFS gateway. Visit the following URL:

`https://ipfs.io/ipfs/QmZPG1xjGVvmtPwvxBFZC42GWuRisdm6KwSAusAEUUL`

10. You can replace the hash with your file hash and share it with anyone in the world. You do not need to become a peer to access the file hosted on IPFS.
11. You can use the following command to access or download the file from the command line:

`$ ipfs get <hash of the file>`

After execution of this command, you should be able to view the downloaded file in the file system.

We will learn more about the command line of IPFS in the next section.

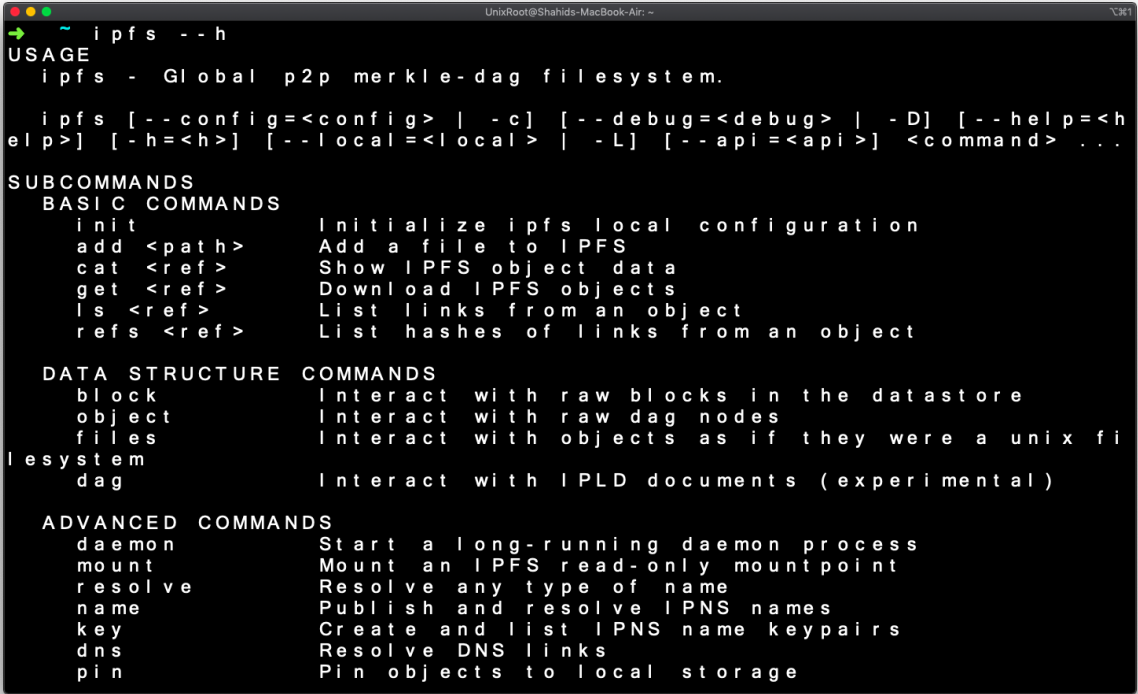
Interplanetary File System command line

IPFS provides a command line utility for developers to perform IPFS actions using a command prompt or Terminal.

To check the list of available IPFS commands, run this in your Terminal:

```
$ ipfs --h
```

It will print all the available commands in your Terminal screen like the following figure:



```
UnixRoot@Shahids-MacBook-Air: ~  
➔ ipfs --h  
USAGE  
  ipfs - Global p2p merkle-dag filesystem.  
  
  ipfs [--config=<config> | -c] [--debug=<debug> | -D] [--help=<help>] [-h=<h>] [--local=<local> | -L] [--api=<api>] <command> ...  
  
SUBCOMMANDS  
  BASIC COMMANDS  
    init          Initialize ipfs local configuration  
    add <path>    Add a file to IPFS  
    cat <ref>     Show IPFS object data  
    get <ref>     Download IPFS objects  
    ls <ref>      List links from an object  
    refs <ref>    List hashes of links from an object  
  
  DATA STRUCTURE COMMANDS  
    block         Interact with raw blocks in the datastore  
    object        Interact with raw dag nodes  
    files         Interact with objects as if they were a unix filesystem  
    dag           Interact with IPLD documents (experimental)  
  
  ADVANCED COMMANDS  
    daemon        Start a long-running daemon process  
    mount         Mount an IPFS read-only mountpoint  
    resolve       Resolve any type of name  
    name          Publish and resolve IPNS names  
    key           Create and list IPNS name keypairs  
    dns           Resolve DNS links  
    pin           Pin objects to local storage
```

Figure 6.8: IPFS help command

There are two types of IPFS commands:

- **Offline:** These commands can be executed without running an IPFS instance.

- **Online:** These commands require an IPFS node running in your system.

You can check all the command details on the IPFS official website: **<https://docs.ipfs.io/reference/api/cli/>**

Now, let us learn how to configure IPFS running in your system or in a cloud server to meet your application requirements.

Configuring Interplanetary File System node

IPFS stores the configuration in a file located at the home folder location of any operating system. The configuration is stored as a **JavaScript Object Notation (JSON)** file. You can view the file at **~/.ipfs/config**.

Open the file in your editor to view the contents. Let us understand the key aspects of the configuration.

Addresses

The following are the default addresses on which IPFS is running and should look like this:

```
"Addresses": {  
  "API": "/ip4/127.0.0.1/tcp/5001",  
  "Announce": [],  
  "Gateway": "/ip4/127.0.0.1/tcp/8080",  
  "NoAnnounce": [],  
  "Swarm": [  
    "/ip4/0.0.0.0/tcp/4001",  
    "/ip6:::/tcp/4001"
```

```
]
}
```

You can configure **API**, gateway addresses. Generally, it is set up to a local host address that can be changed to the custom IP address required. **Swarm** addresses are addresses where local IPFS nodes will listen to connect to other IPFS nodes. API address is where IPFS exposes its API endpoint so that developers can interact with it.

Gateway is the optional address that can be used to view and host static files. You can make this address public if you want others to store files in your IPFS node. Leave it blank if you do not want **Gateway** to boot.

Bootstrap

It is a list of addresses in an array that the IPFS node connects to after startup. IPFS comes with default bootstrap addresses that you can alter. It is suggested otherwise because IPFS, as an organization, maintains various servers to provide a constant bootstrap connection to new peers.

There are other configuration parameters as well, but it is good to leave it default.

Let us host a website in the IPFS and see how it works.

Hosting a static website in IPFS

IPFS is a distributed file system, so we can host a static website directly into the peer-to-peer network of IPFS.

In this section, we will create a website and then host it in the IPFS network. Let us create a simple website.

Our website will look like the following screenshot:

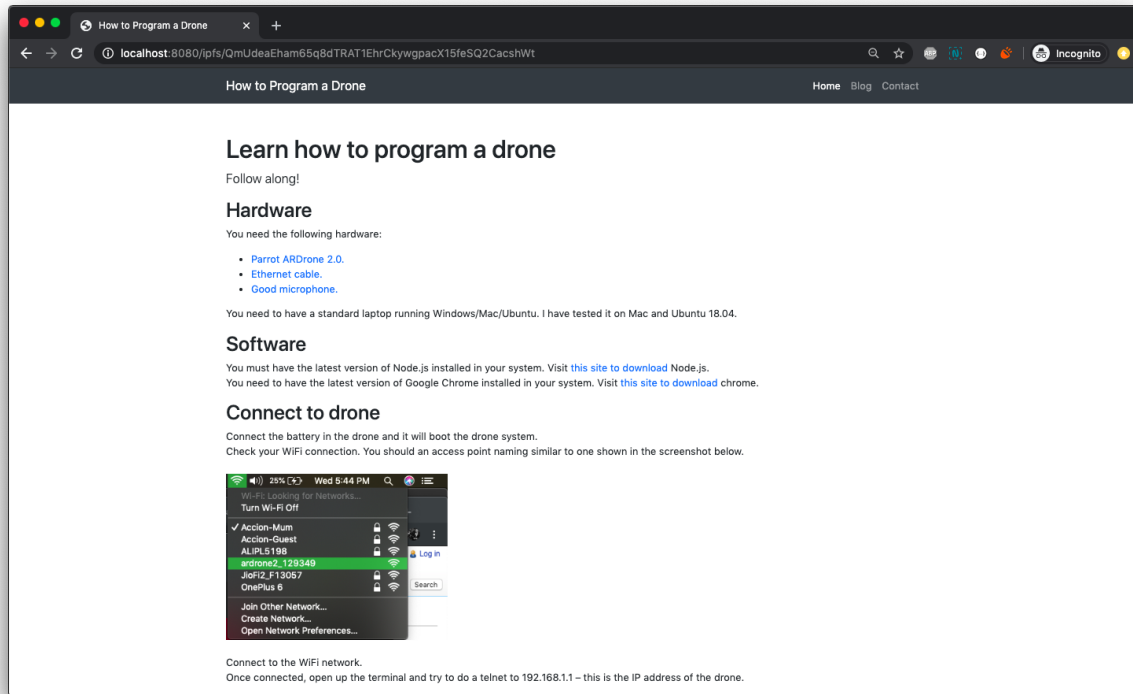


Figure 6.9: IPFS website hosting

You can visit the website using the global IPFS network gateway. Here is the URL:

<http://ipfs.io/ipfs/QmUdeaEham65q8dTRAT1EhrCkywgpacX15feSQ2CacshWt>

Let us check the code of the website. This website contains one page and a few images that we will download from remote resources. You can also include images from the local system itself.

Here is the header of the website HTML:

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
<meta charset="utf-8">
```

```
<meta name="viewport" content="width=device-width,
initial-scale=1, shrink-to-fit=no">
<meta name="description" content="Learn how to
program a drone using Nodejs.">
<meta name="author" content="ShahidShaikh">
<title>How to Program a Drone</title>
<!-- Bootstrap core CSS -->
<link
href="https://stackpath.bootstrapcdn.com/bootstrap/4.3.1/
css/bootstrap.min.css" rel="stylesheet">
</head>
```

As you can see, we have added bootstrap as our **cascading style sheets (CSS)** library to style the content, and we added a few meta tags to handle the view area of the website.

Let us look at the content code of the website:

```
<body>
<!-- Navigation -->
<nav class="navbarnavbar-expand-lgnavbar-dark bg-dark
static-top">
<div class="container">
<a class="navbar-brand" href="/">How to Program a
Drone</a>
<button class="navbar-toggler" type="button" data-
toggle="collapse" data-target="#navbarResponsive" aria-
controls="navbarResponsive" aria-expanded="false" aria-
label="Toggle navigation">
```

```
<span class="navbar-toggler-icon"></span>
</button>
```

Now, we add the code to show some menus for navigation:

```
<div class="collapse navbar-collapse"
id="navbarResponsive">
<ul class="navbar-nav ml-auto">
<li class="nav-item active">
<a class="nav-link" href="/">Home
<span class="sr-only">(current)</span>
</a>
</li>
<li class="nav-item">
<a class="nav-link"
href="https://codeforgeek.com">Blog</a>
</li>
<li class="nav-item">
<a class="nav-link"
href="mailto:shahid@codeforgeek.com">Contact</a>
</li>
</ul>
</div>
</div>
</nav>
```

Next, we need to add some basic content to the body of the HTML page:

```
<!-- Page Content -->
<div class="container">
<div class="row align-items-center">
<div class="col-lg-12">
<h1 class="mt-5">Learn how to program a drone</h1>
<p class="lead">
    Follow along!
</p>
<h2>Hardware</h2>
<p>
    You need the following hardware:
<ul>
<li>
<a href='https://amzn.to/2TCMcvd'>Parrot ARDrone 2.0.
</a>
</li>
<li>
<a href='https://amzn.to/2OvhWBE'>Ethernet cable.</a>
</li>
<li>
<a href='https://amzn.to/2JGka28'>Good microphone.</a>
</li>
</ul>
```


You need to have a standard laptop running Windows/Mac/Ubuntu. I have tested it on Mac and Ubuntu 18.04.

</p>

<h2>Software</h2>

<p>

You must have the latest version of Node.js installed in your system. Visit this site to download Node.js.

You need to have the latest version of Google Chrome installed in your system. Visit this site to download chrome.

</p>

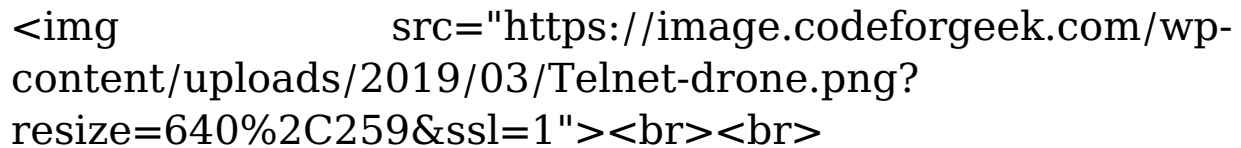
<h2>Connect to drone</h2>

<p>

Connect the battery in the drone and it will boot the drone system.

Check your WiFi connection. You should an access point naming similar to one shown in the screenshot below.

Connect to the WiFi network.
Once connected, open up the terminal and try to do a telnet to 192.168.1.1 - this is the IP address of the drone.

 src="https://image.codeforgeek.com/wp-content/uploads/2019/03/Telnet-drone.png?resize=640%2C259&ssl=1">

</p>

<h2>Programming the drone</h2>

<p>

Let's code our app. First, create a new folder and switch to it using the terminal.

<code>

mkdir drone && cd drone

</code>

Create a new Node project using the following command.

<code>

npm init --y

</code>

Let's install the required dependency.

<code>

npm install --save ar-drone

</code>

Here is the code.

<script

src="https://gist.github.com/codeforgeek/514ee981176e344ea4e871f0407b9fed.js"></script>

</p>

<h2>Running the code</h2>

<p>

Connect to the drone WiFi and run the code using the following command.

<code>node app.js</code>

and see the drone fly and follow the commands as mentioned in the code.

</p>

<p>

Now, we add some basic styling to the page for the users:

Here is the small demo.

<style>.embed-container { position: relative; padding-bottom: 56.25%; height: 0; overflow: hidden; max-width: 100%; } .embed-container iframe, .embed-container object, .embed-container embed { position: absolute; top: 0; left: 0; width: 100%; height: 100%; }</style><div class='embed-container'>

<iframe src='https://www.youtube.com/embed/97yUXCds3tY' frameborder='0' allowfullscreen></iframe></div>

</p>

</div>

</div>

</div>

<!-- Bootstrap core JavaScript -->

<script src="vendor/jquery/jquery.min.js"></script>

```
<script  
src="vendor/bootstrap/js/bootstrap.bundle.min.js">  
</script>  
</body>  
</html>
```

We have added the **jQuery** library in the footer of the page to avoid loading the library during the initial page load. This trick improves the speed of the website.

The preceding code is a bit lengthy, but just go through it and understand the syntaxes. It is basic HTML code with standard tags.

Copy, paste and save the code in an **index.html** file, and place it inside the folder.

Now, open your Terminal, navigate to the folder that contains the code, and run the following this command to host the website in the IPFS:

```
$ ipfs add -r <folder name>
```

The preceding command will add the file present in the folder in a recursive manner. You should receive multiple hashes in your Terminal, and you need to access the **index.html** file, which is an entry point of the website, as illustrated in the following screenshot:

```
UnixRoot@Shahids-MacBook-Air: ~/Desktop/Code/Book/Decentralized
→ Decentralized ipfs add -r howtoprogramdrone
added QmUdeaEham65q8dTRAT1EhrCkywgpacX15feSQ2CacshWt
howtoprogramdrone/index.html
added QmZ9nJxCCaCR2gQZhqtcBdfEAWhULGiY9QvVr8Y34zZgLM
howtoprogramdrone
5.02 KiB / 36.01 KiB [====>-----]
→ Decentralized
```

Figure 6.10: Adding folder in IPFS

If you are adding multiple files of the website, you shall receive multiple hashes. To access the website, use the hash of **index.html**.

This is how you can deploy a static website in the IPFS network. In the next section, we will learn to build and use IPFS in the backend system.

Building a full-stack app using IPFS

In the last section, we understood how to host a static website in the IPFS network. In this section, we will build an application using IPFS.

We will build a file upload system where the user can upload a file in the IPFS network and get the link to view the uploaded file, just like any other web application uploads files. The only major difference is that we will store the file in the IPFS network.

Here is the technology stack that we will use:

- **Backend:** Node.js framework.
- **Frontend:** HTML, CSS, and jQuery framework
- **Storage:** IPFS network

Latest Node.js installed should be in your system. If not, visit the official Node.js website and get the latest version.

Node.js website: <https://nodejs.org/en/download/>

Let us look at the following steps:

1. Create a new directory and navigate to it using the Terminal/Command Prompt.
2. Create a new Node.js project using the following command:

```
$ npm init -y
```

3. Now, let us install the required dependencies that we will use in our project. These dependencies are also called node modules:

```
$ npm install --save express multer ipfs-api
```

4. Now, wait for a few seconds and let node install the dependencies. **Express** is a module that is widely used to build a web application.

Multer is a node module that is used to handle file uploads in the node ecosystem.

ipfs-api is a node module that is an interface for the IPFS gateway. We will use this module to perform the IPFS operation.

5. Now, create a new file, name it **ipfs.js**, and copy paste the following code:

```
const IPFS = require('ipfs-api');

const ipfsOptions = {
  host: 'localhost',
  port: '5001'
};

const ipfs = new IPFS(ipfsOptions);

function addFile(bufferData) {
  return new Promise((resolve, reject) => {
    ipfs.files.add(bufferData, (err, file) => {
      if(err) {
        reject(err);
      }
      resolve(file);
    });
  });
}

module.exports = {
  addFile: addFile,
};
```

In the preceding code, we are first connecting to the IPFS that is running on a local machine on **5001** ports. You can change it if the port and host details are different in your machine.

We have created a different function to add files to the IPFS network. IPFS takes buffers as an input. We are using the **files.add()** function provided by the IPFS module to add the file.

The preceding function returns a promise object so that it can be resolved later. It is the best practice to write code in server-side JavaScript.

6. Next, let us create a server in node. Create a new file called **app.js** and copy paste the following code:

```
const express = require("express");
const multer = require('multer');
const fs = require('fs');
const app = express();
const ipfs = require('./ipfs');
const storage = multer.diskStorage({
  destination: function (req, file, callback) {
    callback(null, './uploads');
  },
  filename: function (req, file, callback) {
    callback(null, file.fieldname + '-' + Date.now());
  }
});
```



```

const upload = multer({ storage :
storage}).single('userPhoto');

app.get('/', (req,res) => {
res.sendFile(__dirname + "/index.html");
});

app.post('/api/photo', (req,res) => {
upload(req,res, async (err) => {
if(err) {
return res.end("Error uploading file.");
}

letfileObject = fs.readFileSync(__dirname + '/' +
req.file.path);

let response = await
ipfs.addFile(Buffer.from(fileObject));

let link =
`http://localhost:8080/ipfs/${response[0].hash}`;

res.end(`File is uploaded - Click <a
href="${link}">here</a> to view the file`);

});

});

app.listen(3000,function(){
console.log("Working on port 3000");
});

```

Let us understand the code:

- a. First, we require the modules in the following code:

```
const express = require("express");
const multer = require('multer');
const fs = require('fs');
const app = express();
const ipfs = require('./ipfs');
```

We also include the code we wrote for the IPFS file upload.

- b. In the next line of code, we are configuring the **multer** module to upload our files in the local system first:

```
const storage = multer.diskStorage({
  destination: function (req, file, callback) {
    callback(null, './uploads');
  },
  filename: function (req, file, callback) {
    callback(null, file.fieldname + '-' + Date.now());
  }
});

const upload = multer({ storage :
storage}).single('userPhoto');
```

In the preceding code, we have added the uploads directory to store our files. We are also configuring **multer** to allow the user to upload one file at a time from the **userPhoto** HTML attribute.

- c. In the next line of code, we are creating a route for our web application. Routes are a way to define the structure of a web application by mapping the URL to the business logic code:

```

app.get('/', (req,res) => {
res.sendFile(__dirname + "/index.html");
});

app.post('/api/photo', (req,res) => {
upload(req,res, async (err) => {
if(err) {
return res.end("Error uploading file.");
}

let fileObject = fs.readFileSync(__dirname + '/' +
req.file.path);

let response = await
ipfs.addFile(Buffer.from(fileObject));

let link =
`http://localhost:8080/ipfs/${response[0].hash}`;
res.end(`File is uploaded - Click <a
href="${link}">here</a> to view the file`);
});
});

```

In the first route, we are delivering the home page to the user. We will look over that code in a moment.

In the next route, we are trying to upload the file in our system. Once the file is uploaded locally, we are reading the file and converting it in the buffer and then adding it into IPFS using the function we wrote earlier.

7. Once IPFS returns us the hash of the file, we are returning it to the user in a hyperlink so that the user

can view it in the browser.

Note: Change this line to your custom IPFS host and port if you have made any changes.

```
let link =  
`http://localhost:8080/ipfs/${response[0].hash}`;
```

8. Now, let us look at the last piece of HTML code. Create a new file and name it **index.html**, and copy paste the following code:

```
<html>  
<head>  
<title>File upload using IPFS</title>  
</head>  
<body>  
<h1>File upload system using IPFS</h1>  
<form id="uploadForm"  
  enctype="multipart/form-data"  
  action="/api/photo"  
  method="post">  
  <input type="file" name="userPhoto" />  
  <input type="submit" value="Upload Image"  
    name="submit"><br />  
  <span id = "status"></span>  
</form>  
</body>
```

```

<script
src="http://ajax.googleapis.com/ajax/libs/jquery/1.7.
1/jquery.min.js"></script>

<script
src="http://cdnjs.cloudflare.com/ajax/libs/jquery.for
m/3.51/jquery.form.min.js"></script>

<script>
$(document).ready(function() {
    $('#uploadForm').submit(function() {
        $('#status').empty().text("File is uploading...");
        $(this).ajaxSubmit({
error: function(xhr) {
status('Error: ' + xhr.status);
        },
success: function(response) {
        $('#status').empty().html(response);
        }
    });
return false;
    });
});
</script>
</html>

```

In the preceding code, we have included **jQuery** and **jQuery form** library from the official **content**

delivery network (CDN).

9. We have created a form that submits the file data in a multipart format to the API we have created in **app.js**:

```
<form id="uploadForm"
enctype="multipart/form-data"
action="/api/photo"
method="post">
<input type="file" name="userPhoto" />
```

Notice the **action** attribute and input name. It must be the same as the API route and **multer** file section attribute.

In the JavaScript code, we are listening to the response from the server and printing the output. We are also using AJAX to upload files to avoid page refresh.

10. It is time to run the code. Run the following command in the Terminal/Command Prompt:

```
$ node app.js
```

Your Terminal should print a message saying:

Working on port 3000

It means the app is running successfully. Navigate to **localhost:3000** to view the application. It should look like this:

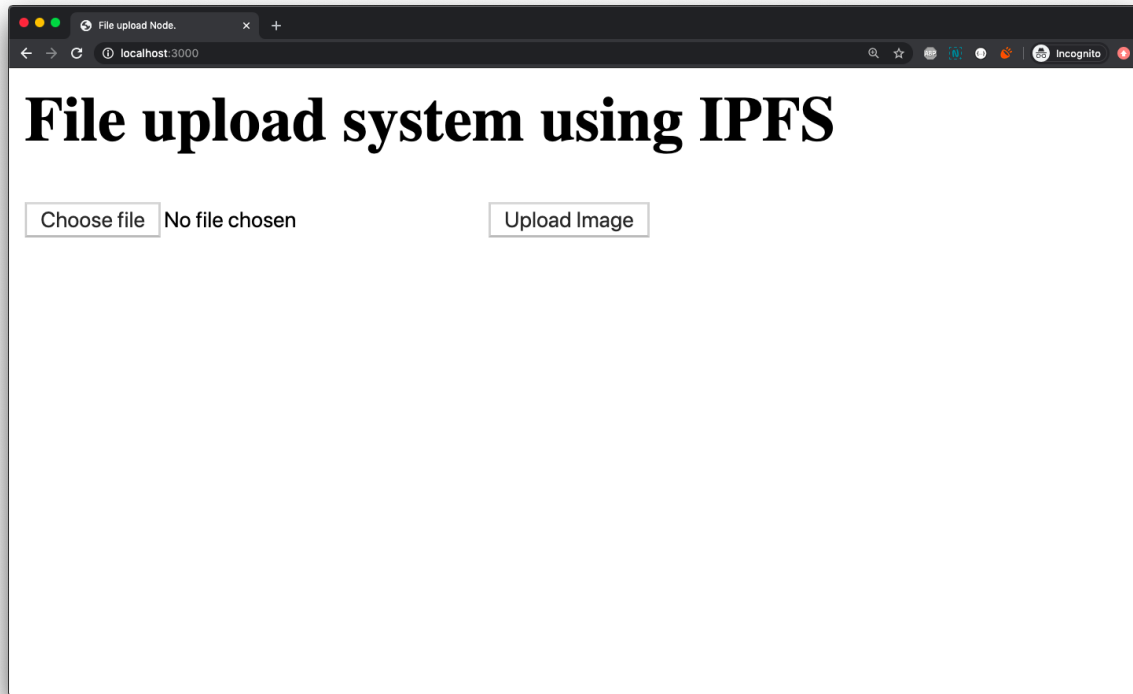


Figure 6.11: Application screen

11. Now, select a file from your system and hit the **Upload Image** button. You should see a response like this:



Figure 6.12: *Uploading files*

12. Now, click on the **here** to view the image in the IPFS network.

You should see an image you have selected from your system. Look at the address bar; it is the IPFS gateway path:

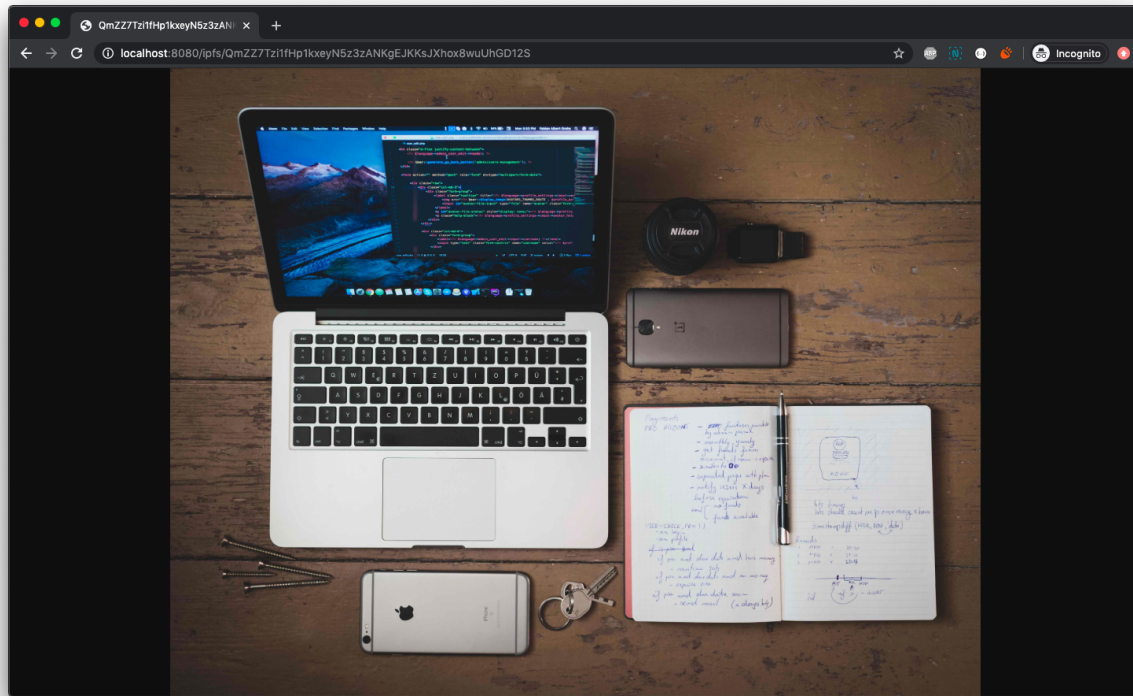


Figure 6.13: Fetching IPFS object

13. You have just built an end-to-end DApp using IPFS as a network and storage.

This app uses the latest technologies such as backend system framework, Node.js, and communicates to the frontend via Ajax.

To deploy this application, you need to deploy the application on a server and connect to the IPFS gateway. You can use the hosted IPFS gateway provided by *Infura*, as discussed in *Chapter 1, Introduction to Blockchain and Decentralized Network*. As of now, we have studied about running single IPFS nodes in our machine, but a single IPFS peer is not enough to maintain data redundancy and availability. This is because, well, failure happens. What if one node fails and data is not replicated to another node? It

will result in data loss, so not the redundant network. To tackle such a scenario, IPFS introduced the IPFS Cluster.

IPFS Cluster

IPFS Cluster is a distributed program that runs alongside IPFS peers and allocates and pins data to other peers to maintain data redundancy and availability.

To understand IPFS Clusters better, let us look at the architecture figure provided by the IPFS Cluster documentation team:

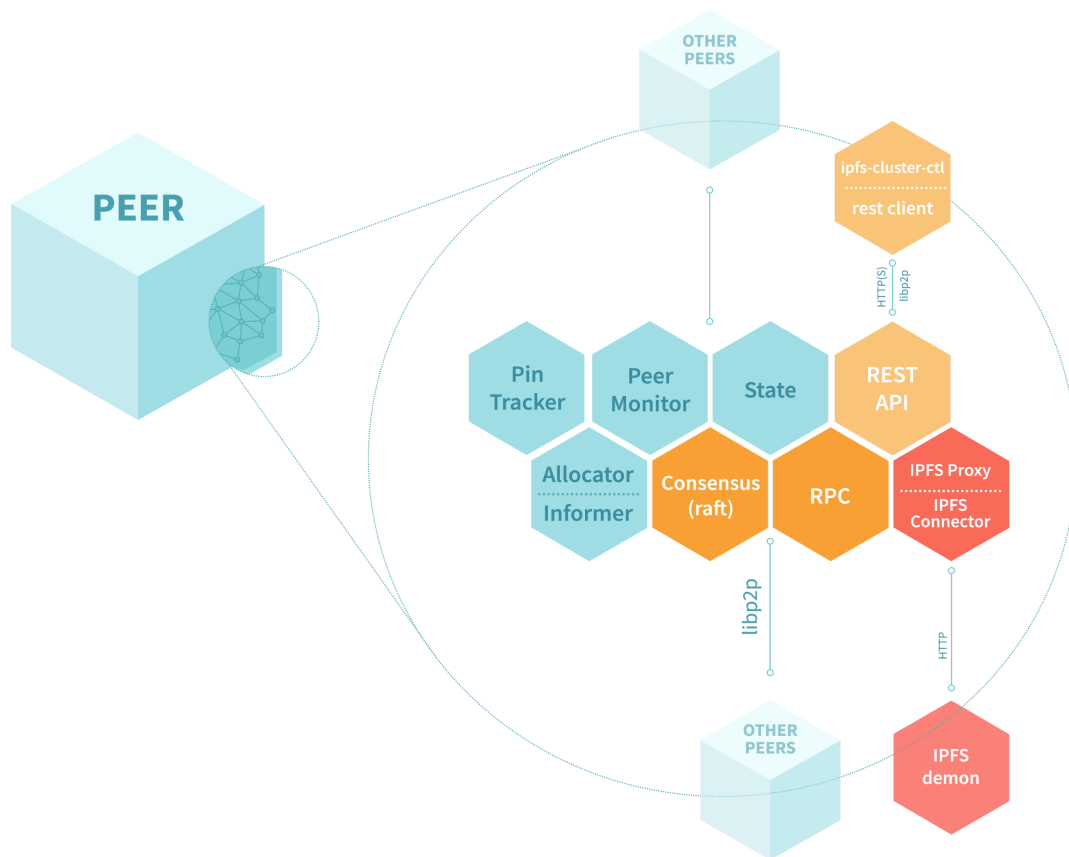


Figure 6.14: IPFS Cluster

As mentioned earlier, the IPFS Cluster program communicates with IPFS peers running in your system. IPFS Cluster communicates using the HTTP API of the IPFS peer, which is running by default on the **5001** port.

IPFS Cluster forms its own network of peers that are running the cluster program and does not interfere with the IPFS network. IPFS Cluster peers maintain global data consensus in order to maintain the data redundancy and availability.

You can check out the IPFS Cluster official project page to download the program and run it in your system.

IPFS Cluster home page: **<https://cluster.ipfs.io/>**

One of the core components of the IPFS and IPFS Cluster is a module called **libp2p**. As we studied in *Chapter 1, Introduction to Blockchain and Decentralized Network*, **libp2p** manages peers and provides a framework to build distributed applications.

Since we have already discussed and learned IPFS in detail, it is the perfect time to explore **libp2p** in depth and understand the core components of IPFS.

libp2p

libp2p is a library, specification, and protocol that helps developers build a peer-to-peer network application.

libp2p consists of the following components:

- **Transport layer protocol:** **libp2p** transport layer protocol is the core of the **libp2p**. This layer ensures data transmission from one peer to another. The transport layer protocol of **libp2p** is evolving every

day, and there will be more protocols in the future that will be supported by the **libp2p**.

- **End-to-end encryption:** **libp2p** transport layers can be configured to use a secure end-to-end encryption using the latest version of TLS, i.e., v1.2. TLSv1.3 is under development.
- **Identity module:** This module ensures that peer is communicating and exchanging data with the correct peer using public-key cryptography to perform an identity check on the data sent and received in the network.
- **Routing:** In the peer-to-peer network, it is essential to have a system to identify and connect to other peers without having to remember the address. Since there is no central system to look for the peer information, **libp2p** provides a routing mechanism to ask peers to provide the network address information, and if they do not have it, ask other peers to fetch the same.
- **Content discovery:** **libp2p** allows us to locate content-based on the hash and uses the distributed hash table to look up content using the hash or CID.
- **Messaging module:** In a peer-to-peer network, peer communication is the key. **libp2p** provides a publish/subscribe, also called a **pub/sub**, communication module to help peers communicate in real-time.

libp2p provides software development kits in Java, C++, Go, and JavaScript. Let us build a simple application using server-side JavaScript using **libp2p**.

We will build a simple application that creates a peer and connects to other peers. We will use the Node.js framework and **js-libp2p**.

Let us create a new node project through the following steps:

1. Create a new folder and navigate to the folder using Command Prompt or Terminal:

```
$ npm init --y
```

2. Install the **libp2p** library and the required dependencies. Run the following command in the Terminal:

```
$ npm install --save libp2p libp2p-websockets libp2p-secio
```

```
$ npm install --save libp2p-mplex libp2p-bootstrap
```

3. Once installed, create a new file and name it **app.js**. Copy pastes the following code:

```
const Libp2p = require('libp2p')
const WebSockets = require('libp2p-websockets')
const SECIO = require('libp2p-secio')
const MPLEX = require('libp2p-mplex')
const Bootstrap = require('libp2p-bootstrap')
// peers addresses
const bootstrapMultiaddrs = [
  '/dns4/ams-
1.bootstrap.libp2p.io/tcp/443/wss/p2p/QmSoLer265
NRgSp2LA3dPaeykiS1J6DifTC88f5uVQKNAd',
  '/dns4/lon-
1.bootstrap.libp2p.io/tcp/443/wss/p2p/QmSoLMeWq
B7YGVlJN3pNLQpmmEk35v6wYtsMGLzSr5QBU3'
]
```

```
async function startNode() {
  const node = await Libp2p.create({
    modules: {
      transport: [WebSockets],
      connEncryption: [SECIO],
      streamMuxer: [MPLEX],
      peerDiscovery: [Bootstrap]
    },
    config: {
      peerDiscovery: {
        autoDial: true,
        [Bootstrap.tag]: {
          enabled: true,
          list: bootstrapMultiaddrs
        }
      }
    }
  })
  node.on('peer:discovery', (peer) => {
    console.log('Discovered %s', peer.id.toB58String())
  })
  node.on('peer:connect', (peer) => {
    console.log('Connected to %s', peer.id.toB58String())
  })
}
```

```
// start libp2p
await node.start()
}
startNode()
```

Let us understand the code:

- a. From *line 1* to *5*, we imported all the required modules with **libp2p**.
- b. Then, we stored two peer addresses that are already running **libp2p** (or IPFS), and we want to connect to those peers.
- c. In the next line, we declared a function to contain all the working code and created a new **libp2p** instance:

```
const node = await Libp2p.create({
  modules: {
    transport: [WebSockets],
    connEncryption: [SECIO],
    streamMuxer: [MPLEX],
    peerDiscovery: [Bootstrap]
  },
  config: {
    peerDiscovery: {
      autoDial: true,
      [Bootstrap.tag]: {
        enabled: true,
        list: bootstrapMultiaddrs
```

```
}  
}  
}  
}))
```

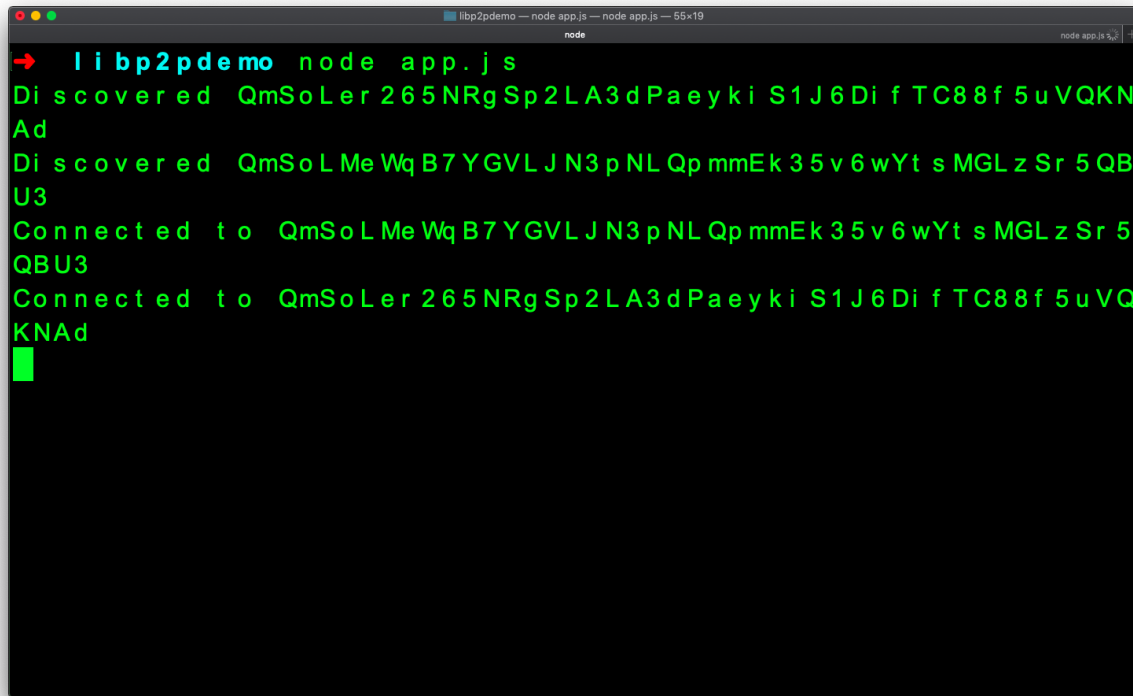
As you can see, we included all the modules we need in the modules object, such as transport, encryption, peer discovery, and so on.

In the config object, we included the peers addresses to connect with.

- d. In the next two lines, we added two event listeners to listen for the events that are fired on pub/sub communication channels.
 - e. In the last line, we started our node.
4. Run the program using the following command to view the output of the code:

\$ node app.js

It should display the following output:

A terminal window titled 'libp2pdemo' with a dark background and light green text. The window shows the command 'node app.js' being executed. The output consists of several lines of Base58-encoded strings, indicating the discovery and connection of peers in a libp2p network. The strings are: 'QmSoLer265NRgSp2LA3dPaeykiS1J6DifTC88f5uVQKNAd', 'QmSoLMeWqB7YGV LJN3pNLQpmmEk35v6wYtsMGLzSr5QBU3', and 'QmSoLer265NRgSp2LA3dPaeykiS1J6DifTC88f5uVQKNAd'. The window also shows a cursor at the end of the last line of output.

```
libp2pdemo node app.js
Discovered QmSoLer265NRgSp2LA3dPaeykiS1J6DifTC88f5uVQKNAd
Discovered QmSoLMeWqB7YGV LJN3pNLQpmmEk35v6wYtsMGLzSr5QBU3
Connected to QmSoLMeWqB7YGV LJN3pNLQpmmEk35v6wYtsMGLzSr5QBU3
Connected to QmSoLer265NRgSp2LA3dPaeykiS1J6DifTC88f5uVQKNAd
```

Figure 6.15: Running application

It means our peer is now connecting to other peers using libp2p code.

Libp2p has other configuration parameters as well. You can check it out at the following link:

<https://github.com/libp2p/js-libp2p/blob/master/doc/CONFIGURATION.md>

Libp2p is the core module of IPFS, and we just studied how to use it as an independent module.

Conclusion

In this chapter, we explored IPFS and its working. Then, we moved to IPFS installation and configuring it in our local system. We also studied the various concepts related to IPFS., and then developed two important applications using IPFS to get a deeper understanding of IPFS and its usage.

Finally, we moved to the libp2p framework to learn the core component of IPFS and its ecosystem.

In the upcoming chapter, we will learn about various decentralized database software.

References

1. <https://ipfs.io/>
2. <https://docs.ipfs.io/>
3. <https://www.cs.cmu.edu/~dga/15-744/S07/lectures/16-dht.pdf>
4. <https://www.cs.umd.edu/~mwh/papers/gpads.pdf>
5. <https://github.com/libp2p/js-libp2p>

Join our book's Discord space

Join the book's Discord Workspace for Latest updates, Offers, Tech happenings around the world, New Release and Sessions with the Authors:

<https://discord.bpbonline.com>



Decentralized Databases

CHAPTER 7

OrbitDB, a Peer-to-peer Distributed Database

Introduction

OrbitDB is a peer-to-peer, serverless, distributed database built on top of IPFS. We have studied about **Interplanetary File System (IPFS)** in detail in the last chapter. In this chapter, we will learn about OrbitDB and how to use it to build **decentralized applications (DApps)**.

Structure

We will cover the following topics in this chapter:

- Introduction to OrbitDB
- Installing OrbitDB
- OrbitDB data models
- Building DApps using OrbitDB
- Technologies

Objectives

After studying this chapter, you will be able to use OrbitDB to build DApps, you will also learn how to use IPFS and OrbitDB together to build DApps. Let us get started.

Introduction to OrbitDB

Here is the definition according to the OrbitDB project page:

OrbitDB is a serverless, distributed, peer-to-peer database. OrbitDB uses IPFS as its data storage and IPFS Pubsub to automatically sync databases with peers. It's an eventually consistent database that uses CRDTs for conflict-free database merges, making OrbitDB an excellent choice for decentralized apps (DApps), blockchain applications and offline-first web applications.

That is a very technical definition. Let us break it down further to understand better:

- OrbitDB is serverless, which means you do not need to maintain a database server when using OrbitDB, unlike other database software. It is provided as a module for various programming languages.
- OrbitDB is distributed; it is designed as a peer-to-peer database with no centralized server.
- OrbitDB uses IPFS as a storage system. OrbitDB acts as a layer between the IPFS and the programming interface.
- OrbitDB is a peer-to-peer database, and to synchronize the changes in the peer to other peers, it uses IPFS **publish/subscribe (pub/sub)** system to communicate with other peers.

- OrbitDB uses the eventual consistency model; it is the consistency model in which the latest updated version of the data item is returned to the user, maintaining the availability and consistency of data. To achieve this eventual consistency, OrbitDB uses **Conflict-free Replicated Data Types (CRDTs)**.
- OrbitDB is packaged as a Node.js library. JavaScript was chosen because of its popularity and ubiquity in the programming community and for its interoperability with the JavaScript implementation of IPFS, called **js-IPFS**.

Let us move on to the installation of OrbitDB.

Installing OrbitDB

OrbitDB is a serverless application, so you do not need to run a server as we do for other databases like MySQL.

Let us install OrbitDB in our application through the following steps:

1. Here is the command to install OrbitDB and IPFS in our project:

npm install @orbitdb/core @orbitdb/liftoff helia

The **liftoff** module will help us quickly spin the OrbitDB database.

2. Once installed, you can use the following code to connect to the IPFS and create an OrbitDB data store:

```
import { createOrbitDB } from '@orbitdb/core'

import { DefaultLibp2pOptions } from '@orbitdb/liftoff'

const libp2p = createLibp2p({
  ...DefaultLibp2pOptions })
```

```
const ipfs = await createHelia({ libp2p })
console.log('Connected to IPFS');
const orbitdb = await createOrbitDB({ ipfs })
console.log('OrbitDB instance created.')
const db1 = await orbitdb.open('db1')
await db1.add('hello world!')
console.log(await db1.all())
```

3. Upon execution of the preceding program, you should get the following output:

```
$ node app.js
```

```
$ Connected to IPFS
```

```
$ OrbitDB instance created.
```

```
$ hello world!
```

4. Now, we can create a new database or load an existing database and use it to build our DApps.

Before we move towards building our application, we should learn the data models offered by OrbitDB.

OrbitDB data models

OrbitDB is a multi-model database and provides various types of databases for different types of data models; here are the following data models:

- Events
- Documents
- Key-value
- Indexed key-value

Let us look at each data model in brief.

Events

It is an immutable and append only data model used to store traversable records. One of the use cases for such types of data models is a message queue where messages are stored in the order and traversed back to the starting index, record by record.

The events model is an immutable data store, so data cannot be altered once it is written.

To create a events store, refer to the following code:

```
const db = await orbitdb.open('users');
const hash = await db.add({ name: 'John' });
// to return all logs
const allRecords = db.iterator({ limit: -1 }).collect().map((e) => e.payload.value);
console.log(allRecords); // prints all the logs
// return only specific record
const record = db.get(hash).map((e) => e.payload.value);
console.log(record); // prints specific log
```

Let us look over the next data model.

Documents

As the name suggests, it is a document-based data model where we can store custom documents and index them using custom fields. If you are familiar with MongoDB or Elasticsearch, this data model will be very useful for you.

You can use the document data model to build custom database applications, blogs, e-commerce applications, and

so on.

We will also use the document data model to build our DApps in the upcoming section.

Let us look at the example code:

```
const db = await orbitdb.open('users',{type: 'documents'});  
// add new record  
  
const hash = await db.put({ _id: 'hash of the record', name:  
'Shahid', followers: 500, following:120, profilePicture: 'hash  
of the image probably IPFS' });  
  
// fetch the record  
  
const profile = db.get('hash of the record');  
  
// delete the record  
  
const hash = await db.del('hash of the record');  
  
// fetch all the record  
  
const allProfiles = db.get('');
```

You can run the custom query to fetch the documents, just like we do in other databases. Check out the following example:

```
const all = db.query((doc) =>doc.followers>= 500);  
// should returns all the users who have followers  
equal to or more than 500
```

The documents data model is quite useful and generic and can be shaped to support any application.

Let us learn about the next data model, key-value.

Key-value

The key-value store, as the name suggests, lets you store data in the key-value format. The usage of the key-value data store is vast and can be used in the range of applications.

Let us look at an example:

```
const db = await orbitdb.open('users', { type: 'key-value' });  
// add new key value data  
const hash = await db.put('1', { name: 'Shahid' });  
// You can also do the above operation using a set()  
function  
const hash = await db.set('1', { name: 'Shahid' });  
// fetch the information using the key  
const value = db.get('1');  
console.log(value); // prints the data i.e {name: 'Shahid'}  
// delete the data using the key  
const hash = await db.del('1');
```

The indexed key-value database is similar to key-value but adds an index to the database for faster queries.

Let us build a DApps using OrbitDB.

Building DApps using OrbitDB

Let us build our full-fledged DApps using OrbitDB and IPFS. We will build an email software that runs on a peer-to-peer network; let us call it **DECENTRALIZED MAIL (Dmail)**.

Before we go into the application's technical aspects, let us see how it looks.

Here is the home page of our application:

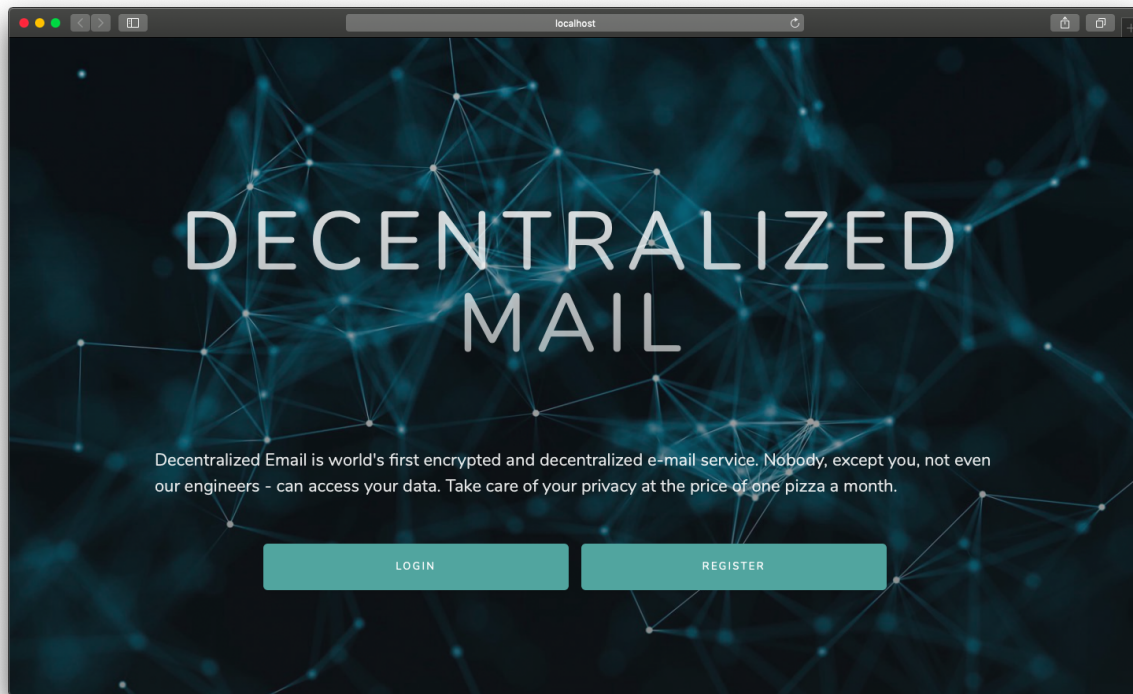


Figure 7.1: DApp homepage

Here is the **Inbox** page where users can read their emails:

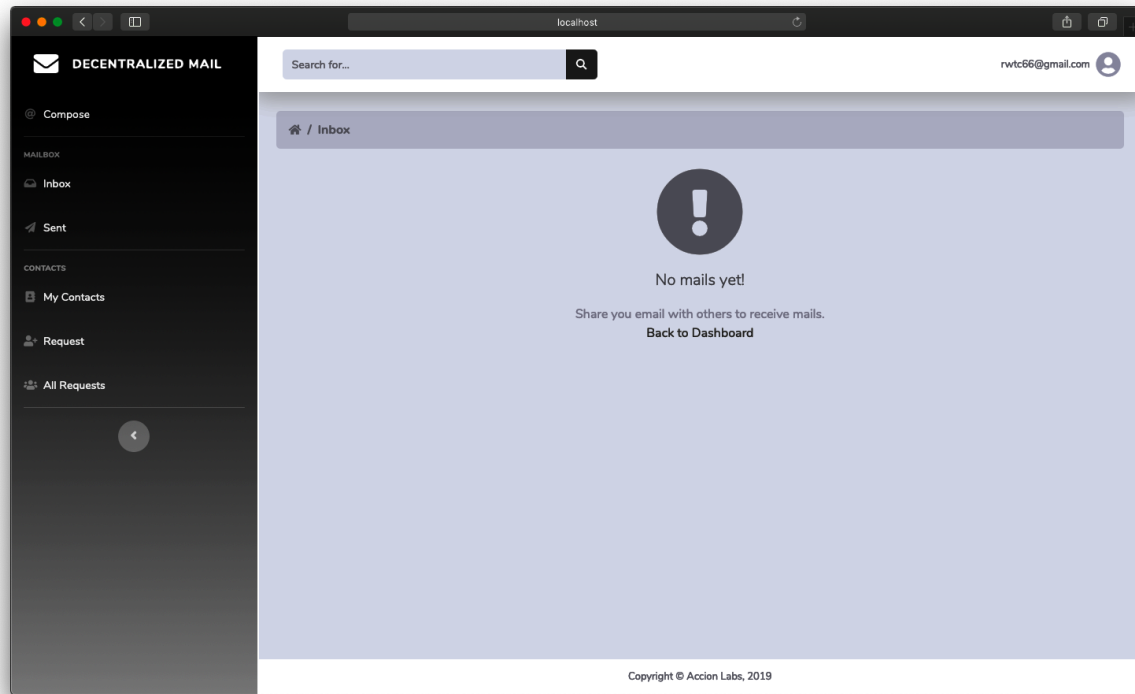


Figure 7.2: Home screen

Users can compose a new email on the following screen, just like with Gmail or any other web-based email application:

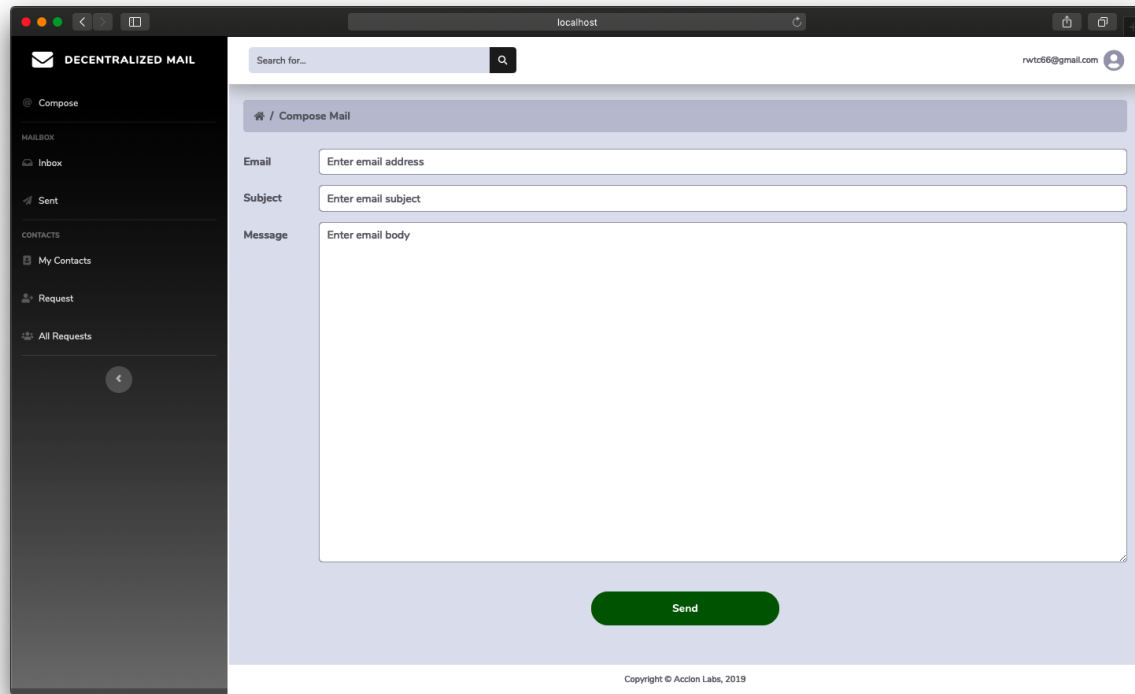


Figure 7.3: *Compose mail screen*

Here, users can check the contacts and add/delete new contacts:

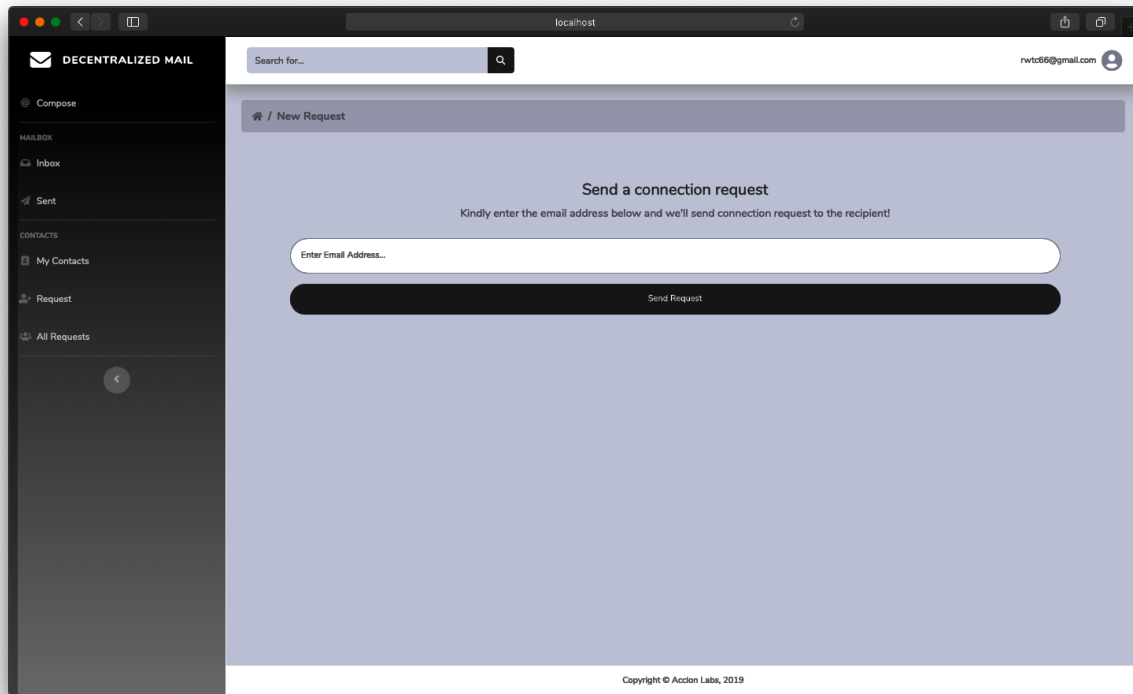


Figure 7.4: Adding contacts

Technologies

Here is our technology stack:

- **Frontend:** HTML, CSS, JavaScript, and jQuery
- **Backend:** OrbitDB, IPFS
- **Middle layer:** Node.js

We are using Node.js to handle the API system so that the frontend can communicate with the database, and the OrbitDB document store data model to store users information and email information.

Let us look over the architecture of the application.

Architecture

We are building a peer-to-peer email application. In this approach, we are sharing the same database across every peer. To do that, we first need to create a database in a genesis peer, and all other peers have to copy the same database.

Once the genesis peer and database are created, another peer can copy the same database and changes will be synchronized using the IPFS pub/sub technique that we studied earlier.

Our email technology is also a little different from traditional email systems. In a traditional system, we use the **Simple Mail Transfer Protocol (SMTP)** to send and receive emails.

In our system, we use the IPFS pub/sub to synchronize changes so that everyone has the same copy of the data, so the emails sent/received will be synchronized.

In a traditional system, you can send emails to anyone if you have their email address. In our system, you can only send emails to users added in your contacts, and whose public key you have.

We encrypt the email using the public key of the receiver and private key of the sender to make it ultra-secure. Only the receipt with the proper private key can decrypt the email and view the content.

It is a very important concept in a peer-to-peer system, where anyone can become a part of the network and view your data. One must ensure that it is secure and not easy to access.

The Node.js server handles our frontend, and the node itself handles the rest of the communication with the

database.

Let us look at the following project directory:

- **package.json**: Contains all the packages required.
- **node_modules**: Folder contains the required modules for the project.
- **orbitdb**: Folder contains the database files.
- **views**: Folder contains the frontend files.
- **utils**: Folder contains the reusable code across the application.
- **dbaddress.js**: File contains the database address.
- **index.js**: Contains the server code.
- **db.js**: Contains the database-related code.
- **package.json**: Contains all the packages required.

Let us understand the code that is coming up next. Since the codebase is large, everything is not pasted here. We will understand the important piece of code and how to run the program. For any doubts about code that is not explained in this chapter, readers can reach out to the writer.

First, let me explain how we are creating a database in a genesis peer and handling the pub/sub between multiple nodes.

Open the **db.js** file and check out the following code.

We are importing the required modules required to handle databases:

```
const IPFS = require('ipfs-api');  
const OrbitDB = require('orbitdb');
```



```
const uuid = require('uuid/v4');
const bcrypt = require('bcrypt');
const fs = require('fs');
const Identities = require('orbit-db-identity-provider');
```

We are creating a function called **loadDB()**, where we add the required code to make the connection with the database and load it in the memory:

```
async function loadDB() {
  .....
}
```

To enable pub/sub so that our nodes can talk to each other, we are passing the following parameters in the IPFS configuration:

```
EXPERIMENTAL: {
  pubsub: true
},
relay: {
  enabled: true,
  hop: {
    enabled: true, active: true
  }
},
```

Then, we are checking whether this node is a genesis node, that is, the first node in the system. We are checking this by loading the **dbaddress.js** file. If this file exists, we are loading the databases written in the file:

```

fs.access(filePath, fs.F_OK, async (err) => {
  if(err) {
    // file does not exist, create a new database and load
    it.
  }
  // file does exists, load the databases
});

```

If the file does not exist, we assume that it is a genesis node. We are then creating the new databases and writing the address of each database in the **dbaddress.js** file:

```

userDb = await orbitdb.create('email.user', 'docstore',{
  accessController: {
    write: ['*']
  }
});

userContactsDb = await
orbitdb.create('email.user.contacts','docstore', {
  accessController: {
    write: ['*']
  }
});

userEmailsDb = await orbitdb.create('email.user.data',
'docstore', {
  accessController: {
    write: ['*']
  }
});

```

```
    }  
  });  
  let fileContents = {  
    "user": userDb.address.toString(),  
    "contacts": userContactsDb.address.toString(),  
    "emails": userEmailsDb.address.toString()  
  }  
  // write the db file  
  fs.writeFileSync(filePath, JSON.stringify(fileContents));
```

If the file exists, we are making the connection to the database using the **open()** function provided by OrbitDB. We pass the database address as a parameter to the function:

```
let fileData = fs.readFileSync(filePath, 'utf-8');  
let config = JSON.parse(fileData);  
console.log('Databases exists, loading them in memory\n');  
userDb = await orbitdb.open(config.user);  
userContactsDb = await orbitdb.open(config.contacts);  
userEmailsDb = await orbitdb.open(config.emails);
```

To load the database in the memory, we are using the **load()** function of OrbitDB:

```
userDb.load();  
userContactsDb.load();  
userEmailsDb.load();
```

Once the database is loaded, we are assigning an event to each database. When all databases are ready, we can proceed ahead. In our codebase, we are adding more event listeners to check the working of the database:

```
userDb.events.on('ready', () => {  
  console.log('user database is ready.')  
});  
  
userContactsDb.events.on('ready', () => {  
  console.log('user contacts database is ready.')  
});  
  
userEmailsDb.events.on('ready', () => {  
  console.log('user emails database is ready.')  
});
```

In the end, we are calling our function when the server starts loading the database in the memory:

```
// load the database  
loadDB();
```

Once the database is created, the **dbaddress.js** file or the address of the database can be shared with other nodes so that they can become a part of the network.

This is how we create and load the OrbitDB database for our Dmail application.

There are other functions in the **db.js** file that deal with various aspects of the application, such as log-in, signup, creating an email, and reading an email. Let us see how we create an email and read an email using a public and private key combination.

First, let us look at the code that generates the public and private key pairs. In the **utils** folder, check the code in **genkey.js**:

```
const crypto = require('asymmetric-crypto');

function generateKeys() {
  let pair = crypto.keyPair();
  return {
    privateKey: pair.secretKey,
    publicKey: pair.publicKey
  }
}

module.exports = generateKeys;
```

We are using the **asymmetric-crypto** node module to create a new pair of public and private keys on a random basis. We can also create a public and private key pair by using the input seeds.

We are using the same module to encrypt and decrypt the email. You can check the code placed in the **utils** folder:

```
const crypto = require('asymmetric-crypto');

function encryptEmail(data, pair) {
  try {
    let encryptedEmail = crypto.encrypt(data,
    pair.publicKey, pair.privateKey);
    return encryptedEmail;
  }
  catch(e) {
```

```

        console.log('error occurred in encrypting email');
        return null;
    }
}
module.exports = encryptEmail;

```

Here, we are using the public and private key pair passed to the function to encrypt the data, which is our email.

We are also performing the decryption of emails using the following code:

```

const crypto = require('asymmetric-crypto');
function decryptEmail(emailData, keys) {
    try {
        let result = crypto.decrypt(emailData.email,
            emailData.signature, keys.senderPublicKey,
            keys.privateKey);
        return result;
    }
    catch(e) {
        console.log(e);
        return null;
    }
}
module.exports = decryptEmail;

```

In the decryption part, we are using the public key associated with the email and receiver private key to

decrypt the email. This way, you can only read the email associated with your public key. This, in turns, makes the data very secure and less prone to interception.

Let us look over the server code. Open the **index.js** file and check out the code.

First, as usual, we are importing all the modules we require:

```
const express = require("express");
const bodyParser = require("body-parser");
const redis = require("redis");
const session = require("express-session");
const redisStore = require("connect-redis")(session);
const cookieParser = require("cookie-parser");
const app = express();
const client = redis.createClient();
const router = express.Router();
const db = require("../db");
```

To make our application secure, we are using sessions to maintain the user data in the browser securely.

To store the session information in the server, we are using Redis, which is a key-value store. The reason why we are using Redis and not using the session store in OrbitDB is because of the peer-to-peer nature.

We do not want to share the session of one user to other peers. If we do that, there is a chance that hackers can hijack the sessions and get crucial information out of it.

Redis runs on each node and maintains the sessions for the users connected to that node only. Here is the code where we are using Redis as a session store:

```
// session middleware
app.use(
  session({
    secret: "some-random-words",
    store: new redisStore({
      host: "localhost",
      port: 6379,
      client: client,
      ttl: 10000
    }),
    saveUninitialized: false,
    resave: false
  });
);
```

Redis runs on the **6379** port by default. If you have changed it, make the change in the code as well.

Once the user performs the log-in operation, we simply set the session for that request using the following code:

```
// set session
req.session.key = {
  userId: response.data.userId,
  email: response.data.email,
```



```
publicKey: response.data.publicKey  
};
```

We are storing the information such as **userId**, **email** address, and **publicKey** to identify the user on each request.

To check whether the session is set for the request, we check whether the session key exists:

```
if (req.session.key) {  
    // user is logged in  
} else {  
    // invalid user, redirect to log-in page  
}
```

All the frontend code is placed in the **view** folder. The codebase is simple; we use plain and simple JavaScript to make the API calls and change the screen.

You can run the app using the following command:

```
$ node index.js
```

You should see the Terminal screen similar to the following screenshot:

```
→ decentralized-email git:(master) ✕ node index.js
Listening on 3000 port
loading the databases
Databases exists, loading them in memory

user contacts database is ready.
user database is ready.
user emails database is ready.
```

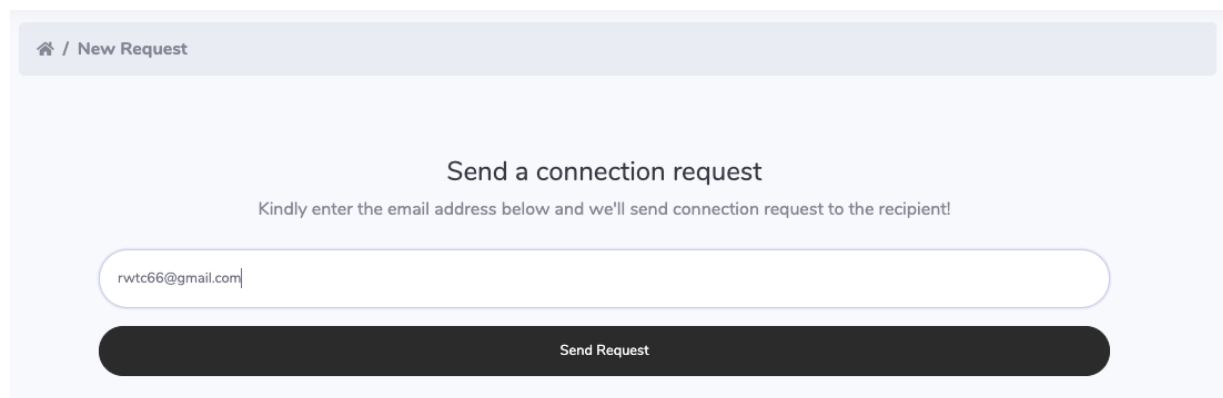
Figure 7.5: Running application

If you are running it the first time, you will get different console messages because of genesis peer operation.

Navigate your browser to **localhost:3000** if you are running it in the local machine or a server address if you are running it on the server.

It would help if your screen were similar to what we have previously shown at the start of this section. An interesting thing about this application is that you can only send and receive an email if you both participate in the contact sharing operation.

Create two users using the signup page and then send a contact request to one of the users:



🏠 / New Request

Send a connection request

Kindly enter the email address below and we'll send connection request to the recipient!

Send Request

Figure 7.6: Connection request screen

Once you send the request, log-in to the second user account and accept the request, as shown in the following figure:

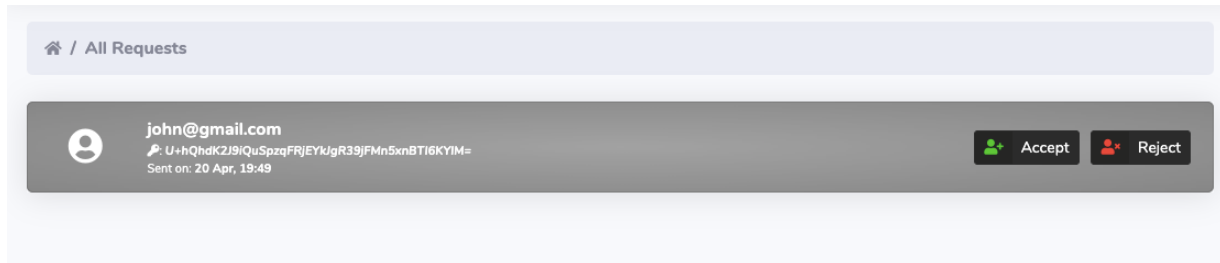


Figure 7.7: View all requests screen

Once you have accepted the request, the mutual agreement has been established. You can send and receive an email between these users.

You can view the user on your **Contacts** page:

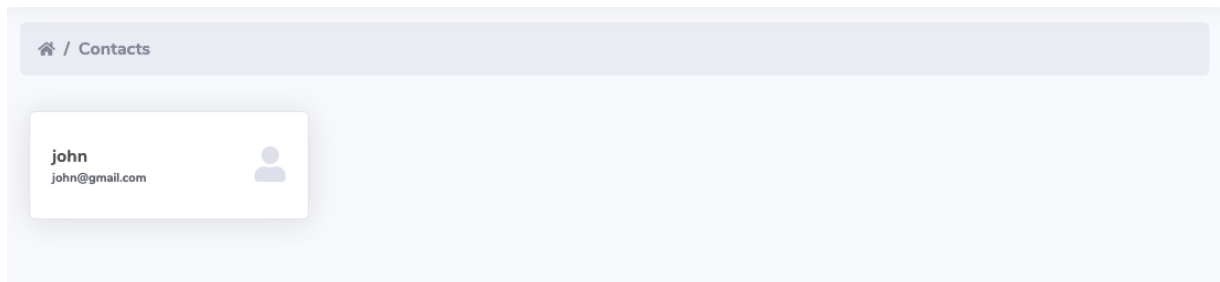


Figure 7.8: View all Contacts screen

Let us send a test email between the users. Compose a new email and send it to the user. You can refer to the following screenshot and see how we are sending it to the user:

Search for...

Q

rwtc66@gmail.com

🏠 / Compose Mail

Email

john@gmail.com

Subject

Hello John

Message

Welcome to our application.

Send

Figure 7.9: Compose mail screen

Once you have sent the email, log-in to the other user account and view the email:

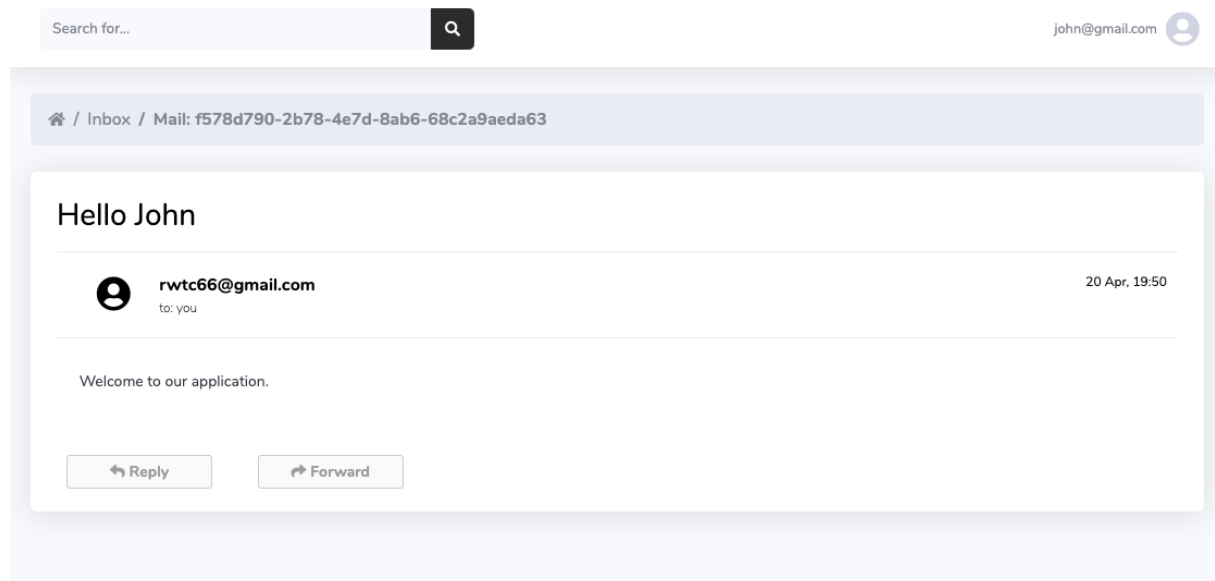


Figure 7.10: View email screen

In the backend, we checked and decrypted the email using the private and public key combination, as we explained earlier.

Conclusion

OrbitDB is one of the key projects in the decentralized application space. The development of this project is crucial for decentralized movement and to make developers transition to the new development stack with a little learning curve. We studied about OrbitDB and the various data models offered by it. We also learned about the OrbitDB installation and how to use it in your project. Moving forward, we built an end-to-end application using OrbitDB as a database and IPFS as a file system.

In the next chapter, we will learn about another popular database that runs on top of the blockchain—BigchainDB.

Join our book's Discord space

Join the book's Discord Workspace for Latest updates, Offers, Tech happenings around the world, New Release and Sessions with the Authors:

[**https://discord.bpbonline.com**](https://discord.bpbonline.com)



CHAPTER 8

BigchainDB

Introduction

In this chapter, we will look at BigchainDB as an open-source database software that provides blockchain features such as decentralization, immutability, and peer-to-peer ownership, and database features such as high transaction rate, low latency, indexing, and querying.

Structure

We will cover the following topics in this chapter:

- Introduction to BigchainDB
- BigchainDB features
- BigchainDB use cases
- Running a BigchainDB node

Objectives

After studying this chapter, you will be able to learn the basics of BigchainDB, how BigchainDB works as a database and the underlying workings of the same. You will also

understand how to run a BigchainDB node. Let us get started.

Introduction to BigchainDB

BigchainDB is a scalable, blockchain-inspired database designed to provide the benefits of decentralized systems—such as immutability, transparency, and decentralized control—while offering the capacity and performance characteristics of traditional databases.

It combines aspects of blockchain technology with distributed database architecture, allowing for **decentralized Applications (DApps)** to be built on top of it.

BigchainDB features

BigchainDB provides blockchain features such as:

- **Decentralization:** No central authority and ownership.
- **Immutability:** Data cannot be altered once written in the BigchainDB.
- **Peer-to-peer network:** BigchainDB provides test and live networks.

BigchainDB provides database features such as:

- **Low latency:** Fast response time due to low latency network calls.
- **Indexing:** Data ingestion into the Blockchain.
- **Querying:** Finding data in the chain.
- **High transaction rate:** Fast in terms of indexing and querying.

You can see the BigchainDB characteristics in the following table:

	Typical blockchain	Typical distributed database	BigchainDB
Decentralization	✓		✓
BFT	✓		✓
Immutability	✓		✓
Ownership of assets	✓		✓
High transaction rate		✓	✓
Low latency		✓	✓
Indexing and querying of structured data		✓	✓

Table 8.1: BigchainDB characteristics

As you can see, BigchainDB is a club of blockchain and distributed databases to provide a solution that is a need of the hour.

Let us go through each of the features of BigchainDB.

Decentralization

BigchainDB is a decentralized database. BigchainDB uses Tendermint to achieve decentralization and reach consensus between various peers. Tendermint is a consensus software, using which we can develop and build secure decentralized software with BFT consensus.

To perform fast indexing and querying, each BigchainDB node runs and manages its own local MongoDB instance. MongoDB is a popular NoSQL database. The communication between nodes is achieved using Tendermint protocols.

BigchainDB is a decentralized database, which means there should be no central point of failure. To achieve that, BigchainDB node is a self-owned and self-operated entity. Since there is no single owner of the network, if one node fails, other nodes will maintain the network.

BigchainDB nodes can be and are located in many geographical places across the globe. The local legal jurisdiction and cloud providers work differently in every country, so an issue with one node or collective nodes in a country of concern (such as *China* or *North Korea*) does not affect them all.

Due to this decentralization, even if a node stops responding to requests and eventually fail, the rest of the network will continue to operate. In fact, up to one-third of

the BigchainDB nodes can fail, and the rest of the network will still work.

Immutability

BigchainDB is an immutable database, which means information cannot be altered or erased once added successfully in a BigchainDB network. At least not without investing great computing/ hashing power, which is very difficult and time and money-consuming. If some data somehow manages to get changed or erased, that is detectable in the network and results in a great level of data integrity and fault tolerance.

BigchainDB does not provide any API that allows us to change or modify the existing data. It is one of the approaches used by BigchainDB to maintain data immutability.

In BigchainDB, every node has a full copy of all the data in a locally managed MongoDB database instance. Hence, there is no single MongoDB database as a point of failure, just like we handle data in Bitcoin or other decentralized systems.

If one or more BigchainDB nodes are corrupted or destroyed due to various reasons, the other nodes would not be affected and will still have a copy of all the data that can be later used to rebuild the corrupted node.

To maintain the integrity of the data up to the standard, all the transactions in BigchainDB are cryptographically signed. After a transaction is stored, changing its data will change the signature, which can be easily detected due to signature mismatch.

Owner-controlled assets

BigchainDB stores and exchanges information in the form of assets. Assets could be anything ranging from currency to house to simple chairs.

BigchainDB has in-built support for owner-controlled assets, that is, only the owner (or owners) of an asset can transfer it to other entities. The owners are typically users with a set of private keys. Owner-controlled assets cannot be transferred by anyone else, not even by the node operator.

There is only one built-in asset in blockchains like Bitcoin or Ethereum, but BigchainDB lets users create as many exclusive assets as they need. Users cannot create assets that appear to be already present in the network, maintaining the ownership of assets and the integrity of the existing data.

BigchainDB always validates every transaction to ensure that it is not trying to transfer a value that was already transferred by another transaction; that is, it prevents double spending.

High transaction rate

BigchainDB provides a high transaction rate. As mentioned in the introduction section, BigchainDB runs on the Tendermint, so the performance stats are very much the same.

Tendermint processes thousands of transactions per second in a distributed network.

Low latency and fast finality

BigchainDB provides low latency; that is, it adds new blocks in the network instantly. BigchainDB claims that they usually take only a few seconds (or less) for a transaction to be included in a new block, which is impressive.

Once a block is added in the BigchainDB network, it is replicated across the peers; once done, there is no way it can be changed or revoked.

Indexing and querying structured data

As mentioned in the introduction section earlier, each node in a BigchainDB network has its own local MongoDB database instance. Each node has access to the full power of MongoDB for indexing and querying the stored data (transactions, assets, metadata, and blocks, all of which are JSON objects).

The data ownership goes to the node owner, and they are free to decide how much of that data they want to expose to external users. For example, one node owner might decide to index geospatial and geographical data and offer optimized geospatial queries via an API. In contrast, another operator might decide to offer a GraphQL API. It is totally up to the node owners.

MongoDB, being a NoSQL database, is very powerful in terms of storing information and querying it.

Let us explore the use cases of BigchainDB.

BigchainDB use cases

BigchainDB is immutable, decentralized database software that can serve a large number of use cases. BigchainDB can

be used wherever there is a need for decentralized, immutable information representing any digital assets.

There are many industry sectors and organizations that can directly benefit from BigchainDB features. In this section, we will cover a few BigchainDB use cases.

Supply chain

In a typical supply chain network, multiple parties are collaborating and exchanging information, goods, and assets.

The supply chain is a huge network of buyers and sellers, and they generate tons of data in the form of invoices, payment records, goods information, and so on. One of the major challenges faced by the supply chain network is the security of information passing through and between various entities involved in the supply chain.

As mentioned in the preceding paragraph about the data generation capability of the supply chain, when the size of the chain grows, data grows exponentially. With data growth, the complexity of the systems grows as well.

To handle the size of the data supply chain generated while maintaining transparency and security of assets, we need a system like blockchain with immutable data stores that cannot be modified by anyone except the owner of the asset.

One of the core concerns with the decentralized database or blockchain technologies is latency. Due to the widespread use of the network, databases running on decentralized systems may take time to respond. That is where BigchainDB shines, bringing along its query capabilities and high throughput performance.

BigchainDB can be used in a supply chain use case to tackle the various issues we discussed, mainly, data management and security. BigchainDB can handle large amounts of data while letting users run quick run-time queries.

Intellectual property rights management

Blockchain proves to be the key solution for intellectual property rights management. For example, consider an artist creating art: Be it musical, or painting, or anything else.

Blockchain technology, in general, has several features that can be used to provide proof to artists claims over their assets. Once an art asset is registered on a blockchain with proper attribution, it can be used to prove the ownership of the IP rights.

One of the common events that occur in the IP is the sale and transfer of the assets from one owner to another based on various agreements. BigchainDB can be used to record assets, and transfer them between users.

Identity

Identity management is one of the most critical use cases when it comes to managing user-specific data, such as credentials, roles, access management, and so on.

Identity theft is a major concern today; we must ensure that the identity of a human or a machine is secure and temper-proof.

Identity management systems generate tons of data. Due to this large-scale data generation, we need a database that

can scale according to the volume of the data.

BigchainDB has characteristics like decentralization, local MongoDB node for fast query and indexing, and immutability that makes BigchainDB a natural fit for identity management systems.

Internet of Things

Internet of Things (IoT) is the digital representation of physical objects that communicate with the network and inject data in the network in a periodic manner. IoT devices can be anything, and their data ingestion can generate petabytes of data. To handle this amount of information, we need a high throughput, fast, tamper-resistant system that can also provide quick results. That is where BigchainDB comes in.

Data governance

Data governance is always a controversial issue. In a democratic country like *India*, *USA*, and *UK*, the data generated by people is always the concern for governments and other agencies due to various reasons. To bring data governance into effect without making people feel that they are being monitored, there is no other solution than the blockchain.

BigchainDB can be used to handle such a large amount of data while allowing the government to watch over assets transfer and take action when necessary.

Audit trails

Audit trails are the records of every action being performed in the system or network of systems. BigchainDB can be

used on audit trails as they are a natural fit.

Audit trails are used in banking, supply chain, utility market, access control, and so on. Due to a large amount of data generation and the nature of the record, BigchainDB can be used to store, manage, query audit trails records.

Let us proceed and set up the BigchainDB node in your local system.

Running a BigchainDB node

BigchainDB requires a MongoDB instance running in your system. So before installing and running BigchainDB, we need to install and configure MongoDB in our system:

1. Visit the official website of MongoDB and download the software according to the operating system you have.
2. Once MongoDB is installed, run the MongoDB daemon process using the following command:

mongod --dbpath=.

You can change the path of the MongoDB **dbpath** variable if you want to store the data at different locations.

3. Once MongoDB is running, we can install the BigchainDB in our system using the following command:

sudo pip3 install bigchaindb

4. After installation, you can run the following command to start the BigchainDB server:

bigchaindb -y configure mongod

5. Start the BigchainDB server:

bigchaindb start

6. Now, navigate to **http://localhost:9984** to verify the BigchainDB installation.

You should get the following response:

```
{
  "api": {
    "v1": {
      "assets": "/api/v1/assets/",
      "docs":
        "https://docs.bigchaindb.com/projects/server/en/v1.2.0/http-client-server-api.html",
      "outputs": "/api/v1/outputs/",
      "statuses": "/api/v1/statuses/",
      "streams":
        "ws://localhost:9985/api/v1/streams/valid_transactions",
      "transactions": "/api/v1/transactions/"
    }
  },
  "docs":
    "https://docs.bigchaindb.com/projects/server/en/v1.2.0/",
  "keyring": [
    "6qHyZew94NMmUTYyHnkZsB8cxJYuRNEiEpXHe1ih9QX3",
    "AdDuyrTyjrDt935YnFu4VBCVDhHtY2Y6rcy7x2TFeiRi"
  ]
}
```

```
],  
  "public_key":  
    "NC8c8rYcAhyKVpx1PCV65CBmyq4YUbLysy3Rqrg8L  
    8mz",  
  "software": "BigchainDB",  
  "version": "1.2.0"  
}
```

BigChainDB is successfully installed in your system.

Conclusion

BigchainDB is a powerful and promising software that can bring the best of both the worlds: Blockchain and traditional databases; which is the need of the hour. We studied the features of BigchainDB and its various use cases. We also studied how to set up a simple BigchainDB node for local development.

In the next chapter, we will study another blockchain-based database called Ties.DB.

Join our book's Discord space

Join the book's Discord Workspace for Latest updates, Offers, Tech happenings around the world, New Release and Sessions with the Authors:

<https://discord.bpbonline.com>



CHAPTER 9

Amazon Quantum Ledger Database

Introduction

Amazon Quantum Ledger Database (Amazon QLDB) is a fully managed *Amazon* ledger database. Amazon QLDB offers central trusted authority ownership to the information present in the ledger. Amazon QLDB provides a transparent, immutable, and cryptographically verifiable transaction log of all your application changes.

Note: This service has now been discontinued.

Structure

In this chapter, we will cover the following topics:

- Amazon Quantum Ledger Database
- Features of Amazon Quantum Ledger Database
- Amazon Quantum Ledger Database use cases
- Using Amazon Quantum Ledger Database

- Amazon Quantum Ledger Database drivers

Objectives

After studying this chapter, you will be able to learn the basics of Amazon QLDB, how to use Amazon QLDB in the **Amazon Web Services (AWS)** console, and how to use drivers to connect to Amazon QLDB.

Let us get started with the introduction.

Amazon Quantum Ledger Database

As stated earlier, Amazon QLDB is a fully managed ledger database. As a developer, you do not have to worry about the installation, configuration, and administration of the database. *Amazon* handles everything for you.

Unlike other decentralized databases we covered in the earlier chapters, Amazon QLDB is centrally owned, which means a single or group of authorities has ownership; hence, it is categorized as a private database.

Amazon QLDB provides an immutable ledger database, so data cannot be modified once written. If a modification is needed, a new record is created, and history is maintained with the old and new records so that one can verify the changes.

Amazon QLDB is cryptographically secure. We can easily verify the records with the SHA-256 hash and signature. In the current chapter, we will learn how to do this.

Amazon QLDB is available in the AWS console as a cloud service. Hence, you pay for what you use.

Amazon uses SQL-like query language to perform operations on the data. It is a welcome move for a large set

of developers who want to use the immutable database with blockchain functionality, but it gets passed on due to the learning curve.

Amazon QLDB also provides drivers for programming languages such as Java, .NET, JavaScript, and Python.

In the next section, we will learn more about the features of Amazon QLDB.

Features of Amazon Quantum Ledger Database

Amazon QLDB provides the following features:

- **Ledger database:** A transaction in a traditional database is generally stored in the database and tables. The database maintains the state of the transaction and takes action in case of an error or rollback situation.

The transaction log is a core component of the **Structured Query Language (SQL)** database systems. It plays an important role in replaying transactions in the event of system failure, disaster recovery, or data replication.

In traditional database systems, transaction logs are not immutable and serve a purpose in terms of transaction failure or rollback.

In Amazon QLDB, the ledger, structurally similar to a transaction log, is the core component of the database. The ledger is an immutable, append-only data structure that stores the transaction data, along with the associated metadata.

The ledger manages data like a SQL database; you can create a table and store your information inside

the table.

- **Immutable:** Amazon QLDB is an immutable database that stores the records of all changes to the data that cannot be altered or modified in any case.

Once the data is committed and the history of the change is recorded in the system, there are no APIs or administration-level features available that can change the historical records.

- **Cryptographically secure:** Amazon QLDB stores data in the blocks that are chained together with a cryptographically hashing function similar to blockchains. This hashing function ensures transactional data integrity using a cryptographic verification method.

For each transactional record, the QLDB provides a digest—a hash value that points to the full hash chain as of a point in time, along with the Merkle audit proof.

- **SQL compatible query language:** Amazon QLDB provides a SQL compatible query language. It shrinks the learning curve of developers. Any database programmer familiar with the SQL can use Amazon QLDB in a short period. Amazon QLDB achieves this using PartiQL as its query language and *Amazon Ion* as its document-oriented data model.

PartiQL is an open-source, SQL compatible query language. You can use PartiQL to insert, query, and manage your data with traditional SQL operators.

You can refer to it on the PartiQL Project homepage (<https://partiql.org/>).

Amazon Ion is similar to JSON, with more *Amazon-centric features*.

- **High availability:** Amazon QLDB is serverless and runs on the cloud, and it is highly available using the battle-tested AWS Global Infrastructure.

Amazon automatically replicates the data across various AWS regions to support the availability and integrity of the data.

- **Enterprise-grade:** Amazon QLDB meets the **atomicity, consistency, isolation, and durability (ACID)** properties. With this, Amazon QLDB is enterprise-grade and can be used with the production systems with almost zero faults.

Amazon Quantum Ledger Database use cases

Amazon QLDB can play a vital role in the following domains:

- **Finance:** Banks and financial institutions run on a ledger to keep track of credit and debit transactions. Instead of building a complex ledger from scratch, we can use Amazon QLDB and focus on the business part instead of handling the technical aspects of the ledger.
- **Manufacturing:** Manufacturing companies work on a supply chain system. Numerous parties are involved in the chain to produce and manufacture a simple item. A ledger database can play a key role in such a system to record transactions and historical records to avoid conflict and keep track of money, goods, and so on in the system.
- **Insurance:** Insurance companies can use the QLDB ledger to perform transparent claim resolution and store the transaction history. Ledger databases can

also store audit records and play a major role in conflict resolution between insurance companies and individuals.

- **Payroll systems:** Payroll systems record the history of transactions such as salaries paid, insurance claims, and performance history. These records can be systematically maintained using QLDB and a ledger, and companies can stay transparent with the ownership of the records with the employees and other stakeholders.
- **Supply chain:** Supply chain management is among the most tedious tasks ever. There has to be a revolution in technology to support the wide audience playing a key role in the chain.

With the QLDB ledger, all the supply chain records can be stored as a history of the data with the proper changes records made by various parties. Parties in the supply chain often access data like the item's location, the shipping details, who handles the shipment, and such. Amazon QLDB can play a major role in this field.

Using Amazon Quantum Ledger Database

To use Amazon QLDB, you need to have an AWS account. Create a new account on AWS if you do not have it already. Perform the following steps:

1. Click on the following link to create a new account in AWS:

<https://aws.amazon.com/account/>

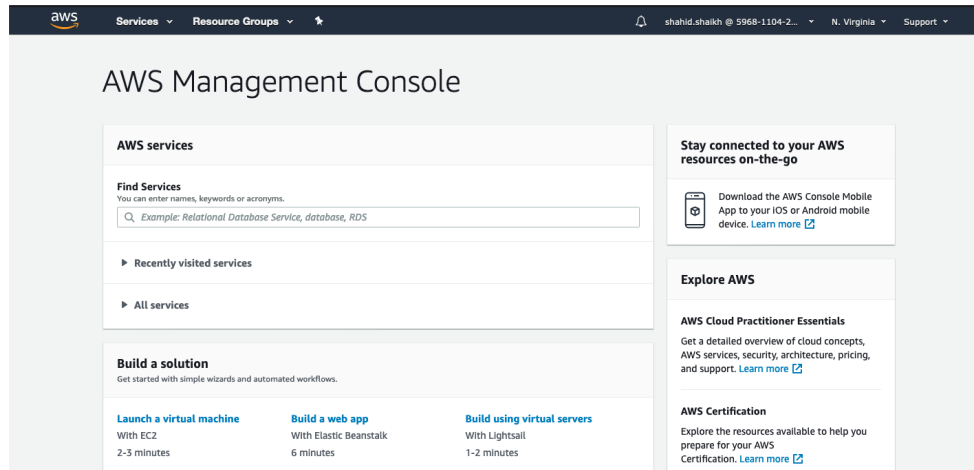


Figure 9.1: AWS screen

2. Search for QLDB and click on it. You should see a screen similar to the one shown here:

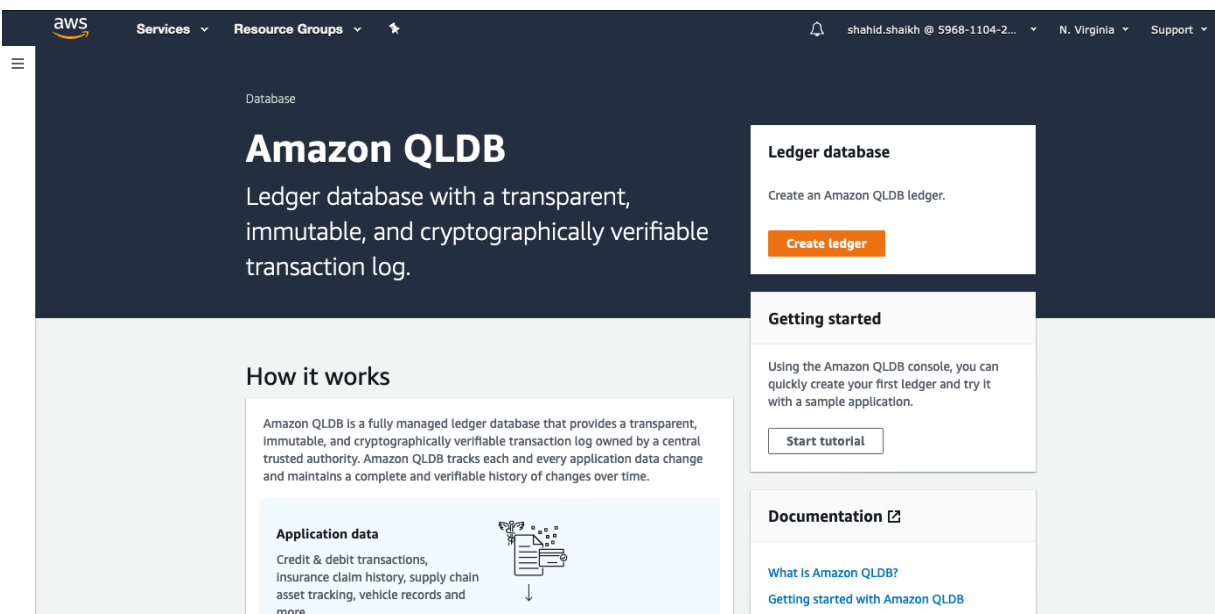


Figure 9.2: QLDB homepage

3. Click on the **Create ledger** button.
4. Provide the name of the ledger and click on the **Create** button. Once the ledger is created, it should be listed in the ledger screen:

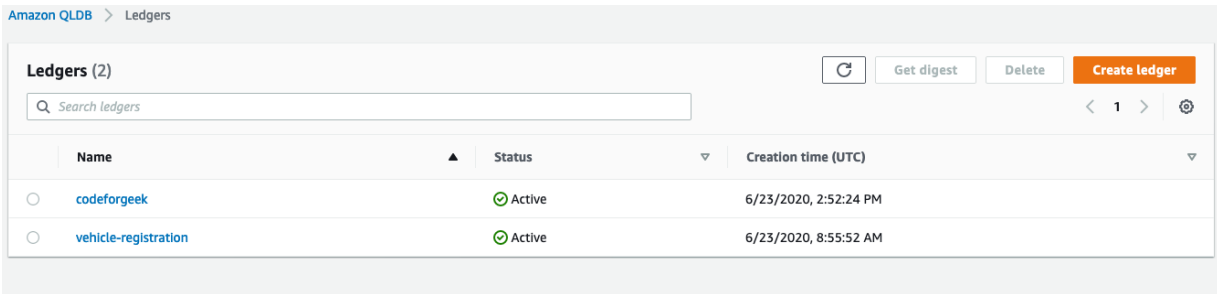


Figure 9.3: View ledger screen

- Click on your ledger name and then click on the **Query editor** from the left menu. It should open a screen similar to the following figure:

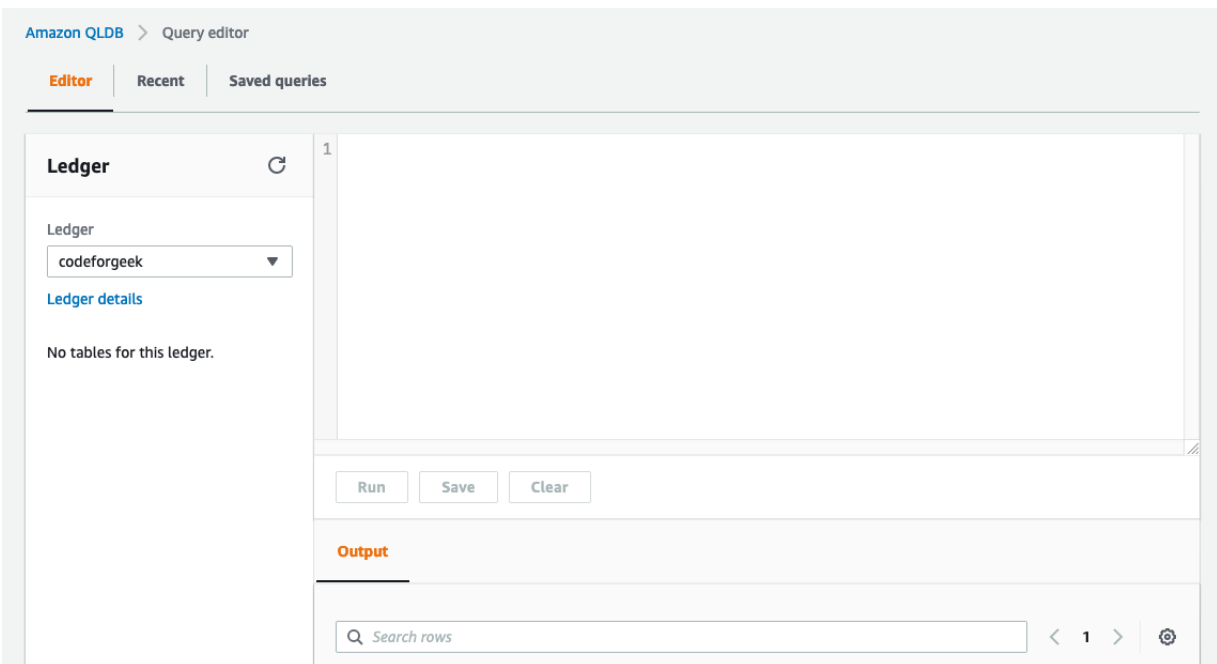


Figure 9.4: Query editor

- Now, run this query to create a new table in the ledger:

CREATE TABLE users

- Paste the query in the query editor and click on the **Run** button. It should create a new table.
- Now, let us add some information to the table. Copy and paste the following query:

INSERT into users VALUE {'id': 1, 'name': 'Shahid', 'location': 'Mumbai'}

9. Now, fetch some information from our table. Run the following query to fetch records:

select * from users

It will return the information present in the table.

10. Let us perform the update query and observe the changes in the QLDB. Run the following query to update the record:

UPDATE users as u SET u.name='ShahidShaikh' where u.id=1

11. Upon the successful query execution, it will return the ID of the document. Copy the ID.

Let us query the changes and see the history of the record through the following steps:

1. To retrieve the history of the record, we use the **history()** syntax. Here is the query:

select * from history(users) as h where h.metadata.id='id from the update query'

This query should return the history of changes made toward the document identified using the ID.

Here is the output:

Rows returned (3)		View as Ion
Search rows		< 1 >
data	metadata	
{id:1,name:"Shahid",location:"Mumbai"}	{id:"LiIKQRLTNpC4vNAej2e2o0",version:0,txTime:2020-06-23T14:58:27.468Z,txId:	
{id:1,location:"Mumbai",name:"Shahid Shaikh"}	{id:"LiIKQRLTNpC4vNAej2e2o0",version:1,txTime:2020-06-23T15:01:22.705Z,txId:	
{id:1,location:"Mumbai",name:"Shahid Shaikh"}	{id:"LiIKQRLTNpC4vNAej2e2o0",version:2,txTime:2020-06-23T15:03:54.157Z,txId:	

Figure9.5: QLDB records

2. Similarly, you can delete the record from the ledger. Copy/paste and run the following query:

DELETE from users as u where u.id=1

Similarly, if you run the history query, you will see the deleted record as well in the history log.

Amazon Quantum Ledger Database drivers

Amazon QLDB provides the drivers for the following programming languages:

- Java
- Python
- Node.js
- .NET

Additionally, you need the *Amazon SDK* to bring the AWS common functionality required for integration with the QLDB.

Let us install the SDK for Node.js:

1. Create a new folder and switch to it using the Terminal:

```
$ mkdir qlldbtest && cd qlldbtest
```

2. Create a package. Json file using the **npm init** command:

```
$ npm init -y
```

3. Install the QLDB driver using the following command:

```
$ npm install amazon-qlldb-driver-nodejs@1.0.0 -save
```

4. Install the dependencies drivers:

```
$ npm install aws-sdk --save
```

```
$ npm install ion-js@4.0.0 --save
```

```
$ npm install jsbi@3.1.2 -save
```

5. Once the driver is installed, we can perform the connection. Refer to the following sample:

```
const qlldb = require('amazon-qlldb-driver-nodejs');  
const main = function() {  
  const driver = new qlldb.QldbDriver('quick-start', {});  
};  
main();
```

We are connecting to the default QLDB ledger that does not require any additional configuration. We have not made a connection to our QLDB ledger database. You can refer to the documentation page for more information about the drivers and visit the following link:

<https://docs.aws.amazon.com/qldb/latest/developerguide/getting-started-driver.html>mentioned

Conclusion

Amazon QLDB is a promising database with an enterprise-grade solution. Amazon QLDB can be used to develop many applications that require a ledger: virtually every major application.

In the next chapter, we will learn about the decentralized application already making it into the mainstream.

Join our book's Discord space

Join the book's Discord Workspace for Latest updates, Offers, Tech happenings around the world, New Release and Sessions with the Authors:

<https://discord.bpbonline.com>



CHAPTER 10

DTube

Introduction

In this chapter, we will learn about DTube, a decentralized version of *YouTube*. It is a video hosting service managed by the network and peers instead of a centralized authority. DTube allows content creators to earn cryptocurrency without showing ads. So, there are no tracking, privacy concerns, and moderations that we have in the existing services.

Structure

In this chapter, we will learn the following:

- Introduction to DTube
- Features of DTube
- DTube architecture
- Workings of DTube

Objective

After reading this chapter, you will learn the basics of DTube and understand the features of DTube. You will also learn the technologies that run DTube, that is, decentralized video hosting service.

Introduction to DTube

DTube is the world's first decentralized video hosting service. It is an ad-free and fair platform for content creators and viewers.

You can visit DTube by clicking on the following DTube website: **<https://d.tube/>**.

The website looks like the following figure:

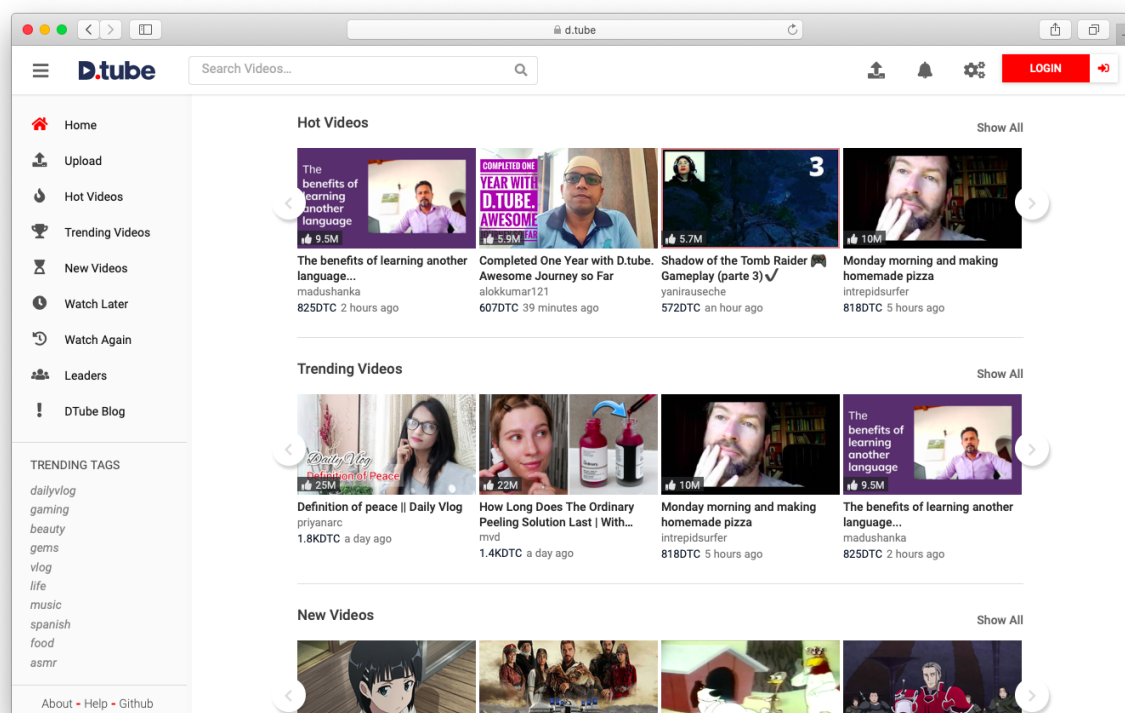


Figure 10.1: DTube homepage

DTube runs on the Steem blockchain and stores content on **Interplanetary File System (IPFS)**. We have studied

IPFS in detail in *Chapter 6, Interplanetary File System*; let us understand the Steem blockchain in a nutshell.

Steem blockchain

Steem is a social blockchain network that allows users to create content and get paid for it without showing ads. Steem mints its cryptocurrency coin and shares it with the content creator and users who engage with the content via upvotes or comments.

It also allows developers to build their applications on top of Steem blockchain.

Click the following link to visit the website and learn more about it: **<https://steem.com/>**.

DTube uses IPFS to store content. It also shows the power of distributed file storage service that can play a major role in applications, like video hosting services.

As a user, you can create an account on DTube and watch videos. You can upvote videos, comment on them, and earn cryptocurrency for that engagement.

Let us check out some features of DTube.

Features of DTube

DTube has the following features for consumers and content creators:

- Content ownership to the content creator.
- Zero moderation and censorship.
- Payment is based on the engagement on the content.
- Payment to content consumers based on engagement.
- Zero transaction fees.

- Decentralized content storage using IPFS.
- Channel creation, subscription, and alerts are similar to *YouTube*.
- Minimal learning curve due to features similarity with *YouTube*.

Now, let us understand the architecture of DTube.

DTube architecture

DTube employs multiple technologies to make the site work, and the architecture consists of the following technologies:

- **Frontend:** JavaScript Meteor framework.
- **Distributed blockchain database:** Avalon
- **Media storage:** Steem and IPFS.

Let us learn, in brief, about the architecture components of DTube:

- **Frontend:** DTube frontend is built on HTML, CSS, and JavaScript. It uses a popular JavaScript framework called **Meteor** to build **single page applications (SPAs)**
- **Distributed blockchain database:** DTube uses a distributed database called **Avalon**. It is a database built for blockchain-based applications, specifically for social networking applications like DTube.
- **Media storage:**
 - DTube uses IPFS and Steem to manage storage. It uses IPFS to store content in the network and Steem to record the engagement on the content created by the user.
 - Let us understand how DTube works.

Workings of DTube

In DTube, you can create an account and watch videos, just like on *YouTube*.

The only difference here is that you do not see ads. As a content consumer, if you upvote a video and become part of the conversation in the comments, you are paid based on how valuable your contribution to the content is.

You can create your channel, upload your content, and get paid based on how good your content is and how many people are engaged in the form of views, votes, and comments.

You might ask where the cryptocurrency is coming from. Well, Steem blockchain mints new Steem coins every 24 hours to distribute to the network.

DTube uses Avalon blockchain to store the content uploaded to the site. Along with Avalon blockchain, it uses IPFS to store static files, such as videos, photos, features, and images.

DTube has its coin called **DTube Coin (DTC)**. Avalon blockchain mints DTC every day to distribute it to the users, and it is called a **reward pool**.

Content creators are paid based on the engagement on the content, and engagement, such as page views, votes, and comments decide the payments in the form of DTC.

In a nutshell, this is how DTube works. DTube is an open-source project, and you can check out the source code at the DTube GitHub: [**https://github.com/dtube**](https://github.com/dtube).

Conclusion

In this chapter, we learned that DTube is an extremely interesting and user-friendly decentralized application. It uses technologies we are already familiar with and provides a safe and content-focused alternative to YouTube.

In the next chapter, we will learn about the Ocean Protocol—a decentralized marketplace to exchange data.

Join our book's Discord space

Join the book's Discord Workspace for Latest updates, Offers, Tech happenings around the world, New Release and Sessions with the Authors:

[**https://discord.bpbonline.com**](https://discord.bpbonline.com)



CHAPTER 11

Ocean Protocol

Introduction

In this chapter, we learn about Ocean Protocol as a decentralized data exchange protocol developed for information exchange by people and to earn cryptocurrency to either sell or consume data. Ocean Protocol developed a decentralized way to buy/sell data with a focus on privacy. We will also look at its use cases and workings.

Structure

In this chapter, we will cover the following topics:

- Introduction to Ocean Protocol
- Use cases of Ocean Protocol
- Workings Ocean Protocol
- Ocean Protocol architecture

Objectives

After reading this chapter, you will be able to understand the basics of Ocean Protocol and learn how Ocean Protocol

works.

Introduction to Ocean Protocol

You probably think of real estate, stocks, oil, and water as the largest assets in the market.

However, the biggest asset in the world right now is data. As the saying goes: *Data is the new oil*.

But who owns data? I guess you already know the answer. It is *Google*, *Facebook*, and other so-called free services that own the data generated by you. They use this data to train their algorithms or, worse, manipulate voters during elections (You can read about the *Cambridge Analytica scandal* in the 2016 elections of *United States*, to get in-depth information).

These large conglomerates do not share this data with parties that need it. For example, a cancer researcher can use such data to perform a predictive analysis. It is almost impossible to find as much data as these companies possess.

Ocean Protocol is developed to tackle this problem. It provides a secure and privacy-focused platform to exchange data with monetary benefits. Data owners can share data by keeping the privacy, and data consumers can access data after paying the owner the requested amount in cryptocurrency.

You can use Ocean Protocol to set up your marketplace that enables users to buy/sell data. Ocean Protocol will manage the security and privacy of the transaction.

Let us learn about the features of the Ocean Protocol.

Use cases of Ocean Protocol

Ocean Protocol can be used in a variety of applications. Here are a few examples:

- **Autonomous vehicles:** Manufacturers of autonomous cars require lots of data to train the artificial intelligence program that drives the car. Organizations assume that they need at least 500 million to 1 billion miles driven records to train the algorithm, so that it can drive safely on roads and make accurate decisions. Ocean Protocol can be used to develop a marketplace where individual drivers can submit their data and get paid for it. Car manufacturers can use the data to train the algorithms.
- **Medical research:** Tons of researchers require data to finish their research. At the time of writing this book, we are dealing with the COVID-19 pandemic, and almost every biotechnology company is on a deadline to develop a vaccine to make humans immune to this virus.

The only way researchers can figure this out is by using data, which can help researchers make effective decisions. Ocean Protocol aims exactly toward situations like this, where people who own data get paid for sharing it privately with researchers.

- **Global datastore:** Ocean Protocol can be used to build a global data store where almost all data related to technology, science, virology, and medicines are available and developers can easily use it to build/train algorithms.

Let us understand how Ocean Protocol works.

Workings of Ocean Protocol

There are three main actors in the protocol. They are as follows:

- **Data providers:** If you own some data and want to sell it, you are a data provider. A data provider is a legal custodian of data. For example, you might be using a fitness watch, or know someone who uses a fitness watch. They use a watch for a specific period, say a month, and that generates numerous data about the body, such as sleep time, heartbeats, and so on.

Now, suppose you want to sell this data. First, you need to generate and upload the data to some service, such as *Google Drive*, *Dropbox*, and so on.

Once you have the free link of the data, you can hook it up with Ocean Protocol and place it in the marketplace. Ocean Protocol never stores your actual data; it stores the metadata of your records and puts it in the marketplace. Ocean Protocol handles the security of the metadata so that there is no breach of privacy.

- **Data marketplace:** Once you are ready to sell the data, so you can list it in the third-party data marketplace built using Ocean Protocol. The data marketplace then lists your data, and it is ready for the consumer.
- **Data consumers:** A data consumer is an entity that uses your data to build their applications, train their algorithms, and so on. Ocean Protocol manages the data and ensures that the consumer has paid the amount listed in the metadata.

Once a customer has paid the amount, they will have access to the data, and the provider will be paid an amount in cryptocurrency.

For detailed information, you can visit the ocean Protocol official website. Link to the official ocean website: <https://oceanprotocol.com/>

Let us understand the ocean Protocol architecture.

Ocean Protocol architecture

The Ocean Protocol architecture diagram, by the Ocean Protocol team, can be seen in the following figure:

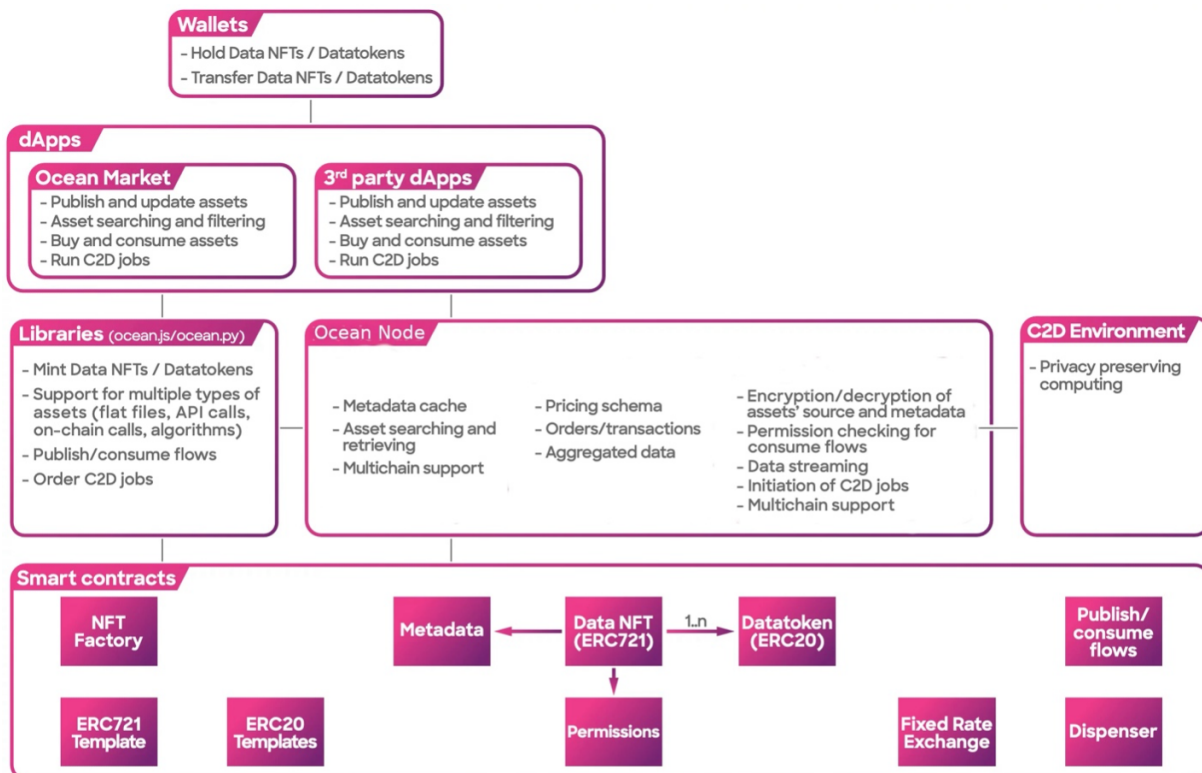


Figure 11.1: Ocean Protocol architecture

Ocean Protocol consists of the following layers:

- **Layer 1:** Foundation blockchain layer
- **Layer 2:** The middle layer

- **Layer 3:** The application layer
- **Layer 4:** Wallets

Let us look at them in detail:

- **Layer 1:** Layer one consists of the robust blockchain layer and Ocean Protocol smart contracts that can be executed on any Ethereum blockchain and other compatible networks.
- **Layer 2:** Layer 2 consists of high-level libraries and protocols to deal with the frontend and smart contracts tiers.

Here are the libraries that are used in the middle layer:

- **Ocean.js:** It is a JavaScript library that interacts with ocean protocol components. **Ocean.js** provides a rich set of functionalities to developers.
- **Ocean.py:** It is a Python library for the Python developer community and data scientists. It also provides a rich set of functionalities.
- **Layer 3:** Layer 3 consists of the ecosystem of DApps, marketplace and applications. The Ocean Market is a hub where data enthusiasts trade the value of the data.
- **Layer 4:** This is the top layer that consists of Web3 Wallets. This acts as a gateway to the Ocean ecosystem.

Conclusion

In this chapter, we learned that Ocean Protocol is a revolutionary software that allows users to own their data and give access to those they trust. It allows users to publish their data and earn incentives once it is consumed.

It also brings back control of the data where it belongs—with you, the user.

Join our book's Discord space

Join the book's Discord Workspace for Latest updates, Offers, Tech happenings around the world, New Release and Sessions with the Authors:

[**https://discord.bpbonline.com**](https://discord.bpbonline.com)



Index

Symbols

51% attack

A

Amazon QLDB

Amazon Quantum Ledger Database (Amazon QLDB)

- drivers

- features

- use cases

- using

Application Blockchain Interface (ABCI)

atomic broadcast

B

BigchainDB

- decentralization

- features

BigchainDB node

running

BigchainDB use cases

audit trails

data governance

identity management

intellectual property rights management

Internet of Things (IoT)

supply chain network

Bitcoin

bits

block

double spending problem

fees

Merkle tree

nonce

transaction data

transactions

TXID

UTXO

weight

working

Bitcoin Lightning Network

benefits

Bitcoin Liquid Network

Bitcoin network

bits

blockchain

- characteristics

- decentralization

- federated blockchain

- history

- private blockchain

- public blockchain

- types

blockchain core components

- block

- chain

- miners

- node

- transaction

blockchain industry

- implications

blockchain scalability

- use cases and applications

blockchain scalability challenges

Byzantine Fault Tolerance (BFT)

Byzantine Generals problem

C

candidate block

cascading style sheets (CSS) library

Channel Module

child chains

Connection Module

consensus

- Byzantine Fault Tolerance

- Proof of Stake

- Proof of Work

consensus algorithms

- Delegated Proof of Stake (DPoS)

- Proof of Stake (PoS)

- Proof of Work (PoW)

consortium blockchain

contract structure

- enum

- events

- function modifier

- functions

- state variables

- struct

cross-chain atomic swaps

cryptocurrency

CryptoKitties

D

DApps

- building, with OrbitDB

- protocols

- working

DApps stack

- frontend

- IPFS

- smart contracts

- storage

- Swarm Bee

DApps, with Ethereum

- advantages

- decentralized approach

- decentralized finance (DeFi)

- decentralized message communications protocols

- decentralized Web

Delegated Proof of Stake (DPoS)

denial of service (DoS) attacks

ÐΞVp2p Wire Protocol

directed acyclic graph (DAG)

distributed hash table (DHT)

DTube

- architecture

- features

- workings

DTube Coin (DTC)

E

ether

Ethereum

- components

DApps

history

Ethereum client

Ethereum Name Service (ENS)

Ethereum networks

F

fault tolerance

fault tolerant algorithms

federated blockchain

Filecoin

G

Ganache

installing

Ganache Remote Procedure Call (RPC) server

Geth

installing

H

Hardhat

Hashcash

hash time locked contracts (HTLCs)

hubs

for interoperability in blockchain

key components

I

IBC modules

- Channel Module
- Connection Module
- Packet Module
- Relayer Module

Infura

- using

Inter-Blockchain Communication (IBC) protocol

- features

- overview

- technical components

- use cases and applications

Interledger Protocol (ILP)

interoperable blockchain

- security considerations

Interplanetary File System (IPFS)

- command line utility

- configuring

- content-based addressing

- default addresses

- default bootstrap addresses

- directed acyclic graph (DAG)

- distributed hash table (DHT)

- features

- full-stack app, building

- installing

- node, configuring
- static website, hosting
- working

IPFS Cluster

IPFS Stack

- data definition and naming layer
- data layer
- networking layer

J

JavaScript Object Notation (JSON)

JSON-RPC API

L

layer 2 scaling solutions

- Lightning Network
- paynet channels
- plasma framework
- rollups
- sidechains
- state channels

LevelDB

libp2p

libpeer-to-peer project

- features

Lightning Network

M

Mainnet

Merkle DAG

Merkle tree

N

nonce

O

Ocean Protocol

- architecture

- use cases

- workings

optimistic rollups

OrbitDB

- DApps, building with
installing

OrbitDB data models

- documents

- events

- key-value

P

Packet Module

packet relay

Parity

PartiQL

payment channel

peer-to-peer email application, with OrbitDB

- architecture
- peer-to-peer (P2P)
- plasma framework
 - components
- Practical Byzantine Fault Tolerance (PBFT)
 - working
- private blockchain
- Proof of Stake
- Proof of Stake algorithm
 - challenges
- Proof of Stake (PoS)
- Proof of Work
- Proof of Work consensus algorithm
 - challenges
- public blockchain

R

- Relayer Module
 - relays
 - for blockchain interoperability
 - use cases and applications
- reward pool
- rollup chain
- rollups
 - optimistic rollups
 - zero-knowledge rollups
- Rootstock (RSK) sidechain

S

SHA-256 algorithm

sharding

- advantages

shards

sidechains

- properties

Simple Mail Transfer Protocol (SMTP)

single page applications (SPAs)

smart contracts

Solidity

- data types

- program, writing

state channels

Steem

Structured Query Language (SQL) database systems

Swarm Bee

synchronization

T

technology stack

Tendermint Core

- limitations

- setting up

Testnet

testnet blockchain

- synchronizing

thereum VMs (EVMs)
transaction IDs (TXIDs)

V

Visual Studio Code (VS Code) editor

W

Waku

Web 3.0

World Wide Web (WWW)

Z

zero-knowledge rollups

Join our book's Discord space

Join the book's Discord Workspace for Latest updates, Offers, Tech happenings around the world, New Release and Sessions with the Authors:

<https://discord.bpbonline.com>

